

UNIVERSIDADE FEDERAL DO PAMPA

Nícolas Naves Rezende Faria

**G-FShield: Seleção Distribuída e Observável de
Características para Detecção de Intrusão em
Sistemas Ciber-Físicos**

Alegrete
2026

Nícolas Naves Rezende Faria

**G-FShield: Seleção Distribuída e Observável de
Características para Detecção de Intrusão em
Sistemas Ciber-Físicos**

Dissertação apresentada ao Programa de
Pós-Graduação em Engenharia de Software
(PPGES) como requisito parcial para a ob-
tenção do título de Mestre em Engenharia de
Software.

Orientador: Prof. Dr. Silvio E. Quincozes

**Alegrete
2026**

Ficha catalográfica provisória

Esta versão da dissertação ainda não possui a ficha catalográfica definitiva inserida.
Substitua este bloco pela versão validada pela biblioteca antes da entrega final.

Nícolas Naves Rezende Faria

**G-FShield: Seleção Distribuída e Observável de
Características para Detecção de Intrusão em
Sistemas Ciber-Físicos**

Dissertação apresentada ao Programa de
Pós-Graduação em Engenharia de Software
(PPGES) como requisito parcial para a ob-
tenção do título de Mestre em Engenharia de
Software.

Dissertação defendida e aprovada em _____ de _____ de _____.
Banca examinadora:

Prof. Dr. Silvio E. Quincozes

Orientador
Unipampa

Membro da banca 1

Examinador(a)
Unipampa

Membro da banca 2

Examinador(a)
Unipampa

RESUMO

A seleção de características reduz a dimensionalidade de dados usados por sistemas de detecção de intrusão, mas implementações sequenciais dificultam distribuir a busca e inspecionar resultados intermediários. Esta dissertação apresenta o *G-FShield*, uma arquitetura distribuída e orientada a eventos para GRASP-FS aplicado a sistemas ciber-físicos. A proposta separa geração da lista restrita de candidatos e busca local, usa Apache Kafka para comunicação e persiste eventos, métricas, estados, identificadores e marcas temporais consultados por uma camada web.

Uma campanha reproduzível, limitada a dez dias, avaliou 24 configurações G-FShield, dois monólitos e um baseline com todas as 51 características. Cada braço teve oito sementes independentes, o mesmo particionamento estratificado de treino/validação/teste, Weka J48 3.8.6 como avaliador final e teto agregado de oito CPUs e 16 GiB. Foram obtidas 216 execuções válidas. A configuração ReliefF-VND-IWSSR, escolhida pela validação, alcançou F1 macro mediano de 0,9448 no teste com 11 características, redução dimensional de 78,4%; o baseline obteve 0,9378. Nenhuma das 72 comparações exploratórias ajustadas por Holm apresentou $p < 0,05$.

Uma segunda campanha causal pareou 30 sementes sob a mesma sequência RF-VND-IWSSR, função objetivo, dados, prazo e teto de seis CPUs e 12 GiB. O G-FShield avaliou 0,1074 candidato/s contra 0,0739 do monólito e sobrepôs praticamente toda a busca local à construção, mas demorou mais para atingir F1 macro de validação 0,94 e consumiu mais recursos por candidato. Uma terceira campanha, pré-especificada com 30 novas sementes, confirmou vantagem em rendimento de soluções: a diferença média pareada foi de +0,933 subconjunto distinto com F1 de validação $\geq 0,945$ por execução (IC95%: 0,233–1,667; $p = 0,010135$), com a salvaguarda de não inferioridade no teste satisfeita. A vantagem demonstrada é, portanto, maior paralelismo, vazão e rendimento de soluções qualificadas sob prazo e teto comuns, não menor latência ou eficiência por recurso. Escala horizontal, tolerância a falhas, recuperação e generalização ainda requerem experimentos específicos.

Palavras-chave: seleção de características, detecção de intrusão, sistemas ciber-físicos, GRASP-FS, sistemas distribuídos, observabilidade.

ABSTRACT

Feature selection reduces the dimensionality of data used by intrusion detection systems, but sequential implementations make it difficult to distribute search and inspect intermediate results. This dissertation introduces *G-FShield*, a distributed, event-driven architecture for GRASP-FS applied to cyber-physical systems. It separates restricted-candidate-list generation from local search, uses Apache Kafka for communication, and persists events, metrics, states, identifiers, and timestamps queried through a web layer. A reproducible campaign, bounded to ten days, evaluated 24 G-FShield configurations, two monoliths, and an all-51-feature baseline. Each arm used eight independent seeds, the same stratified training/validation/test split, Weka J48 3.8.6 as the final evaluator, and an aggregate ceiling of eight CPUs and 16 GiB. The campaign produced 216 valid runs. The validation-selected ReliefF–VND–IWSSR configuration achieved median test macro-F1 0.9448 with 11 features, a 78.4% dimensionality reduction; the baseline achieved 0.9378. None of the 72 exploratory Holm-adjusted comparisons had $p < 0.05$.

A second causal campaign paired 30 seeds under the same RF–VND–IWSSR sequence, objective, data, deadline, and ceiling of six CPUs and 12 GiB. G-FShield evaluated 0.1074 candidate/s versus 0.0739 for the monolith and overlapped virtually all local search with construction, but took longer to reach validation macro-F1 0.94 and consumed more resources per candidate. A third, prespecified campaign with 30 new seeds confirmed an advantage in solution yield: the paired mean difference was +0.933 distinct subsets with validation F1 ≥ 0.945 per run (95% CI: 0.233–1.667; $p = 0.010135$), while satisfying the held-out noninferiority guardrail. The demonstrated advantage is therefore greater parallelism, throughput, and qualified-solution yield under a common deadline and ceiling, not lower latency or resource efficiency. Horizontal scaling, fault tolerance, recovery, and generalization still require dedicated experiments.

Keywords: feature selection, intrusion detection, cyber-physical systems, GRASP-FS, distributed systems, observability.

LISTA DE FIGURAS

Figura 1 – Resumo da arquitetura do <i>G-FShield</i> ; a margem em branco do arquivo original foi removida somente na inclusão LaTeX.	21
Figura 2 – Arquitetura genérica da etapa Distributed RCL Generation (DRG). . .	25
Figura 3 – Instanciação da etapa Distributed RCL Generation (DRG) com as quatro métricas implementadas.	26
Figura 4 – Arquitetura genérica da etapa Distributed Local Search (DLS). . . .	29
Figura 5 – Instanciação da etapa Distributed Local Search (DLS) com os controladores e operadores implementados.	30
Figura 6 – Topologia de implantação e fronteiras de execução do <i>G-FShield</i>	33
Figura 7 – Ciclo de execução do <i>G-FShield</i> , da solicitação aos estados apresentados na interface.	36
Figura 8 – Pipeline de ingestão, persistência e apresentação da telemetria observada.	37
Figura 9 – Fluxo funcional do cliente web e de suas consultas à Application Programming Interface (API).	38
Figura 10 – Serviços do backend, armazenamentos e integração com a mensageria. .	39
Figura 11 – Distribuição do F1 macro no teste para os 27 braços e oito sementes por braço.	48
Figura 12 – F1 macro mediano no teste das 24 configurações distribuídas.	49
Figura 13 – Compromisso entre F1 macro mediano e redução dimensional das soluções finais.	51
Figura 14 – F1 mediano por classe no teste para a configuração distribuída escolhida na validação e os comparadores.	53
Figura 15 – CPU e memória observadas por arquitetura sob o teto agregado comum de oito CPUs e 16 GiB.	55
Figura 16 – Comparações pareadas de tempo até F1 macro 0,94, vazão de candidatos, CPU utilizada e sobreposição entre construção e busca local. . .	59
Figura 17 – Trajetória mediana da melhor solução na validação durante os 45 minutos de seleção. As faixas representam o intervalo interquartil entre sementes.	61
Figura 18 – Contagem pareada de subconjuntos distintos com F1 macro de validação $\geq 0,945$ em 2.700 s, nas 30 sementes confirmatórias.	63
Figura 19 – F1 macro dos subconjuntos congelados sob o protocolo XGBoost comum. Traços horizontais mostram os comparadores fixos com todos os atributos e com os atributos do log histórico IWSHAP.	69
Figura 20 – Tela de autenticação.	80
Figura 21 – Painel de visão geral.	81
Figura 22 – Tela de performance.	82
Figura 23 – Tela de comparação de algoritmos.	83

Figura 24 – Atividade analítica agregada.	84
Figura 25 – Detalhe da atividade analítica.	85
Figura 26 – Tópicos analíticos.	86
Figura 27 – Feed analítico persistido.	87
Figura 28 – Tela de comparação de execuções.	88
Figura 29 – Comparação com sementes selecionadas.	89
Figura 30 – Detalhe de uma execução específica.	90
Figura 31 – Soluções iniciais.	91
Figura 32 – Resultados consolidados.	92
Figura 33 – Progresso da execução.	93
Figura 34 – Melhores soluções.	94
Figura 35 – Configuração de execução.	95
Figura 36 – Configuração de conjuntos de dados.	96
Figura 37 – Configuração de operações.	97
Figura 38 – Diálogo de reset completo.	98
Figura 39 – Listagem de datasets.	99
Figura 40 – Listagem de usuários.	100
Figura 41 – Edição de usuário.	101
Figura 42 – Poster do vídeo demonstrativo da plataforma.	102
Figura 43 – Documentação Swagger da API do backend.	103
Figura 44 – Diagrama do funcionamento do front-end.	104
Figura 45 – Diagrama da arquitetura do backend.	105
Figura 46 – Versão genérica do DRG com entrada de dados abstrata.	106
Figura 47 – Resumo visual da arquitetura completa do sistema.	107

LISTA DE TABELAS

Tabela 1 – Síntese dos trabalhos relacionados citados no manuscrito.	18
Tabela 2 – Drivers, decisões e limites da arquitetura.	23
Tabela 3 – Recursos e concorrência da configuração local de referência.	40
Tabela 4 – Limites efetivos no servidor inspecionado.	41
Tabela 5 – Resultados finais no teste reservado e recursos observados (medianas de oito execuções).	46
Tabela 6 – Campanha causal: medianas por arquitetura e comparações pareadas em 30 sementes.	57
Tabela 7 – Replicação arquitetural no X-CANIDS: medianas de 30 sementes por arquitetura e cenário.	65
Tabela 8 – Subconjuntos avaliados pelo mesmo XGBoost e no mesmo teste intocado. Valores de G-FShield e monólito são medianas de 30 sementes; comparadores fixos têm uma execução.	67

LISTA DE ABREVIATURAS E SIGLAS

- API – Application Programming Interface
- CPS – Cyber-Physical System
- DLS – Distributed Local Search
- DRG – Distributed RCL Generation
- GR – Gain Ratio
- GRASP-FS – Greedy Randomized Adaptive Search Procedure for Feature Selection
- IDS – Intrusion Detection System
- IEC 61850 – IEC 61850
- IG – Information Gain
- IWSS – Incremental Wrapper Subset Selection
- IWSSr – Incremental Wrapper Subset Selection with randomization
- Kafka – Apache Kafka
- RVND – Randomized Variable Neighborhood Descent
- SU – Symmetrical Uncertainty
- VND – Variable Neighborhood Descent

SUMÁRIO

1	INTRODUÇÃO	12
2	FUNDAMENTAÇÃO TEÓRICA	14
2.1	Sistemas Ciber-Físicos e Detecção de Intrusão	14
2.2	Seleção de Características	14
2.3	Greedy Randomized Adaptive Search Procedure for Feature Selection (GRASP-FS)	15
2.4	Distribuição e Orquestração	15
2.5	Observabilidade	15
2.6	Síntese	15
3	TRABALHOS RELACIONADOS	16
3.1	Detecção de Intrusão em Sistemas Ciber-Físicos	16
3.2	Seleção de Características e GRASP-FS	16
3.3	Arquiteturas Distribuídas e Observabilidade	17
3.4	Lacuna Identificada	17
3.5	Tabela Comparativa	17
4	DESENVOLVIMENTO	20
4.1	Visão Geral e Contrato de Entrada	20
4.2	Drivers e Decisões Arquiteturais	22
4.3	Geração Distribuída da Lista Restrita de Candidatos	23
4.4	Busca Local Distribuída	28
4.5	Orquestração por Eventos	32
4.6	Telemetria Persistida e Camada Web	34
4.7	Configuração de Referência e Reprodutibilidade	40
4.7.1	Manifesto da campanha formal	42
4.8	Síntese	43
5	AValiação	44
5.1	Protocolo Experimental	44
5.2	Qualidade Preditiva e Redução Dimensional	45
5.3	Tempo, Recursos e Interpretação Estatística	54
5.4	Experimento Causal Pareado da Arquitetura	56
5.5	Confirmação Independente do Rendimento de Qualidade	62
5.6	Replicação no X-CANIDS e Comparação com o IWSHAP	64
5.7	Observabilidade e Rastreabilidade	70
5.8	Ameaças à Validade	70

6	CONCLUSÃO	72
	REFERÊNCIAS	75
	APÊNDICES	79
	APÊNDICE A – GALERIA DO FRONT-END	80
A.1	Acesso e Visão Geral	80
A.2	Performance e Algoritmos	82
A.3	Analytics	84
A.4	Comparação de Execuções	88
A.5	Linha do Tempo e Resultados	91
A.6	Configurações e Administração	95
	APÊNDICE B – VÍDEO E DOCUMENTAÇÃO	102
	APÊNDICE C – DIAGRAMAS DE ARQUITETURA	104

1 INTRODUÇÃO

Sistemas ciber-físicos (CPS) conectam software, redes e processos físicos em ambientes nos quais desempenho e confiabilidade precisam coexistir. Em infraestruturas críticas, uma falha de segurança pode afetar sistemas computacionais, ativos físicos e continuidade de serviço (CÁRDENAS; AMIN; SASSTRY, 2008; MITCHELL; CHEN, 2014; ALCARAZ; LOPEZ, 2010). Sistemas de detecção de intrusão (IDS) precisam, portanto, lidar com dados de alta dimensionalidade, restrições de tempo e necessidade de rastreabilidade operacional.

A seleção de características reduz a dimensionalidade e a redundância antes da classificação (HALL, 1999; SAEYS; INZA; LARRAÑAGA, 2007; GUYON; ELISSEEFF, 2003). O GRASP original combina construção gulosa-randômica e busca local (RESENDE; RIBEIRO, 2016). A adaptação GRASP-FS para seleção de características é tratada por Quincozes et al. (2020), Quincozes et al. (2022b), Quincozes et al. (2022c). Nas implementações monolíticas estudadas, ranking, construção, refinamento e registro da execução permanecem no mesmo processo, o que restringe a separação de responsabilidades e a inspeção de etapas.

Esta dissertação propõe o *G-FShield*, uma arquitetura distribuída e orientada a eventos para GRASP-FS em Intrusion Detection Systems (IDS) aplicados a Cyber-Physical Systems (CPS). A Distributed RCL Generation (DRG) calcula quatro rankings independentes e constrói soluções iniciais; a Distributed Local Search (DLS) combina VND ou RVND com Bit-Flip, IWSS e IWSSR para refiná-las. Apache Kafka conecta os estágios, e uma camada web consulta eventos, métricas, estados, identificadores, timestamps e resultados intermediários persistidos. Observabilidade, aqui, significa inspecionar o estado externo registrado da busca, não explicar causalmente a decisão do classificador.

O objetivo geral é projetar, implementar e avaliar essa decomposição distribuída sob um protocolo rastreável. Os objetivos específicos são:

- separar geração de candidatos, controle de vizinhanças, busca local, verificação e monitoramento;
- disponibilizar uma camada web para acompanhar execuções e artefatos persistidos;
- comparar qualidade, redução dimensional, latência, vazão, rendimento de soluções e recursos com monólitos e um baseline sob dados, avaliador e teto computacional comuns;
- preservar proveniência suficiente para reconstruir as execuções e explicitar propriedades ainda não medidas.

A avaliação é orientada por quatro perguntas. **RQ1:** sob o protocolo comum, quais qualidade preditiva e redução dimensional são obtidas pelas configurações G-FShield, pelos monólitos e pelo baseline com todas as características? **RQ2:** como ranking da RCL, controlador de vizinhança e preferência de busca local se relacionam com a solução

e, mantendo o algoritmo, qual efeito a execução distribuída exerce sobre latência, vazão, sobreposição e rendimento de soluções qualificadas? **RQ3**: quais CPU e memória são observadas em cada arquitetura sob o mesmo teto agregado, sem confundir consumo com eficiência? **RQ4**: quais eventos, identificadores, métricas e artefatos permitem reconstruir uma execução, e quais propriedades de observabilidade e recuperação permanecem não avaliadas?

A contribuição inclui a arquitetura implementada, uma campanha formal com 27 braços, uma campanha causal com 30 pares que mantém RF-VND-IWSSR idêntico e varia a execução distribuída ou sequencial e uma confirmação independente com 30 novos pares sobre rendimento de soluções de alta qualidade. A análise distingue qualidade, latência, vazão, paralelismo, rendimento e custo por recurso, em vez de pressupor superioridade arquitetural. As campanhas usam divisão estratificada de treino/validação/teste, avaliador final comum e manifestos verificáveis. A independência de corpus é arquitetural, não um resultado experimental: a entrada deve ser tabular, rotulada e compatível com o contrato dos serviços. A avaliação cobre apenas o corpus ERENO e o classificador J48.

O restante da dissertação está organizado da seguinte forma. O Capítulo 2 apresenta os fundamentos; o Capítulo 3 discute trabalhos relacionados; o Capítulo 4 descreve arquitetura e implementação; o Capítulo 5 detalha protocolo e resultados; e o Capítulo 6 sintetiza conclusões, limitações e trabalhos futuros.

2 FUNDAMENTAÇÃO TEÓRICA

Este capítulo reúne os conceitos que sustentam a proposta do *G-FShield*: CPS, IDS, seleção de características, Greedy Randomized Adaptive Search Procedure for Feature Selection (GRASP-FS), orquestração distribuída e observabilidade.

2.1 Sistemas Ciber-Físicos e Detecção de Intrusão

Os CPS integram componentes computacionais, sensores, atuadores e redes de comunicação em sistemas fortemente acoplados (CÁRDENAS; AMIN; SASTRY, 2008; MITCHELL; CHEN, 2014). Em domínios como subestações digitais e automação industrial, padrões como a série IEC 61850 (IEC 61850) tornam a comunicação interoperável, mas ampliam a superfície de ataque (INTERNATIONAL ELECTROTECHNICAL COMMISSION, 2010; MACKIEWICZ, 2006; AFTAB et al., 2020). Assim, IDS precisam responder a eventos maliciosos sem comprometer a operação do ambiente protegido.

2.2 Seleção de Características

A seleção de características busca subconjuntos de atributos que preservem poder discriminativo e reduzam custo computacional (HALL, 1999; SAEYS; INZA; LARRAÑAGA, 2007; GUYON; ELISSEEFF, 2003). Sejam X um atributo, Y a classe e $H(Z) = -\sum_z p(z) \log_2 p(z)$ a entropia. O ganho de informação é

$$\text{IG}(X; Y) = H(Y) - H(Y | X), \quad (2.1)$$

isto é, a redução da incerteza da classe após observar X (QUINLAN, 1986). O Gain Ratio normaliza esse ganho pela informação intrínseca do atributo,

$$\text{GR}(X; Y) = \frac{\text{IG}(X; Y)}{H(X)}, \quad (2.2)$$

reduzindo a preferência por atributos com muitos valores (QUINLAN, 1993). A Symmetrical Uncertainty torna a dependência simétrica e limitada:

$$\text{SU}(X, Y) = 2 \frac{\text{IG}(X; Y)}{H(X) + H(Y)}. \quad (2.3)$$

Essa forma normalizada é a utilizada na seleção baseada em correlação (HALL, 1999). Por fim, ReliefF atualiza o peso W_j de cada atributo pela diferença para vizinhos próximos da mesma classe (*hit*) e de classes diferentes (*miss*):

$$\Delta W_j = -\frac{\text{diff}(x_j, H_j)}{mk} + \sum_{c \neq y} \frac{p(c)}{1 - p(y)} \frac{\text{diff}(x_j, M_{j,c})}{mk}, \quad (2.4)$$

em que m é o número de amostras e k o número de vizinhos (KONONENKO, 1994; ROBNIK-ŠIKONJA; KONONENKO, 2003). Essas quatro métricas geram ordenações independentes usadas na construção das listas restritas de candidatos.

2.3 GRASP-FS

GRASP é uma metaheurística que alterna construção gulosa-randômica e busca local (RESENDE; RIBEIRO, 2016). Na construção, uma lista restrita de candidatos (RCL) reúne alternativas bem avaliadas; uma delas é escolhida aleatoriamente e compõe a solução inicial. A busca local então aplica vizinhanças para melhorar essa solução. O GRASP-FS adapta esse procedimento à seleção de características, usando critérios de relevância para compor a RCL e operadores de busca para refinar o subconjunto (QUINCOZES et al., 2020; QUINCOZES et al., 2022b; QUINCOZES et al., 2022c).

2.4 Distribuição e Orquestração

Arquiteturas distribuídas decompõem responsabilidades em unidades menores e podem isolar falhas e permitir escala independente quando a implantação assim as configura (TANENBAUM; STEEN, 2007; NEWMAN, 2021). Em sistemas orientados a eventos, Apache Kafka (Kafka) fornece um log distribuído durável que desacopla produtores e consumidores (KREPS; NARKHEDE; RAO, 2011). Quando combinado à containerização, esse modelo torna explícitas as dependências e a implantação dos serviços (ANDERSON, 2015).

2.5 Observabilidade

Observabilidade é a capacidade de inferir o estado de um sistema a partir dos sinais que ele produz, tais como eventos, métricas, logs e rastros temporais (BEYER et al., 2016; USMAN et al., 2022). No *G-FShield*, os serviços registram eventos de execução, métricas, estados, identificadores de *run* e marcas temporais. Esses sinais são persistidos e consultados pela API; a interface web é, portanto, uma visualização dessa telemetria e não a própria observabilidade. Esse arranjo permite reconstruir uma execução e relacionar resultados intermediários aos componentes que os produziram.

2.6 Síntese

O problema tratado combina requisitos de qualidade da solução, escalabilidade arquitetural e observabilidade operacional em cenários de IDS para CPS. Os próximos capítulos situam a proposta na literatura, descrevem suas decisões de implementação e delimitam a evidência experimental disponível.

3 TRABALHOS RELACIONADOS

Este capítulo situa o *G-FShield* na literatura sobre IDS em CPS, GRASP-FS, arquiteturas distribuídas e observabilidade.

3.1 Detecção de Intrusão em Sistemas Ciber-Físicos

Os estudos de Cárdenas, Amin e Sastry (2008), Mitchell e Chen (2014) e Alcaraz e Lopez (2010) mostram que a segurança de CPS deve considerar o acoplamento entre software, rede e ativos físicos. Em subestações digitais, a literatura sobre IEC 61850 destaca tráfego específico e requisitos temporais (INTERNATIONAL ELECTROTECHNICAL COMMISSION, 2010; MACKIEWICZ, 2006; AFTAB et al., 2020). Trabalhos como Quincozes et al. (2021) e Quincozes et al. (2022a) reforçam a importância de corpora realistas na avaliação de IDS.

3.2 Seleção de Características e GRASP-FS

Hall (1999), Saeys, Inza e Larrañaga (2007), Guyon e Elisseeff (2003) apresentam a seleção de características como técnica para reduzir redundância e custo de processamento. Fora da linhagem específica desta dissertação, Yusta (2009) compara estratégias metaheurísticas para seleção de características, e Essegir (2010) combina filtros e *wrappers* em um esquema GRASP. Esses trabalhos tratam o desenho da busca, mas não a operação observável de um pipeline distribuído.

GRASP é fundamentado por Resende e Ribeiro (2016); sua aplicação à seleção de características para IDS é investigada por Quincozes et al. (2020), Quincozes et al. (2022b), Quincozes et al. (2022c). Revisões recentes indicam que metaheurísticas continuam relevantes para detecção de ataques em ambientes industriais e IIoT (TERMANINI et al., 2025; FRANCIS; SOURI; İNANÇ, 2026). Essa trajetória sustenta os componentes algorítmicos reutilizados pelo G-FShield, não uma alegação de novo algoritmo.

Como comparador externo, Scherer et al. (2024) propõem o IWSHAP, que ordena características por valores SHAP e aplica seleção incremental do tipo *wrapper* com XGBoost em tráfego CAN. O artigo relata, para seu experimento de suspensão no X-CANIDS, F1 da classe positiva de aproximadamente 0,9186 com 16 das 688 características. Esses valores são resultados publicados sob protocolo, classificador e escala de dados próprios; por isso, nesta dissertação eles contextualizam a qualidade da seleção, mas não são subtraídos diretamente das métricas do G-FShield. A comparação experimental controlada usa o artefato público fixado em uma revisão, partições comuns de treino, validação e teste e o mesmo avaliador final para todos os subconjuntos.

3.3 Arquiteturas Distribuídas e Observabilidade

Trabalhos anteriores exploram paralelismo em partes do GRASP-FS. A proposta de Silva et al. (2023) distribui a busca local, enquanto a de Faria et al. (2023) distribui a geração da lista restrita de candidatos. Em metaheurísticas de outros domínios, Subramanian et al. (2010) exemplifica busca paralela, mas sem o problema de seleção de características ou a integração operacional aqui estudada.

Em outra linha, sistemas distribuídos fundamentam o desacoplamento entre componentes (TANENBAUM; STEEN, 2007). A decomposição em serviços é discutida por Newman (2021), e logs de mensageria por Kreps, Narkhede e Rao (2011). Trabalhos de confiabilidade operacional tratam sinais de telemetria (BEYER et al., 2016; MATEUS-COELHO; CRUZ-CUNHA; FERREIRA, 2021). Revisões de observabilidade discutem diagnóstico em arquiteturas de serviços (USMAN et al., 2022; FASEEHA et al., 2025). Já Wu et al. (2013) trata seleção de características recebidas em fluxo. Esse problema é distinto: o G-FShield avaliado recebe corpus pré-materializado e não realiza seleção online. Entre os trabalhos analisados, não foi encontrada a reunião de DRG, DLS, persistência de eventos e consulta operacional em um fluxo GRASP-FS; como a busca não foi sistemática, essa conclusão é delimitada ao conjunto efetivamente examinado.

3.4 Lacuna Identificada

O *G-FShield* integra geração distribuída, busca local distribuída, persistência de resultados e visualização de telemetria em uma única arquitetura. Sua contribuição principal é arquitetural e de engenharia de software: a integração ponta a ponta, a orquestração e a rastreabilidade do processo. Os componentes DRG e DLS derivam e estendem a trajetória de trabalhos anteriores, e não são apresentados como um novo algoritmo de busca.

3.5 Tabela Comparativa

A Tabela 1 sintetiza as capacidades discutidas. “Construção dist.” significa distribuir a formação da solução inicial; “refinamento dist.”, distribuir a busca local; “integração E2E”, conectar ambas no mesmo fluxo; e “telemetria”, persistir sinais que permitam reconstruir uma execução. A tabela não atribui seleção online ao G-FShield.

Tabela 1 – Síntese dos trabalhos relacionados citados no manuscrito.

Referência	CPS/ IDS	GRASP- FS	Construção dist.	Refinamento dist.	Integração E2E	Telemetria
Resende e Ribeiro (2016)	–	✓	–	–	–	–
Yusta (2009); Esseghir et al. (2010)	–	Parcial	–	–	–	–
Quincozes et al. (2020–2022)	✓	✓	–	–	–	–
Scherer et al. (2024), IWSHAP	✓	–	–	–	–	–
Silva et al. (2023)	✓	✓	–	✓	Parcial	–
Faria et al. (2023)	✓	✓	✓	–	Parcial	–
Beyer et al. (2016); Usman et al. (2022)	–	–	–	–	–	✓
Wu et al. (2013)	–	–	–	–	–	–
Este trabalho (G-FShield)	✓	✓	✓	✓	✓	✓

Portanto, a tabela sustenta uma diferenciação arquitetural: o G-FShield conecta etapas de seleção e seus sinais operacionais. Uma afirmação de cobertura exaustiva da literatura exigiria uma revisão sistemática ou uma estratégia bibliográfica documentada.

4 DESENVOLVIMENTO

Este capítulo descreve a arquitetura e a implementação do *G-FShield*. A contribuição apresentada aqui é arquitetural e de engenharia de software: integrar a construção de soluções iniciais, o refinamento por busca local, a mensageria, a persistência e a inspeção operacional. Os algoritmos de seleção e de busca empregados derivam de trabalhos anteriores e não são reivindicados como novos.

4.1 Visão Geral e Contrato de Entrada

O *G-FShield* foi estruturado em três camadas: uma camada web de operação, uma camada de orquestração e persistência e uma camada de execução distribuída. A primeira apresenta o estado observado e recebe comandos; a segunda concentra a Application Programming Interface (API), o banco de dados, o *cache* e a integração com a mensageria; a terceira executa a geração distribuída da lista restrita de candidatos (DRG) e a busca local distribuída (DLS). A Figura 1 sintetiza essas fronteiras.

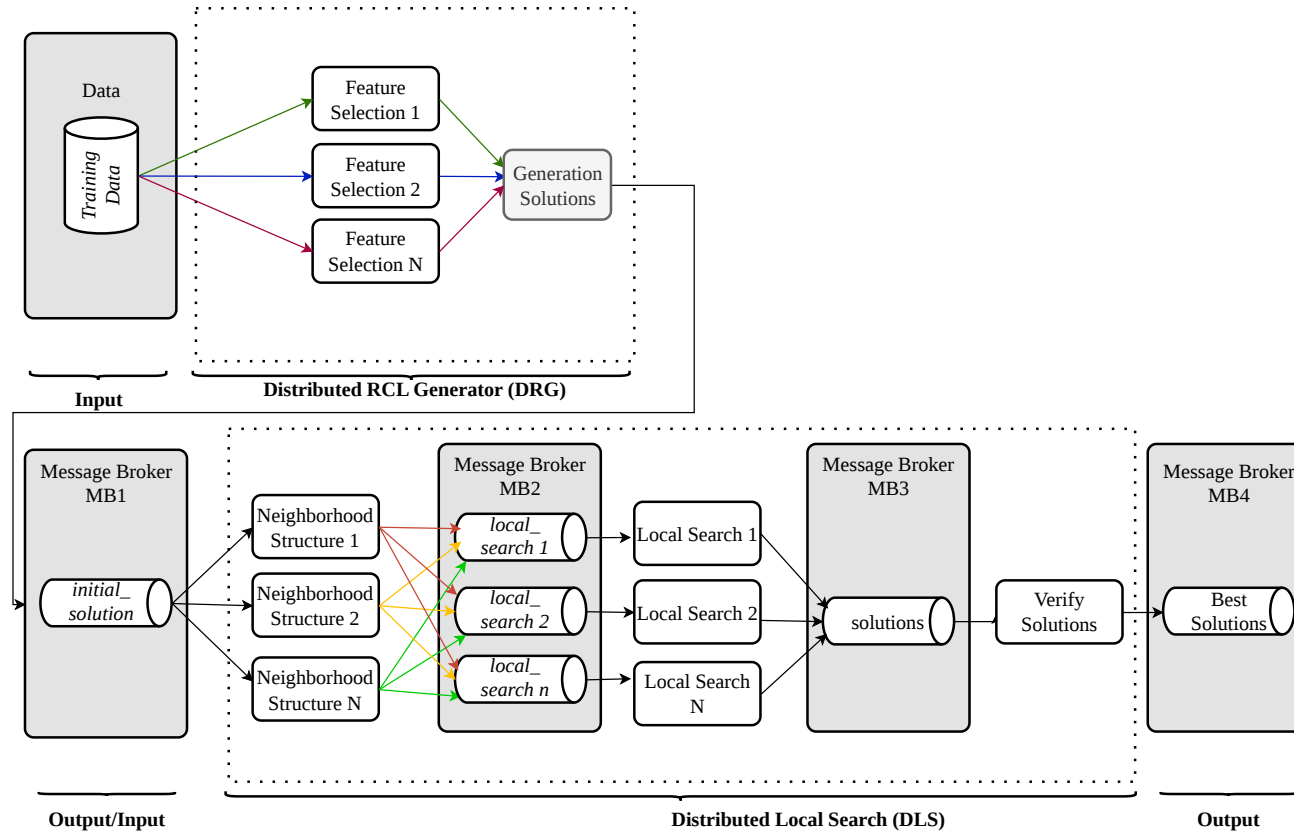


Figura 1 – Resumo da arquitetura do *G-FShield*; a margem em branco do arquivo original foi removida somente na inclusão LaTeX.

A independência em relação à fonte dos dados é condicionada a um contrato concreto. A API algorítmica recebe dois arquivos ARFF distintos, um de treino e outro usado para pontuar candidatos, acessíveis em `/datasets`. Ambos devem possuir o mesmo esquema e rótulo no último atributo. Na campanha formal, o segundo arquivo é a validação; o teste reservado é acessado somente pelo avaliador final. O leitor Weka define o último atributo como classe, e os preditores são identificados a partir de um. Os classificadores aceitos pela aplicação são J48, Naive Bayes e Random Forest; a campanha fixa J48.

Consequentemente, tráfego, protocolos ou *logs* não são consumidos em formato bruto: precisam ser previamente transformados em matrizes rotuladas que satisfaçam esse contrato. A generalidade, portanto, é arquitetural e de formato de entrada, não evidência empírica de desempenho em fontes heterogêneas. Os CSVs históricos referenciam `erenoall.arff`, enquanto o script atual usa a separação ERENO 1k; sem manifesto, não se atribui uma configuração à outra. No `erenoall.arff` disponível, *grayhole* ocupa o índice zero e *normal* o índice um. Embora o nome `normalClass=0` permaneça em rotinas binárias legadas, esse caminho foi desabilitado antes da construção das imagens da campanha formal; a avaliação ativa percorre as nove classes do ARFF e calcula F1 macro.

Cada requisição informa os arquivos de treino e teste, o classificador, o número de gerações, o corte da RCL, o tamanho da amostra inicial, a estratégia VND ou RVND, as buscas locais habilitadas e, opcionalmente, limites de iteração. Esses campos formam a unidade de configuração propagada entre os serviços. Cada solução recebe ainda um `seedId` aleatório, usado para correlacionar os refinamentos que dela derivam.

4.2 Drivers e Decisões Arquiteturais

A decomposição foi orientada por seis drivers: vazão, desacoplamento, rastreabilidade, persistência, inspeção de resultados intermediários e possibilidade de evolução operacional. A Tabela 2 explicita a relação entre esses drivers e as decisões adotadas. A decomposição em serviços e a comunicação assíncrona seguem princípios usuais de sistemas distribuídos e arquiteturas de microserviços (TANENBAUM; STEEN, 2007; NEWMAN, 2021; KREPS; NARKHEDE; RAO, 2011).

As campanhas apresentadas no Capítulo 5 verificam esse driver: a sobreposição mediana aproximou-se de 100%, a vazão de candidatos foi cerca de 45% maior e uma amostra confirmatória independente apresentou maior rendimento médio de subconjuntos distintos com F1 de validação $\geq 0,945$. Essas vantagens coexistiram com pior latência até a primeira solução em 0,94 e maior custo por candidato.

As fases de construção e refinamento possuem custos e padrões de consumo diferentes. Por isso, foram separadas em serviços com limites próprios de CPU, memória e concorrência. Essa separação permite observar e alterar uma responsabilidade sem incorporar toda a busca em um único processo. Ela também torna possível aumentar o número de instâncias de um serviço; contudo, a relação entre réplicas, vazão e custo precisa de

Tabela 2 – Drivers, decisões e limites da arquitetura.

Driver	Decisão de projeto	Escopo da evidência
Vazão	Sobrepôr construção e busca local e permitir consumidores independentes para candidatos distintos.	As campanhas pareadas demonstram maior vazão de candidatos e maior rendimento médio de soluções qualificadas sob teto comum; não demonstram menor custo por candidato.
Desacoplamento	Usar tópicos Kafka entre construção, controladores, operadores, verificador e monitor.	O acoplamento temporal é reduzido, mas a robustez diante de falhas não foi experimentalmente medida.
Rastreabilidade	Propagar <code>seedId</code> , algoritmo, busca, iterações, conjuntos de atributos e métricas.	Reconstrói-se a sequência observada pelo monitor, não uma prova de completude do fluxo.
Persistência	Armazenar lançamentos, execuções, eventos e agregados no PostgreSQL e CSVs no volume de métricas.	Redis é <i>cache</i> ; não substitui o armazenamento persistente.
Inspeção intermediária	Publicar progresso selecionado, soluções refinadas e melhores soluções observadas.	A interface apresenta telemetria; ela não torna o algoritmo explicável.
Evolução operacional	Isolar responsabilidades para permitir replicação e substituição por serviço.	Escalabilidade e tolerância a falhas são objetivos de projeto, não resultados demonstrados.

um experimento de escala específico.

A persistência e o histórico de eventos foram adotados para reduzir a dependência de estado mantido apenas em memória. A configuração avaliada, entretanto, usa um único *broker* Kafka e fator de replicação um. Assim, não se infere alta disponibilidade ou tolerância a falhas a partir da decomposição. Essas propriedades exigiriam injeção de falhas, verificação de reentrega, idempotência, recuperação e reprocessamento.

4.3 Geração Distribuída da Lista Restrita de Candidatos

Na etapa DRG, quatro serviços independentes calculam Information Gain (IG), Gain Ratio (GR), Symmetrical Uncertainty (SU) e ReliefF. Cada serviço pontua somente os preditores do treino, ordena-os em ordem decrescente e forma sua própria RCL com os primeiros `rclCutoff` elementos; o atributo de classe, na última posição, é excluído do ranking. Não há agregador que funda os quatro rankings em uma lista única. Uma solicitação com várias métricas dispara ramos independentes, que publicam soluções iniciais no mesmo tópico.

A construção segue o princípio de uma RCL combinada com escolha aleatória, característico do GRASP (RESENDE; RIBEIRO, 2016). A adaptação para seleção de atributos e as etapas DRG utilizadas aqui derivam de GRASP-FS e de suas extensões distribuídas (QUINCOZES et al., 2020; QUINCOZES et al., 2022b; QUINCOZES et al., 2022c; FARIA et al., 2023). Essas referências fundamentam a decisão algorítmica;

referências de microserviços fundamentam somente a decomposição da implementação.

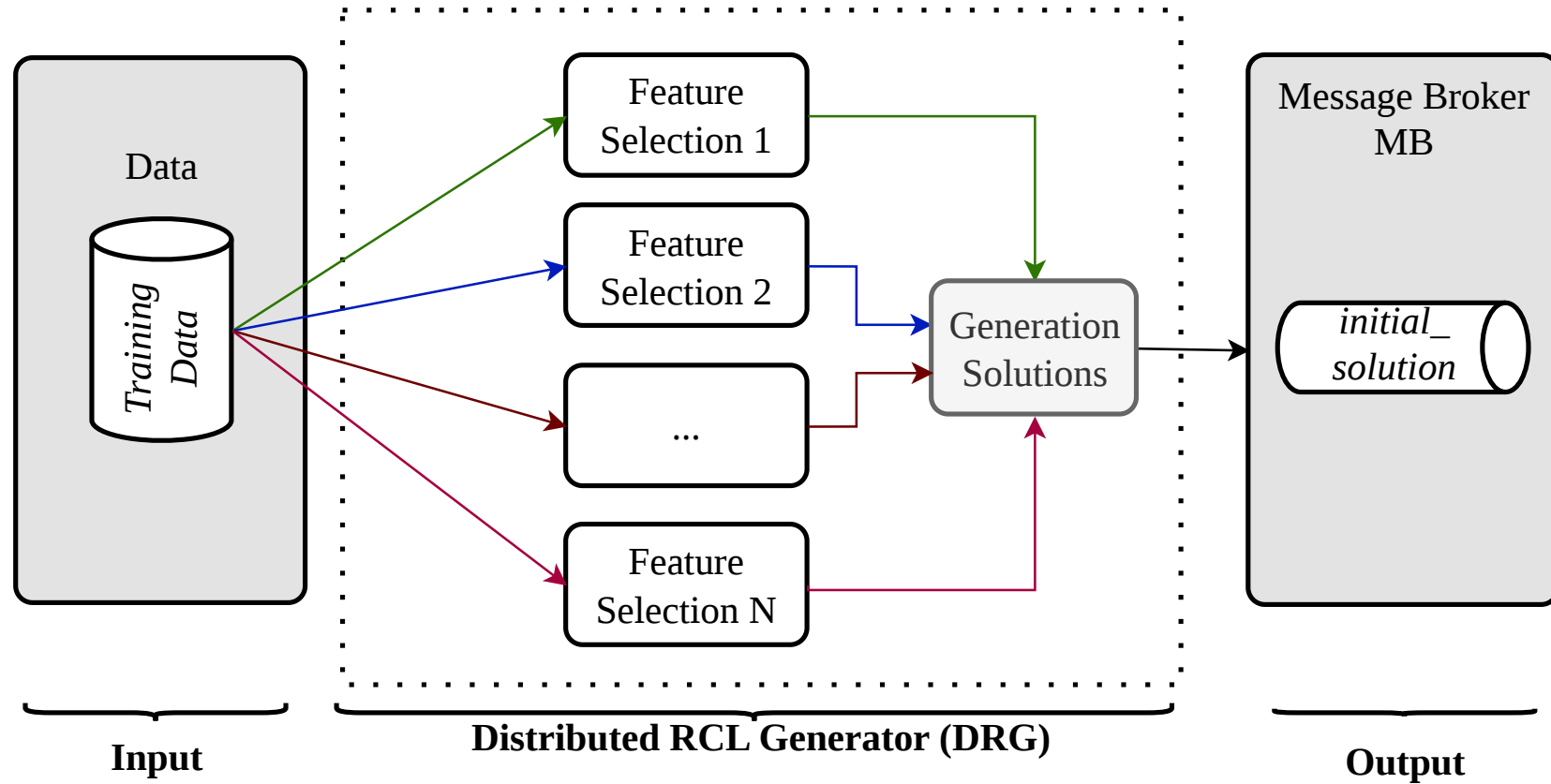


Figura 2 – Arquitetura genérica da etapa DRG.

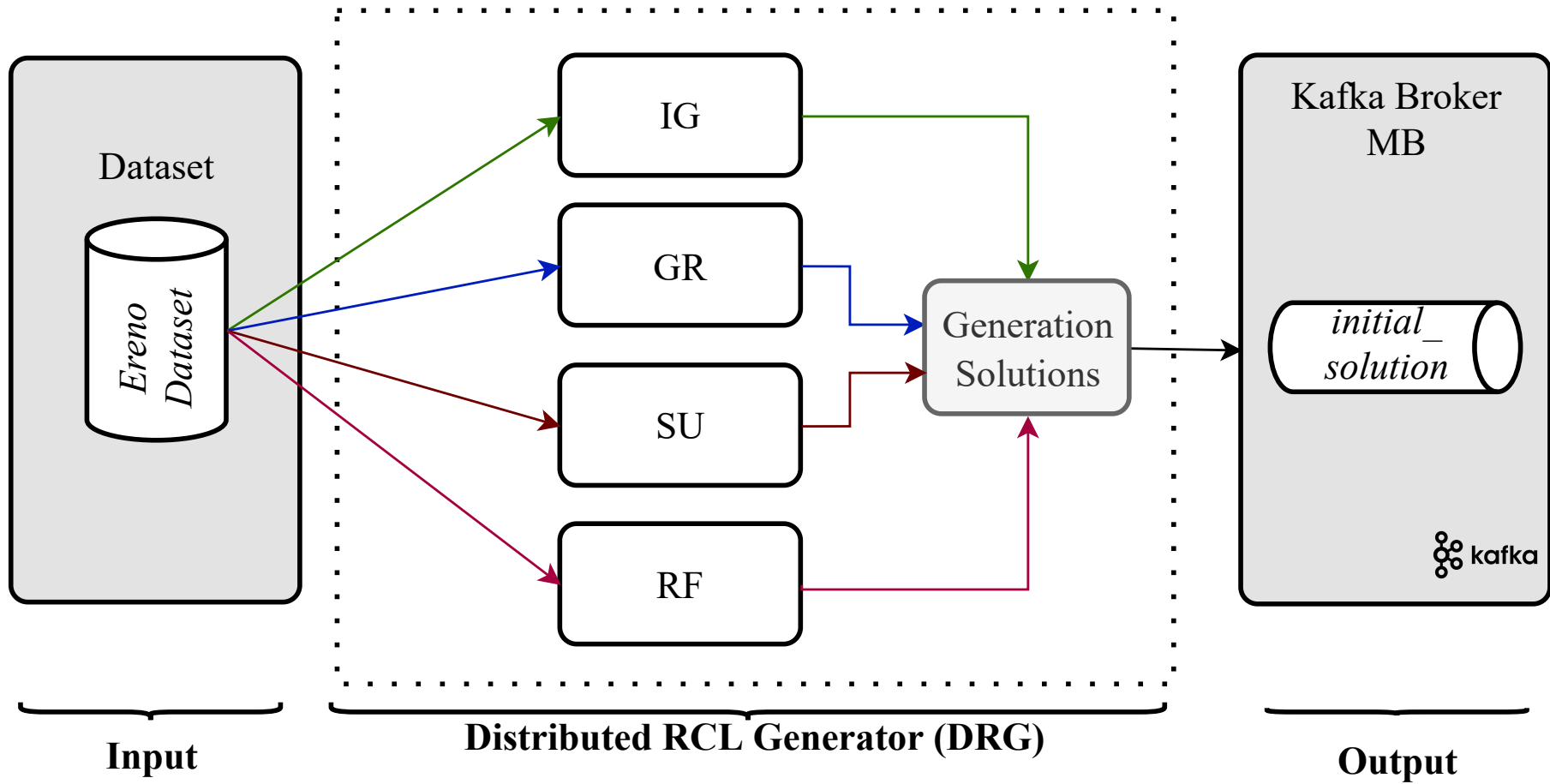


Figura 3 – Instanciação da etapa DRG com as quatro métricas implementadas.

Em cada geração, o serviço copia sua RCL e sorteia `sampleSize` posições uniformemente, sem reposição nessa cópia. Na campanha formal, o gerador é inicializado por `CAMPAIGN_RANDOM_SEED`; o `seedId` identifica a solução, enquanto `campaignId`, `runId`, `requestId` e a semente numérica preservam a proveniência. O subconjunto é avaliado no treino e na validação reduzidos; o teste reservado não participa da busca. A mensagem publicada preserva subconjunto, RCL, métricas internas e configuração de refinamento.

A Listagem 4.1 apresenta esse comportamento em pseudocódigo. `metric` identifica um dos quatro ramos. Os parâmetros são mapeados assim:

- `G`: `maxGenerations`;
- `k`: `rclCutoff`;
- `s`: `sampleSize`.

Listing 4.1 – Formação da RCL e construção aleatória implementadas no DRG.

```
procedure DRG(trainFile, validationFile, metric, G, k, s, classifier,
             neighborhood, enabledSearches, iterationLimits, runSeed)
  train <- readARFF(trainFile); train.class <- lastAttribute
  valid <- readARFF(validationFile); valid.class <- lastAttribute
  rng <- PseudoRandomGenerator(runSeed)

  scored <- empty list
  for each predictor a in train, excluding the class do
    scored.append((metric(train, a), oneBasedIndex(a)))
  end for
  ranking <- sortDescending(scored)
  rcl <- first min(k, size(ranking)) feature identifiers

  for generation <- 1 to G do
    pool <- copy(rcl)
    selected <- empty list
    repeat min(s, size(pool)) times
      p <- UniformInteger(rng, 1, size(pool))
      selected.append(removeAt(pool, p))
    end repeat

    seedId <- randomUUID()
    scores <- trainAndValidate(classifier, train, valid, selected)
    appendServiceMetricsCSV(selected, scores, classifier,
                           trainFile, validationFile)
    publish(INITIAL_SOLUTION_TOPIC,
           {seedId, selected, rcl, scores, metric,
            neighborhood, enabledSearches, iterationLimits})
  end for
```

```
end procedure
```

4.4 Busca Local Distribuída

Os controladores Variable Neighborhood Descent (VND) e Randomized Variable Neighborhood Descent (RVND) recebem cada solução inicial e encaminham mensagens aos operadores Bit Flip, Incremental Wrapper Subset Selection (IWSS) e Incremental Wrapper Subset Selection with randomization (IWSSr). O VND percorre as buscas habilitadas na ordem configurada, compara o melhor resultado do ciclo com sua linha de base e reinicia o ciclo quando ocorre melhora e ainda existe orçamento de despachos. O RVND seleciona aleatoriamente uma busca habilitada a cada passo e continua até o limite de iterações. O uso de VND e da variante randomizada é fundamentado na literatura de vizinhanças variáveis e RVND (MLADENOVIĆ; HANSEN, 1997; DUARTE et al., 2018; SUBRAMANIAN et al., 2010); a distribuição desses operadores segue o GDLS-FS (SILVA et al., 2023).

O Bit Flip troca aleatoriamente um atributo do subconjunto por um atributo da lista de candidatos a cada iteração. O IWSS adiciona sequencialmente o primeiro atributo ainda disponível e preserva o melhor subconjunto avaliado. O IWSSr executa o movimento de adição e, para cada posição do subconjunto resultante, avalia a remoção correspondente, conservando a melhor substituição. Esses operadores de vizinhança seguem as avaliações do GRASP-FS (QUINCOZES et al., 2020; QUINCOZES et al., 2022b; QUINCOZES et al., 2022c). A RCL encaminhada ao DLS ainda contém atributos da solução inicial, mas a campanha canoniza a solução final e rejeita subconjuntos vazios, inválidos ou duplicados. Os operadores treinam no treino, pontuam candidatos na validação e publicam o resultado em `SOLUTIONS_TOPIC`. O verificador mantém o maior F1 interno por `seedId` e publica uma melhora estrita em `BEST_SOLUTION_TOPIC`; isso representa o melhor resultado observado, não certificado global de otimalidade.

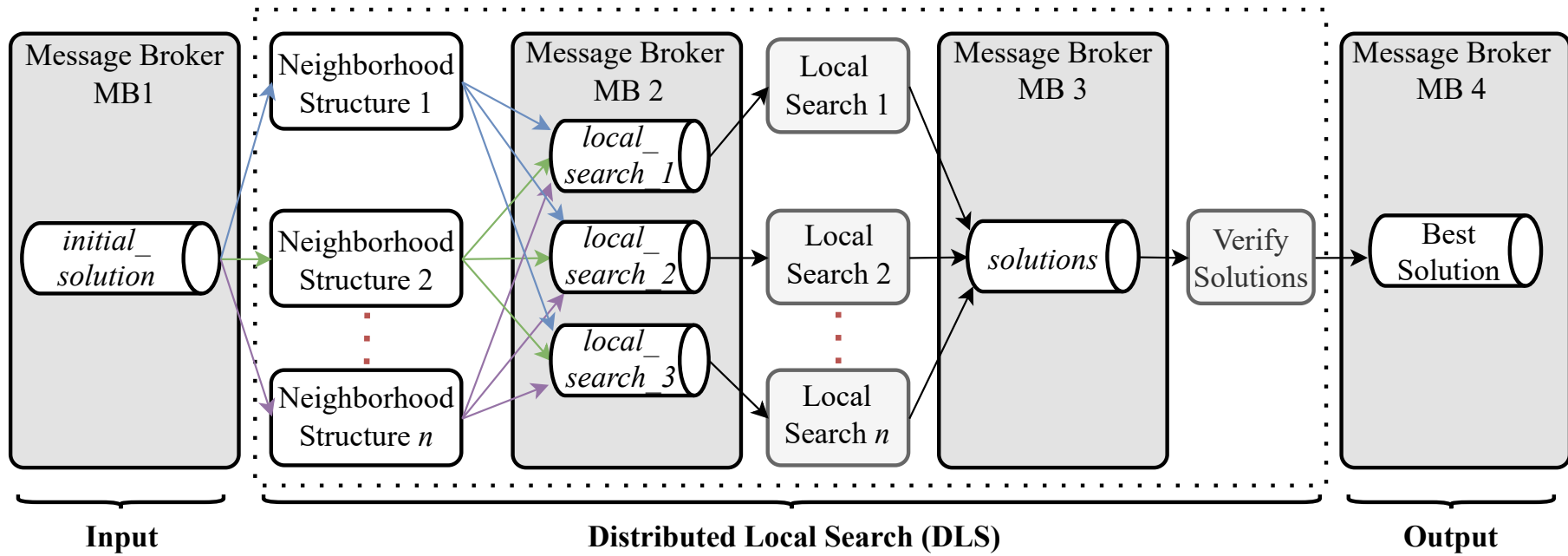


Figura 4 – Arquitetura genérica da etapa DLS.

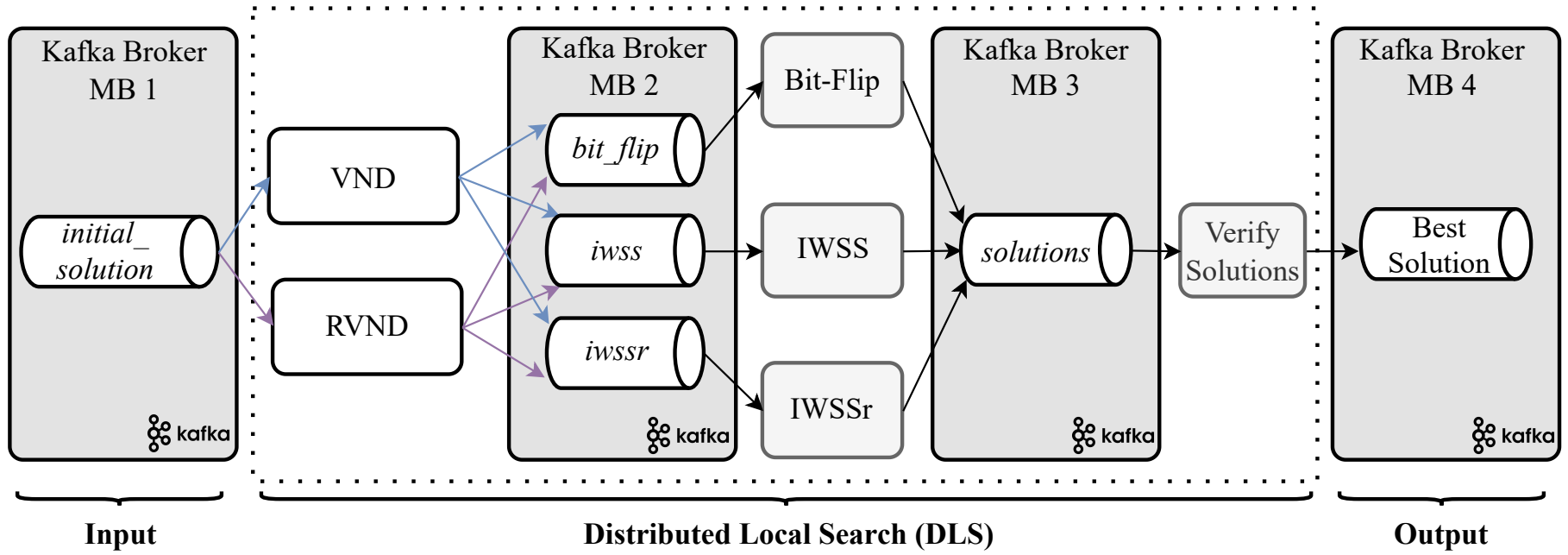


Figura 5 – Instanciação da etapa DLS com os controladores e operadores implementados.

A Listagem 4.2 resume a orquestração efetiva. O orçamento do VND é o número configurado de ciclos multiplicado pela quantidade de buscas habilitadas; o RVND usa diretamente o limite de iterações. Os valores padrão dos dois controladores são dez quando a requisição não fornece um limite.

Listing 4.2 – Refinamento e verificação distribuídos implementados no DLS.

on INITIAL_SOLUTION_TOPIC or NEIGHBORHOOD_RESTART_TOPIC receive seed

best <- seed

searches <- configured searches or [BIT_FLIP, IWSS, IWSSR]

if seed.neighborhood = VND then

dispatchBudget <- maxCycles * size(searches)

repeat while dispatchBudget remains

baseline <- best.F1

for each search in searches in configured order do

candidate <- dispatchAndReceive(search, best)

best <- argmaxF1(best, candidate)

dispatches <- dispatches + 1

end for

if best.F1 <= baseline then stop

end repeat

else if seed.neighborhood = RVND then

repeat until maxIterations dispatches

search <- UniformChoice(searches)

candidate <- dispatchAndReceive(search, best)

best <- argmaxF1(best, candidate)

dispatches <- dispatches + 1

end repeat

end if

for each candidate received by verifier do

if bestSeen[candidate.seedId] is undefined or

candidate.F1 > bestSeen[candidate.seedId].F1 then

bestSeen[candidate.seedId] <- candidate

publish(BEST_SOLUTION_TOPIC, candidate)

end if

end for

end receive

O protocolo formal inicializa a construção, o Bit Flip e o RVND com a semente numérica da execução e a persiste separadamente do UUID `seedId`. Isso torna a sequência pseudoaleatória repetível no ambiente versionado; concorrência, ordem de mensagens e

plataforma ainda podem introduzir variação, razão pela qual cada resultado preserva também timestamps, comando e artefatos.

4.5 Orquestração por Eventos

O Kafka desacopla produtores e consumidores pelos seguintes grupos de tópicos:

- construção: `<INITIAL_SOLUTION_TOPIC>`;
- operadores: `<BIT_FLIP_TOPIC>`, `<IWSS_TOPIC>` e `<IWSSR_TOPIC>`;
- coordenação: `<SOLUTIONS_TOPIC>` e `<NEIGHBORHOOD_RESTART_TOPIC>`;
- progresso monitorado: `<LOCAL_SEARCH_PROGRESS_TOPIC>`;
- melhor resultado observado: `<BEST_SOLUTION_TOPIC>`.

A chave das mensagens de solução é o `seedId`, quando disponível. Essa separação permite que eventos observados alimentem o monitor (KREPS; NARKHEDE; RAO, 2011).

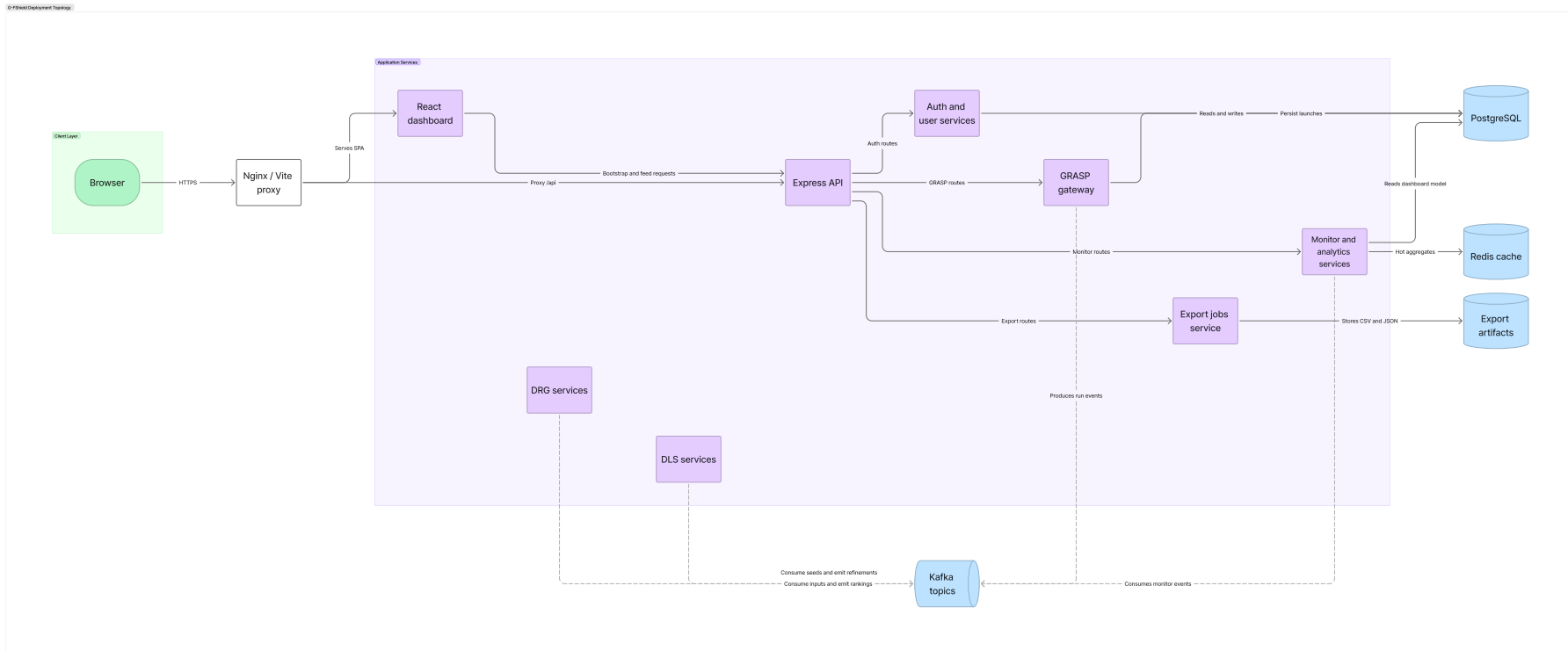


Figura 6 – Topologia de implantação e fronteiras de execução do *G-FShield*.

O Kafka é configurado como log de eventos da execução, mas durabilidade potencial não equivale a recuperação demonstrada. A configuração possui criação automática de tópicos, seis partições e fator de replicação um. Não foram medidos perda ou duplicação de mensagens, completude dos eventos, custo da instrumentação, comportamento após reinício, recuperação de consumidor ou reproprocessamento. Por isso, resiliência e tolerância a falhas permanecem objetivos de projeto.

4.6 Telemetria Persistida e Camada Web

Observabilidade, neste trabalho, significa a possibilidade de inspecionar o estado externo registrado da execução. Não significa explicar causalmente as decisões do classificador. O monitor da API consome cinco tópicos de interesse: solução inicial, reinício de vizinhança, progresso de busca local, solução refinada e melhor solução. Cada evento persistido contém tipo, conteúdo JSON e instante de ingestão pelo backend, além de tópico, estágio, estado, `seedId`, `requestId`, partição e *offset* quando disponíveis. Uma impressão digital evita inserir novamente o mesmo evento observado. Na campanha formal, `campaignId`, `runId`, `requestId`, `seedId`, timestamp UTC do produtor e tempo monotônico são propagados nos objetos experimentais.

O PostgreSQL mantém três níveis principais: o lançamento solicitado, a execução corrente por `seedId` e o histórico de eventos associado. O estado da execução inclui algoritmo da RCL, classificador, busca local, vizinhança, arquivos, iterações, F1, acurácia, precisão, revocação, CPU, memória, tempo e conjuntos de atributos. Modelos de leitura materializados agregam tópicos, algoritmos, recursos e intervalos da linha do tempo. O Redis armazena temporariamente respostas consultadas com frequência; a fonte persistente é o PostgreSQL.

Os serviços Java também escrevem CSVs por componente no volume `/metrics`. Esses arquivos contêm subconjunto, F1, acurácia, precisão, revocação, tempo, CPU, memória, classificador e nomes dos arquivos. Como os CSVs atuais não registram o `seedId`, sua correlação direta com o histórico web não deve ser presumida. Os *logs* Java e Node são emitidos em *stdout*; o Compose não configura armazenamento de logs, política de retenção ou coletor como Loki/ELK. Portanto, eventos e métricas são persistidos pela aplicação, mas a persistência de logs depende do runtime externo e não foi demonstrada.

A combinação de identificadores, eventos e *timestamps* permite reconstruir a linha do tempo que chegou ao monitor e comparar estados de uma mesma semente. Ela não garante que todos os eventos produzidos foram capturados. Na campanha formal, a semente numérica, o `runId` e os checksums complementam essa telemetria; completude do monitor e reprodutibilidade em outro hardware continuam limitadas.

A implementação web usa React e Vite no cliente; Express e Swagger na API; PostgreSQL para persistência; e Redis como *cache* opcional. A interface consulta resumos, eventos paginados, linhas do tempo e exportações CSV ou JSON. Essas tecnologias

descrevem a implementação; por si só, não comprovam produtividade, estabilidade, responsividade ou desempenho.

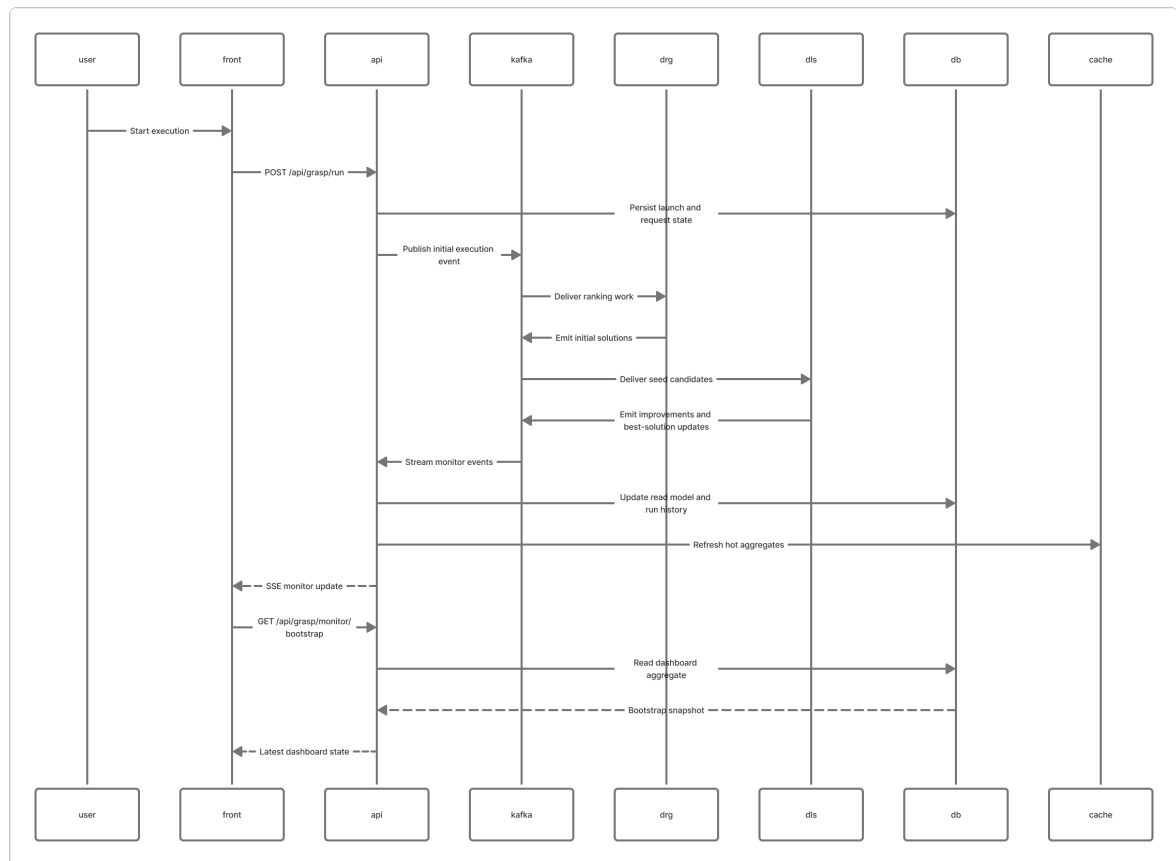


Figura 7 – Ciclo de execução do *G-FShield*, da solicitação aos estados apresentados na interface.

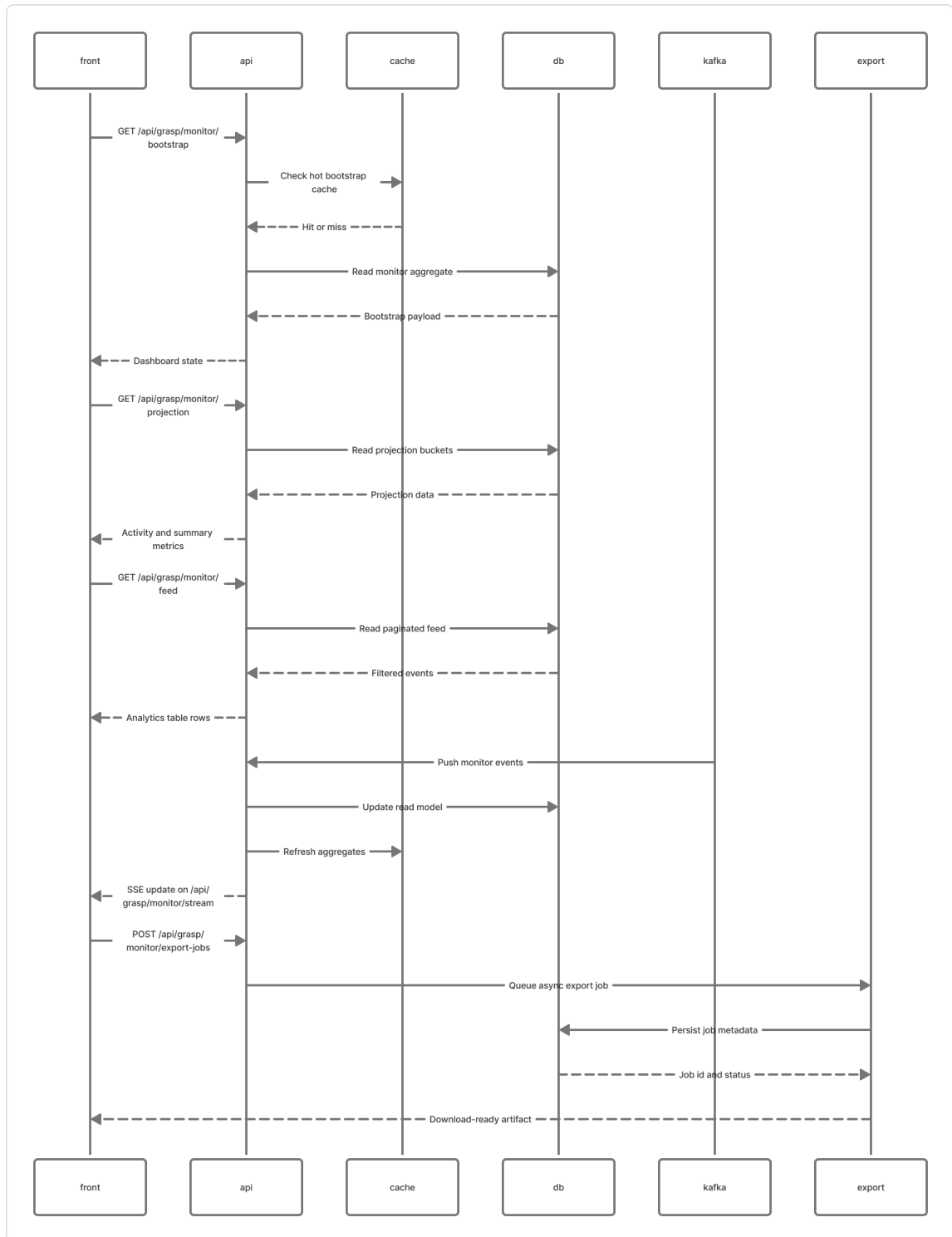


Figura 8 – Pipeline de ingestão, persistência e apresentação da telemetria observada.

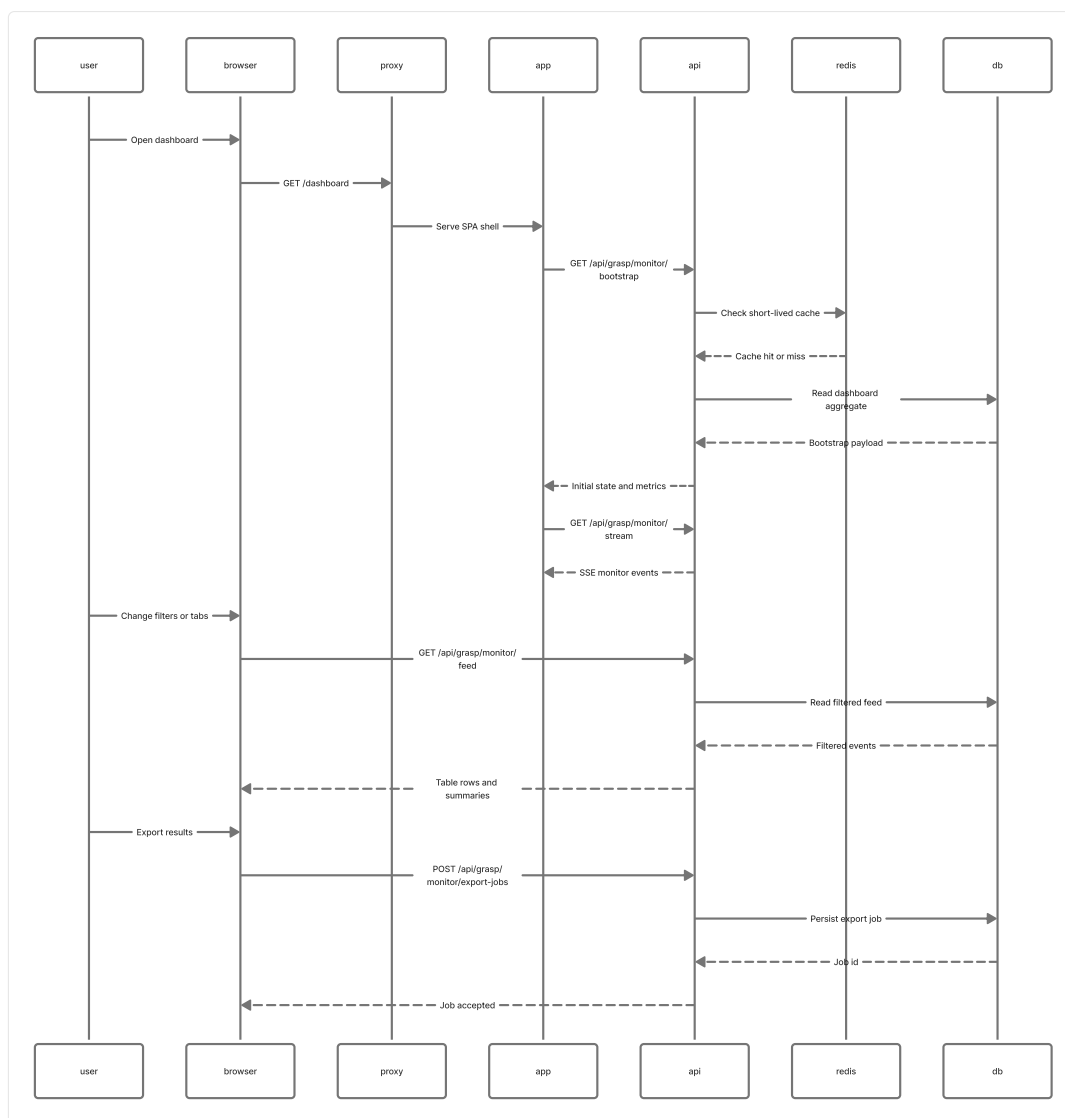


Figura 9 – Fluxo funcional do cliente web e de suas consultas à API.

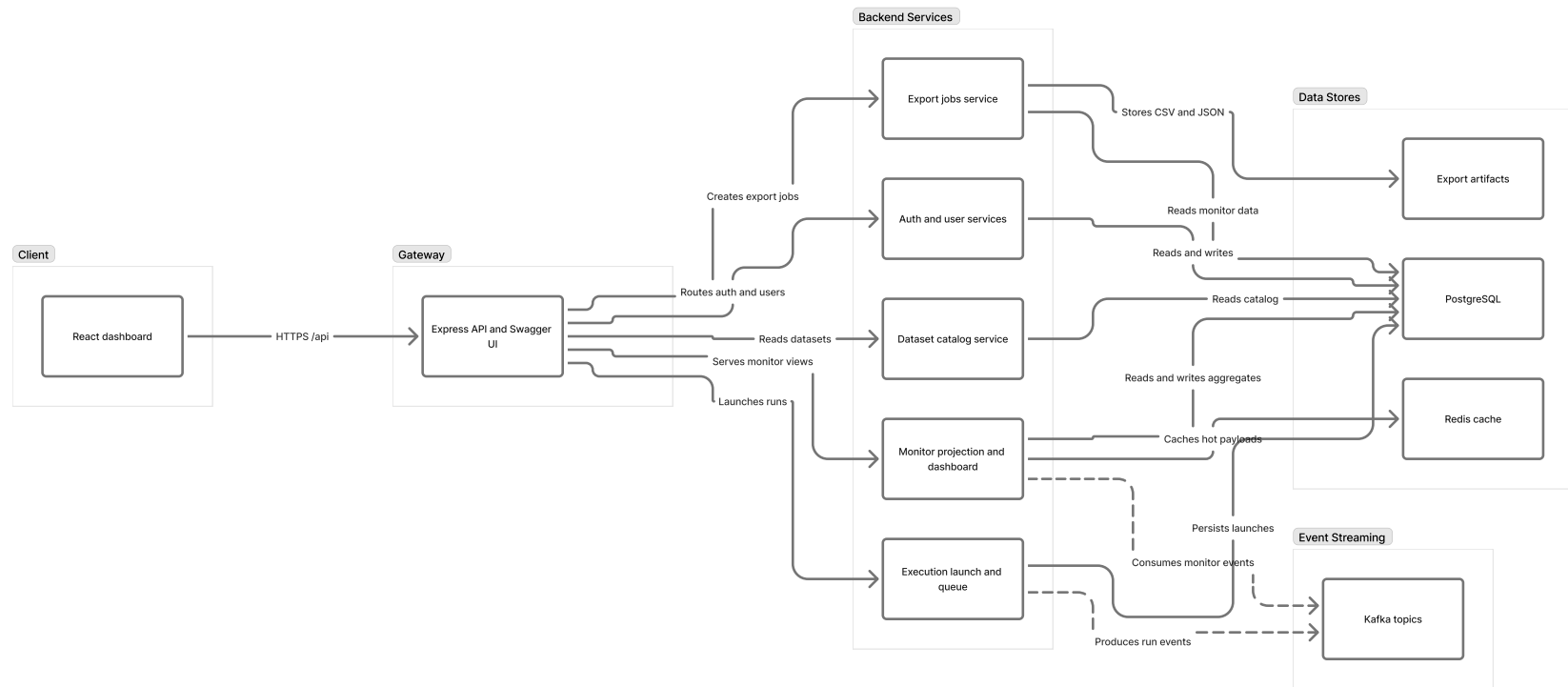


Figura 10 – Serviços do backend, armazenamentos e integração com a mensageria.

As Figuras 7 a 10 complementam a visão abstrata com os fluxos implementados. As capturas da interface, a documentação da API e os diagramas de apoio encontram-se nos Apêndices A, B e C.

4.7 Configuração de Referência e Reprodutibilidade

A Tabela 3 registra a configuração local definida por três arquivos:

- `<docker-compose.yml>`;
- `<docker-compose.local.yml>`;
- `<webservice/api/docker-compose.db.yml>`.

Os valores são limites por contêiner. O *preset* de servidor aumenta parte desses limites e, portanto, não deve ser misturado a uma comparação experimental sem registro explícito.

Tabela 3 – Recursos e concorrência da configuração local de referência.

Grupo de serviços	Qtd.	CPU/unid.	RAM/unid.	Concorrência/configuração
DRG: IG, GR, SU e ReliefF	4	1,50	1536 MB	<i>pool</i> assíncrono: núcleo 2, máximo 4, fila 80
DLS: Bit Flip, IWSS e IWSSR	3	1,00	1536 MB	um consumidor Kafka por serviço
VND, RVND e verificador	3	0,75	768 MB	um consumidor Kafka por serviço
Kafka	1	1,00	1024 MB	seis partições; replicação 1
ZooKeeper	1	0,50	256 MB	porta de cliente 2181
Kafbat UI	1	0,50	512 MB	cliente do Kafka interno
PostgreSQL	1	0,50	512 MB	configuração do banco local
Redis	1	0,25	256 MB	persistência AOF e <i>snapshot</i> configurados

O hospedeiro indicado como servidor de execução foi inspecionado em 25 de agosto de 2026. A máquina, identificada pelo sistema como `mc2-server-01`, possui dois processadores Intel Xeon E5-2620 v4 a 2,10 GHz, com oito núcleos e duas *threads* por núcleo em cada soquete, totalizando 16 núcleos físicos e 32 processadores lógicos em dois nós NUMA. O Docker detectou 67.447.271.424 bytes de memória (64.322,73 MiB; aproximadamente 62,8 GiB), e o sistema tinha 8 GiB de *swap*. O ambiente usa Ubuntu 18.04.6 LTS, arquitetura x86-64 e *kernel* Linux 4.15.0-213-generic. Foram observados Docker Engine/cliente 24.0.2, API 1.43, *storage driver* `overlay2`, *cgroup driver* `cgroupfs`, `docker compose` 2.18.1 e o executável `docker-compose` 2.30.3.

Os rótulos do Docker ligavam os serviços em execução ao projeto `g-fshield` e à sobreposição de `<docker-compose.yml>` com `<docker-compose.server.yml>`. A Tabela 4 registra os limites efetivos observados. Havia uma instância de cada serviço; não havia modo Swarm nem replicação adicional de contêiner. PostgreSQL e Redis pertenciam a uma segunda composição, formada por `<webservice/api/docker-compose.db.yml>` e `<webservice/api/docker-compose.db.server.yml>`. A API e o cliente web não apareciam na lista de contêineres, de modo que não se atribui a eles um limite observado.

Na raiz do repositório, os comandos correspondentes aos arquivos registrados nos rótulos são:

```
docker compose -f docker-compose.yml -f docker-compose.server.yml up -d
docker compose -f webservice/api/docker-compose.db.yml -f webservice/api/docker-
compose.db.server.yml up -d
```

Esses comandos reconstroem a composição observada; não substituem um histórico de terminal ou manifesto que confirme as opções usadas na campanha.

Tabela 4 – Limites efetivos no servidor inspecionado.

Grupo de serviços	Qtd.	CPU/unid.	RAM/unid.	Concorrência/configuração
DRG: IG, GR, SU e ReliefF	4	2,00	2048 MB	<i>pool</i> : núcleo 3, máximo 6, fila 150
DLS: Bit Flip, IWSS e IWSSR	3	1,50	2048 MB	um consumidor Kafka por serviço
VND, RVND e verificador	3	1,00	1024 MB	um consumidor Kafka por serviço
Kafka	1	2,00	2048 MB	seis partições; replicação 1
ZooKeeper	1	0,75	512 MB	uma instância
Kafbat UI	1	1,00	2048 MB	uma instância
PostgreSQL	1	1,00	1024 MB	uma instância
Redis	1	0,25	256 MB	uma instância

O diretório `</home/idscps/nicolas/graspy2>` forneceu a implementação do Monólito 1. O diretório `</home/idscps/nicolas/Graspy>` forneceu a do Monólito 2. Para a campanha formal, ambos foram reconstruídos em imagens versionadas e executados como um único contêiner por braço sob o mesmo teto agregado. O Monólito 1 ordena atributos por Gain Ratio, constrói um subconjunto de cinco a partir de corte 30 e aplica Bit-Flip por até 100 iterações locais. O Monólito 2 usa Gain Ratio, constrói cinco atributos a partir de RCL de tamanho 30 e aplica IWSS sob VND por até 100 iterações. Ambos recebem as mesmas divisões imutáveis de treino, validação e teste, sementes 42–49, limite de 3.000 s, máximo de 500 melhorias aceitas e melhora mínima de 0,0001.

Durante a busca formal, os dois monólitos e o G-FShield usam Weka J48 e F1 macro na validação. A solução de melhor validação é então avaliada uma única vez no teste. Essa adaptação elimina as diferenças históricas de classificador, função objetivo e escala das métricas, mas preserva algoritmos comparadores diferentes: GR–BF no Monólito 1, GR–IWSS no Monólito 2 e as 24 combinações de ranking, controlador e operador no G-FShield. Por isso, a campanha compara soluções completas, mas não isola causalmente apenas o efeito da distribuição.

Os quatro serviços DRG expõem as portas 8086–8089; Bit Flip, IWSS, IWSSR e o verificador usam 8082–8085; RVND e VND usam 8090 e 8091. Kafka expõe 19092, ZooKeeper 2181, JMX 9999 e Kafbat UI 8080. A API, o cliente web, PostgreSQL e Redis usam, por padrão, 4000, 3000, 5432 e 6379. Os conjuntos de dados e os CSVs são volumes vinculados aos diretórios `datasets/` e `metrics/`; PostgreSQL e Redis usam volumes nomeados.

As imagens Java são construídas com Maven 3.9.4 e Eclipse Temurin 17 e executadas no Temurin 17 JRE. A maioria dos módulos de avaliação declara Weka 3.8.0; o `pom.xml` do serviço SU contém declarações duplicadas para 3.8.6 e 3.8.0, de modo que a

árvore de dependências resolvida deve ser arquivada. Os DRG usam Spring Boot 3.0.0 e os DLS, Spring Boot 2.7.5. A infraestrutura fixa as seguintes imagens:

- `<confluentinc/cp-kafka:7.4.0>;`
- `<confluentinc/cp-zookeeper:7.4.0>;`
- `<ghcr.io/kafbat/kafka-ui:v1.4.2>;`
- `<postgres:14>;`
- `<redis:7-alpine>.`

O modo opcional que executa a camada web em contêiner usa `node:24`; o projeto declara Express 4.21.1, React 18.2.0 e Vite 8.0.3. Os arquivos Compose não impõem limites de CPU e RAM à API e ao cliente web iniciados pelos scripts; esses limites também precisam ser fixados em uma reprodução de desempenho.

4.7.1 Manifesto da campanha formal

O identificador da campanha formal é `gfshield-10d-2026-v1`. Sua *tag* é `experiment-10d-v1`, e o *commit* de lançamento é `66565b865a7b`. As imagens foram construídas do *commit* `b38eff3ed9c1`; o manifesto preserva o ID SHA-256 de cada imagem local e os *digests* de Kafka 7.4.0 e ZooKeeper 7.4.0. O manifesto, o Compose resolvido e a proveniência do hospedeiro possuem hashes próprios. Cada execução também arquiva comando, Compose resolvido, logs, métricas, telemetria, resultado final e um manifesto de checksums.

O arquivo `<experiments/10-day-campaign/frozen-compose.yaml>` fixa os serviços usados no protocolo. Cada configuração distribuída inicia um gerador de RCL, VND ou RVND, os três operadores BF/IWSS/IWSSR, o verificador, Kafka e ZooKeeper. Há um consumidor por serviço e uma partição lógica de execução; Kafka usa seis partições, fator de replicação um e criação automática de tópicos. Os serviços Java usam Temurin 17; o avaliador comum usa Weka 3.8.6. Os monólitos são imagens separadas, derivadas de `<graspy2>` e `<Graspy>`. Todos os braços usam `-cpuset-cpus 8-15`, `-cpuset-mems 1`, teto agregado de oito CPUs e 16 GiB. O kernel do hospedeiro não oferece contabilização de *swap* ao Docker; o limite de memória foi aplicado, mas um limite de *swap* separado não pôde ser verificado.

Os dados são montados somente para leitura em `</data>` ou `</datasets>`; resultados, logs e métricas usam volumes vinculados específicos de cada `run_id`. As portas dos geradores são publicadas apenas em 127.0.0.1 durante a execução; Kafka e os demais serviços comunicam-se na rede interna da composição. A camada web não integra o caminho medido da campanha.

O lançamento durável foi realizado no servidor por:

```
python3 experiments/10-day-campaign/campaign_supervisor.py \
  --manifest experiments/10-day-campaign/manifest.yaml \
  --state experiments/10-day-campaign/state/campaign-state.json \
```

```
--supervisor-state experiments/10-day-campaign/state/supervisor-state.json \  
--log experiments/10-day-campaign/logs/campaign-supervisor.log \  
--output-root experiments/10-day-campaign/results
```

O supervisor preserva o prazo absoluto, verifica espaço livre, encerra grupos de processos e limita reinícios. Na execução reportada, ele não reiniciou. A campanha produziu 216 resultados válidos para 27 braços e oito sementes, além de 12 tentativas inválidas preservadas e repetidas. O repositório contém o analisador `<experiments/10-day-campaign/analyze_results.py>`, que verifica checksums, normaliza as métricas e recria tabelas e figuras. Reproduzir os resultados ainda depende de acesso às imagens ou de reconstruí-las no *commit* registrado, ao corpus com os hashes indicados e a um hospedeiro compatível; frequências dinâmicas de CPU, contêineres externos e limitações do hardware físico não são eliminados pela containerização.

4.8 Síntese

O *G-FShield* integra ramos independentes de construção GRASP-FS, controladores e operadores de busca local, mensageria e verificação de melhores resultados observados. Uma camada web consulta dados mantidos em armazenamento persistente. O contrato de entrada, o pseudocódigo, o manifesto e a configuração dos contêineres delimitam implementação e campanha. A arquitetura oferece desacoplamento, inspeção e evolução; a campanha estabelece rastreabilidade experimental, enquanto escalabilidade, tolerância a falhas e portabilidade entre hospedeiros permanecem propriedades a avaliar.

5 AVALIAÇÃO

Este capítulo apresenta a campanha experimental formal, limitada a dez dias, executada no mesmo servidor para o G-FShield, os dois monólitos e um baseline com todas as características. A campanha de 27 braços é identificada pelo manifesto cujo SHA-256 é:

29fdc973ad6b243b297e34fcd539f5826aac4ce8a0870e8a5cea5caacfc9adc4.

Os CSVs históricos de `tmp_metrics/`, nos quais havia a divergência entre 25.873 linhas brutas e 25.836 linhas com F1 válido, não são combinados com a nova campanha. Assim, nenhuma dessas contagens é usada como resultado comparativo.

5.1 Protocolo Experimental

O corpus ERENO preservado possui 199.998 instâncias, 51 preditores e nove classes. Uma divisão estratificada, fixada antes das buscas com semente 20260825, produziu 119.998 instâncias de treino, 40.000 de validação e 40.000 de teste. Os hashes SHA-256 da origem, treino, validação e teste são, respectivamente, 60ebfcdc9c20, 5876bf5570b7, c44a00ca554c e 05d393c5881a; os valores integrais constam do manifesto. O teste permaneceu indisponível para a busca e foi avaliado uma única vez depois da escolha da solução pela validação.

Todas as soluções finais foram reavaliadas pela mesma imagem com Weka J48 3.8.6, árvore podada, fator de confiança 0,25 e mínimo de duas instâncias por folha. A métrica primária é F1 macro na escala de 0 a 1; precisão e revocação macro, F1 ponderado, acurácia e F1 por classe foram preservados como medidas complementares.

A matriz possui 27 braços: 24 configurações distribuídas, dois monólitos e o baseline. Cada braço usa as sementes independentes 42–49, totalizando 216 execuções válidas. As configurações distribuídas cruzam quatro rankings da RCL (IG, GR, SU e ReliefF), dois controladores (VND e RVND) e três preferências iniciais de busca local (BF, IWSS e IWSSR). Os três operadores continuam habilitados no portfólio de cada controlador. ReliefF usa 1.000 instâncias de referência escolhidas de forma reproduzível no treino pela semente da execução; treino, validação e teste do classificador usam as divisões completas.

Cada execução algorítmica recebeu até 3.000 s, com reserva de 300 s para encerramento e avaliação final. A busca também poderia parar após 500 melhorias aceitas, considerando melhora mínima de 0,0001. O G-FShield usa corte 30, solução inicial de cinco atributos e 100 iterações por operador. O Monólito 1 executa GR seguido de Bit-Flip, com corte 30, cinco atributos e 100 iterações locais. O Monólito 2 executa GR seguido de IWSS sob VND, com os mesmos corte e tamanho inicial e 100 iterações locais. O baseline apenas treina e avalia J48 com os 51 preditores. Em cada caso, treino orienta o ajuste, validação escolhe a melhor solução e teste fornece a medida final comum.

Todos os braços foram confinados aos processadores lógicos 8–15 do nó NUMA 1, com teto agregado de oito CPUs e 16 GiB. Os contêineres foram amostrados a cada 30 s por `docker stats`; CPU corresponde à soma em núcleos e memória à soma dos contêineres pertencentes à execução. Outros contêineres presentes no host foram registrados separadamente e não entram nessas somas. A campanha começou em 25 de agosto de 2026 às 08:32 UTC e terminou em 1 de setembro de 2026 às 03:36 UTC, antes do limite de dez dias, sem reinício do supervisor.

Doze tentativas distribuídas terminaram sem solução completa (`<NO_COMPLETE_SOLUTION>`) e foram rejeitadas pelo contrato de artefatos. O orquestrador repetiu somente as combinações semente–braço ausentes até obter oito resultados válidos por braço. Foram, portanto, 228 tentativas e 216 execuções válidas. As 208 execuções de busca terminaram pelo limite de tempo; as oito execuções do baseline terminaram normalmente.

Neste capítulo, *execução* é uma réplica válida e independente de um braço e uma semente. *Solução final* é o subconjunto escolhido na validação e avaliado uma vez no teste. *Registro interno* é uma linha ou evento de candidato produzido dentro dessa execução; registros internos são correlacionados e não são tratados como réplicas nem como soluções ponta a ponta.

5.2 Qualidade Preditiva e Redução Dimensional

A Tabela 5 resume a configuração G-FShield escolhida pelo maior F1 macro mediano de validação, os monólitos e o baseline. Para cada braço, as estatísticas usam oito execuções. RF–VND–IWSSR obteve F1 macro mediano de teste 0,9448, com intervalo interquartil 0,9441–0,9449, 11 atributos e redução de 78,4%. O baseline com todos os atributos obteve 0,9378. A diferença mediana pareada foi 0,0069 em favor da configuração selecionada; trata-se de resultado descritivo, não de superioridade estatística demonstrada.

Tabela 5 – Resultados finais no teste reservado e recursos observados (medianas de oito execuções).

Braço	F1 macro	Precisão	Revocação	Acurácia	S	Redução (%)	Tempo (s)	CPU (núcleos)	RAM med. (MiB)	Pico RAM (MiB)
RF-VND-IWSSR	0,9448	0,9463	0,9432	0,9343	11	78,4	2.815,3	2,104	4.726,0	5.858,3
Monólito 1 (GR-BF)	0,9397	0,9399	0,9396	0,9293	5	90,2	2.712,5	1,069	1.531,4	1.959,4
Monólito 2 (GR-IWSS)	0,8411	0,8571	0,8454	0,8412	5	90,2	2.725,9	1,003	1.394,7	1.788,9
Todas as 51 características	0,9378	0,9402	0,9358	0,9272	51	0,0	72,5	1,044	748,4	820,0

F1, precisão e revocação são macro. CPU é a mediana temporal da soma dos contêineres por execução e depois a mediana entre sementes. RAM mediana e pico seguem a mesma delimitação. O tempo é ponta a ponta, incluindo inicialização, busca, seleção por validação, encerramento controlado e avaliação final.

O Monólito 1 obteve F1 mediano 0,9397; o Monólito 2 apresentou mediana 0,8411 e alta dispersão (média 0,7343; desvio-padrão 0,2013). O baseline é determinístico sob a configuração usada e repetiu 0,9378 nas oito sementes. Entre as 24 configurações distribuídas, o F1 mediano variou de 0,8671 a 0,9448; a mediana entre configurações foi 0,9418. Todas produziram oito subconjuntos finais distintos. O tamanho mediano variou de 5 a 17 atributos, equivalente a reduções de 90,2% a 66,7%.

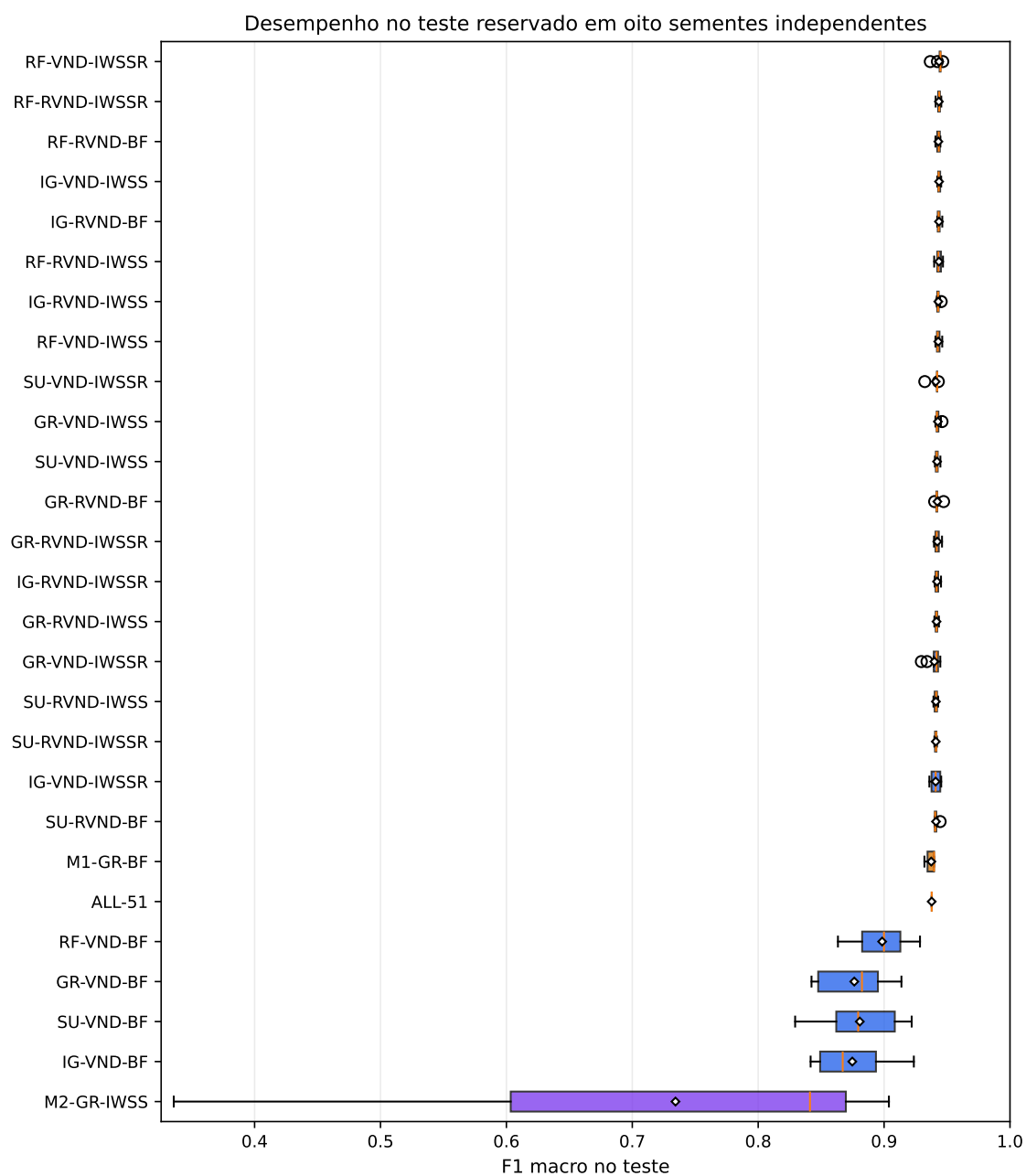


Figura 11 – Distribuição do F1 macro no teste para os 27 braços e oito sementes por braço.

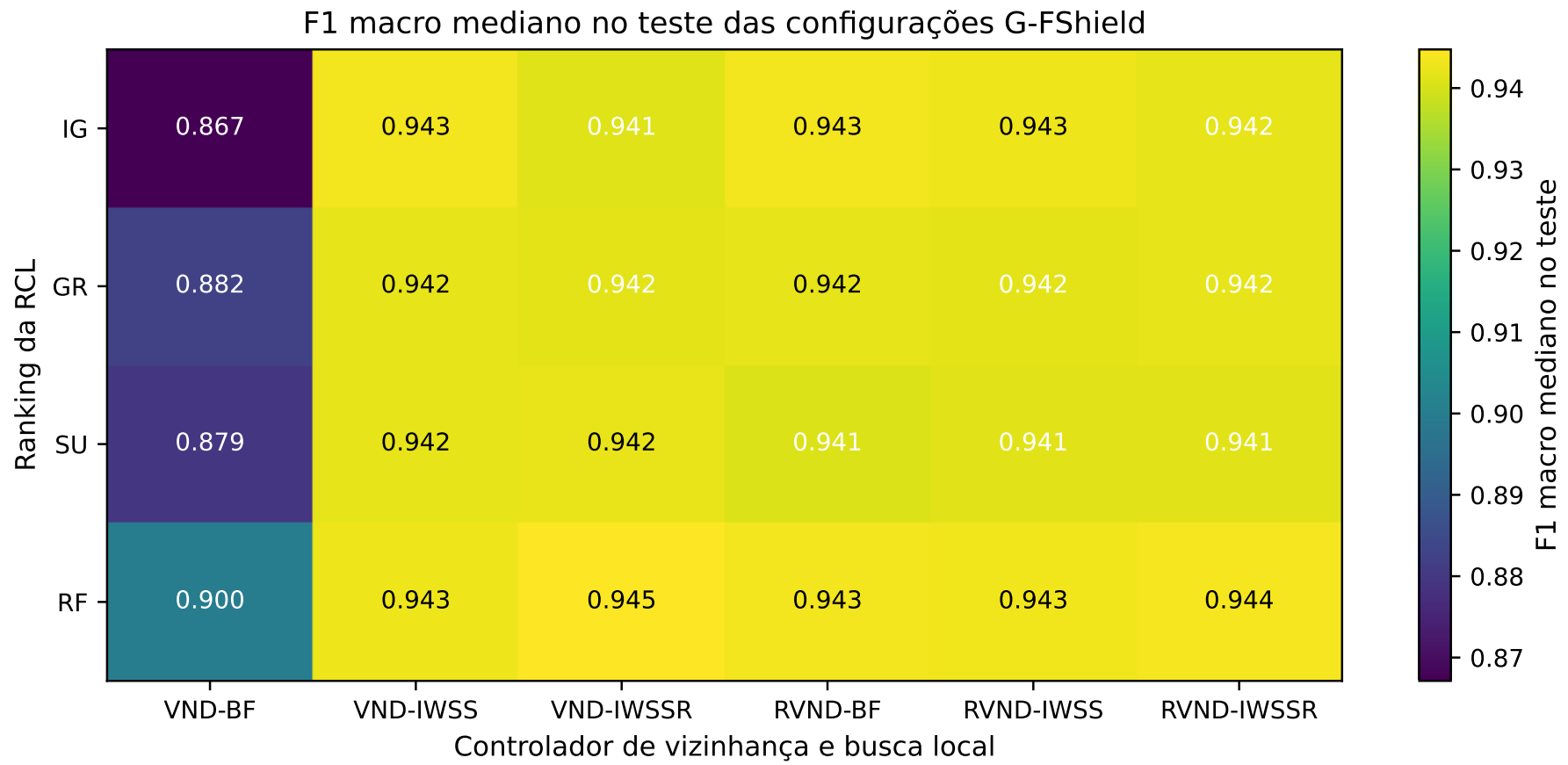


Figura 12 – F1 macro mediano no teste das 24 configurações distribuídas.

A Figura 12 evidencia que as quatro combinações VND–BF permaneceram abaixo das configurações que efetivamente exploraram IWSS/IWSSR. Esse resultado depende do protocolo e do objetivo de busca implementado; não estabelece que um operador seja universalmente inferior. A relação entre qualidade e redução aparece na Figura 13. RF–VND–IWSSR supera descritivamente o baseline em F1 com 40 atributos a menos, enquanto o Monólito 1 usa somente cinco atributos e preserva F1 próximo ao baseline.

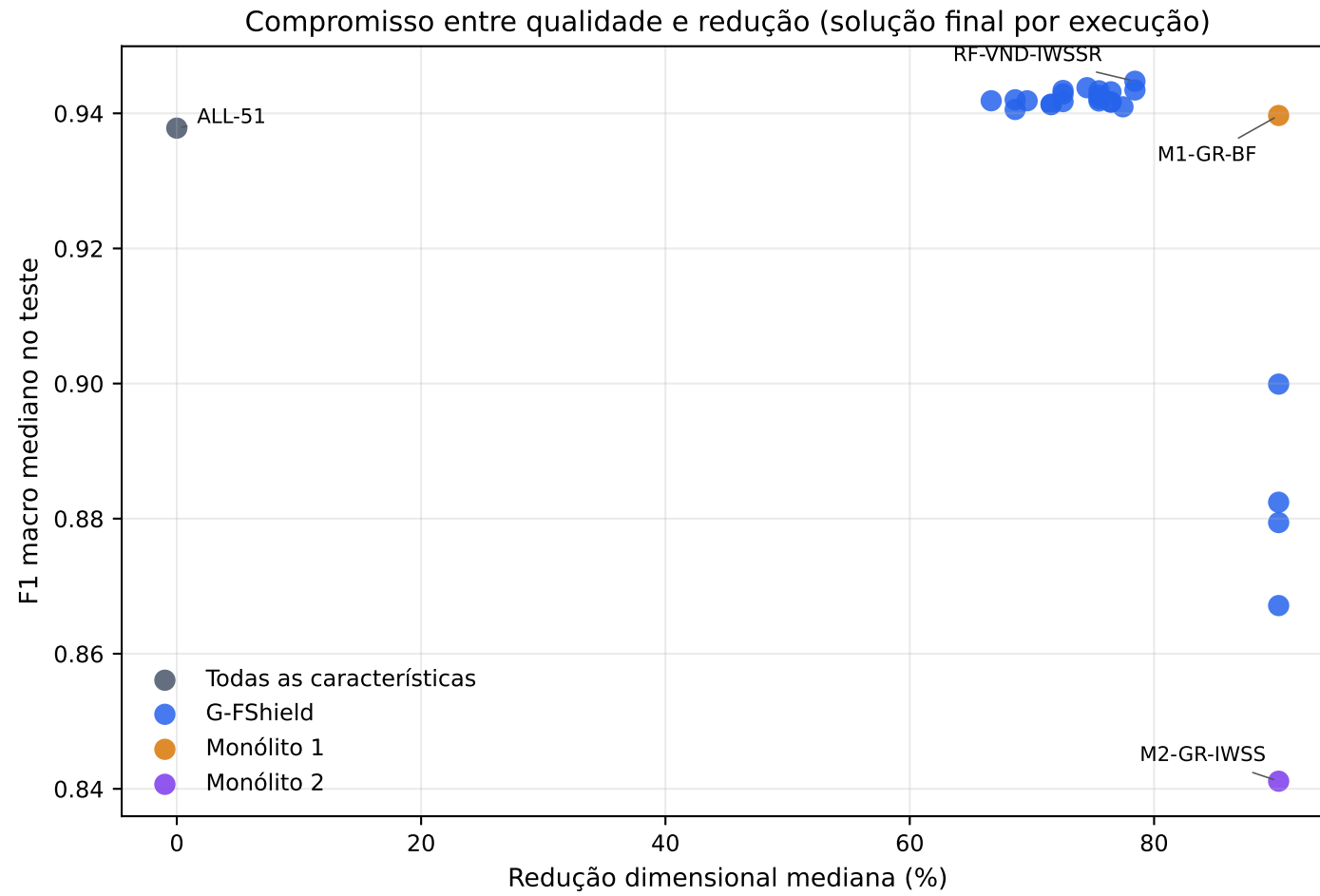


Figura 13 – Compromisso entre F1 macro mediano e redução dimensional das soluções finais.

O desempenho por classe (Figura 14) revela a informação ocultada pela média: para RF-VND-IWSSR, as medianas de F1 são 0,7179 em *grayhole*, 0,8451 em *normal* e pelo menos 0,9565 nas demais sete classes. Portanto, o F1 macro próximo de 0,945 não significa desempenho uniforme entre ataques.

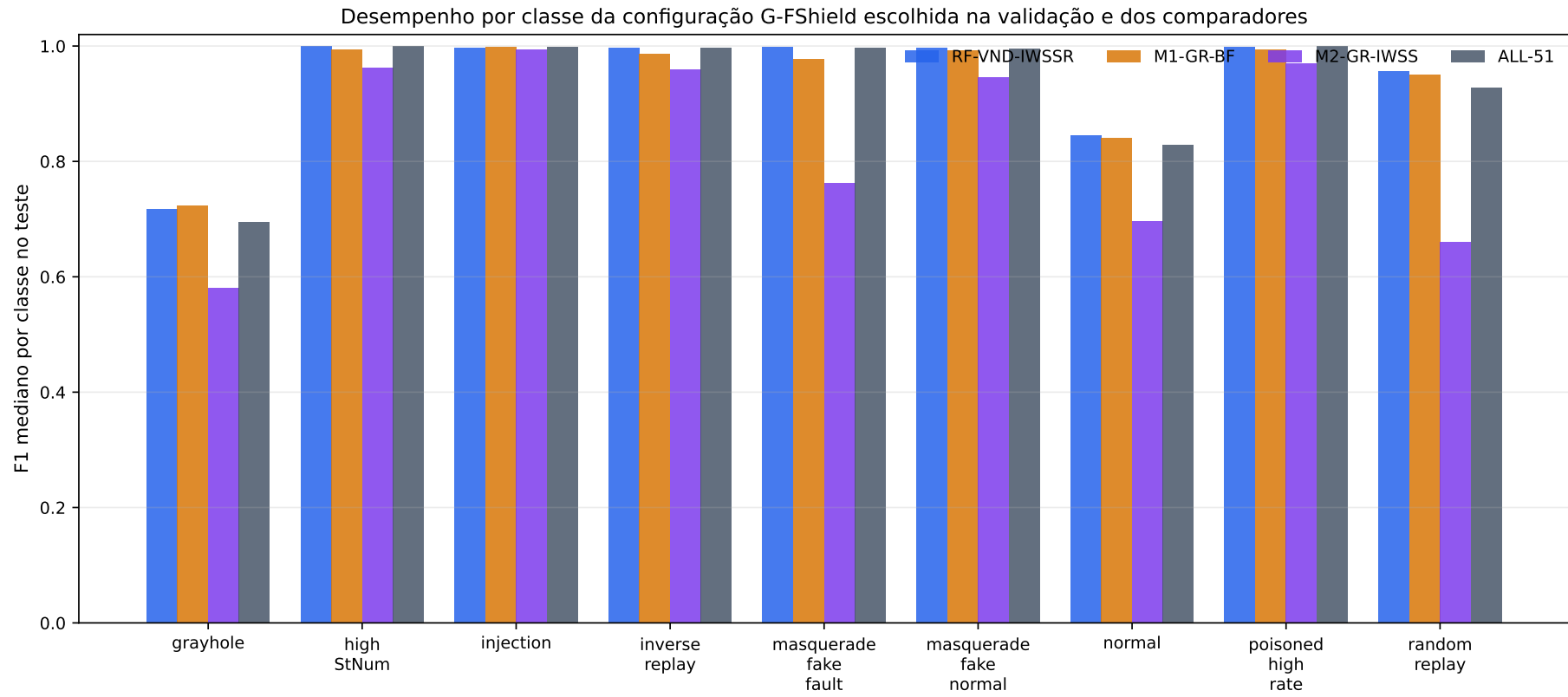


Figura 14 – F1 mediano por classe no teste para a configuração distribuída escolhida na validação e os comparadores.

5.3 Tempo, Recursos e Interpretação Estatística

Como as 208 buscas terminaram pelo mesmo limite, o tempo mediano ponta a ponta ficou entre 2.706 e 2.847 s por configuração. Esses valores não demonstram convergência mais rápida. Nenhuma das 24 configurações distribuídas atingiu $F1 \text{ interno} \geq 0,95$. A auditoria do commit que originou as imagens confirmou que esse escore interno já é $F1 \text{ macro multiclasse}$; a rotina binária com `normalClass=0` estava desabilitada. Ainda assim, a campanha não preservou uma linha do tempo cumulativa equivalente para cada candidato dos dois monólitos: seus CSVs registram a duração individual da avaliação, não o instante desde o início da execução. Portanto, não se calcula uma comparação válida de tempo até o limiar a partir dessa campanha. Esse desfecho requer timestamps monotônicos comparáveis nos dois braços, avaliação apenas na validação e censura no mesmo prazo.

Sob o mesmo teto agregado, as execuções distribuídas usaram mediana global de 3,56 núcleos e pico mediano de 5,45, contra aproximadamente 1,07/1,83 no Monólito 1 e 1,00/1,82 no Monólito 2. A memória distribuída teve mediana global de 5.317 MiB e pico mediano de 6.757 MiB, contra picos medianos de 1.959 e 1.789 MiB nos monólitos. Isso quantifica consumo observado, mas não prova eficiência: o pipeline distribuído agrega vários processos Java e infraestrutura de mensageria, enquanto os braços comparados executam rankings e operadores de busca diferentes.

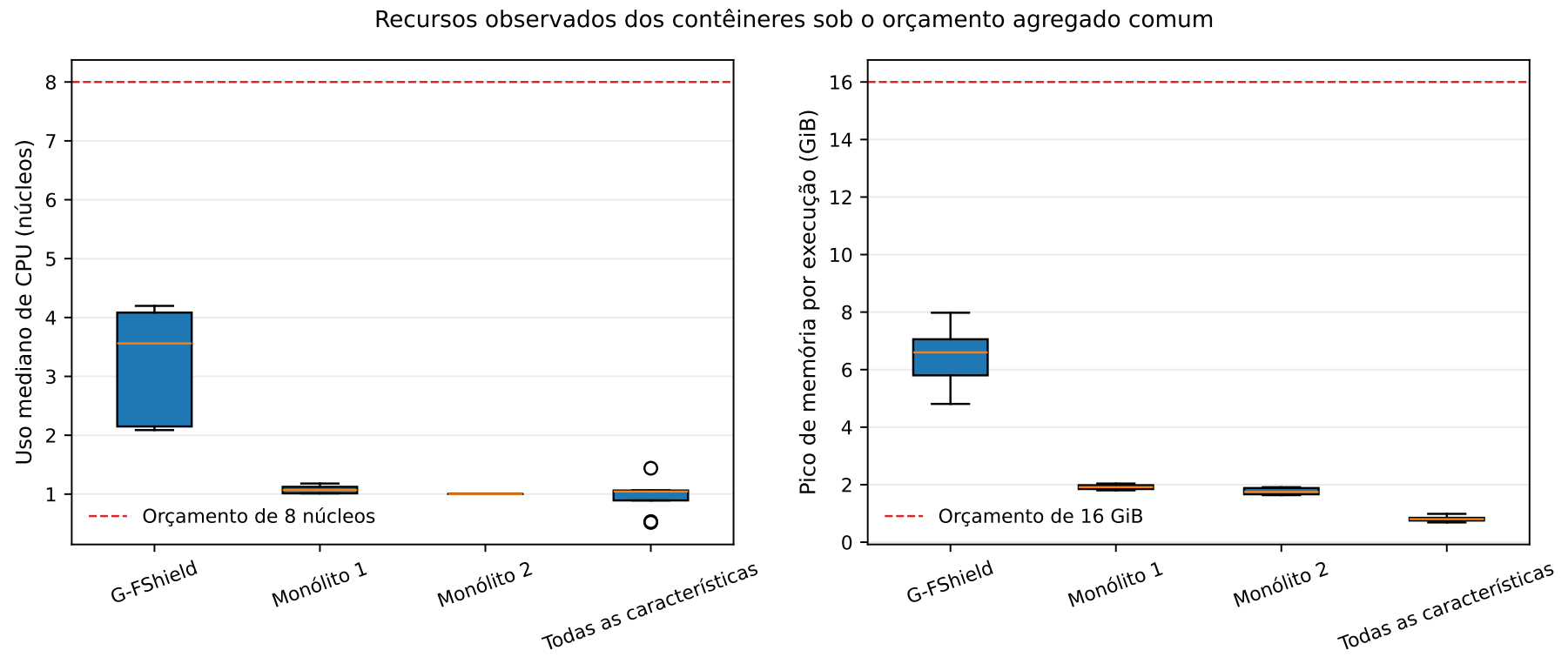


Figura 15 – CPU e memória observadas por arquitetura sob o teto agregado comum de oito CPUs e 16 GiB.

Como análise exploratória, cada configuração distribuída foi comparada aos três comparadores por semente com teste exato bilateral de inversão de sinais e tamanho de efeito bisserial por postos. A correção de Holm foi aplicada separadamente à família de 24 comparações de cada comparador. Para RF-VND-IWSSR, houve sete vitórias e uma derrota diante do baseline e oito vitórias diante de cada monólito; os três valores p ajustados foram 0,1875. Nenhuma das 72 comparações ajustadas ficou abaixo de 0,05. Com oito sementes, os intervalos e testes têm baixo poder; os resultados sustentam diferenças descritivas, não afirmações de significância ou causalidade arquitetural.

5.4 Experimento Causal Pareado da Arquitetura

Para separar arquitetura de algoritmo, uma segunda campanha comparou duas implementações da mesma sequência RF-VND-IWSSR: o pipeline G-FShield e um executável monolítico sequencial. Foram fixados a especificação dos operadores, Weka J48, função objetivo, divisões de dados, parâmetros, sementes, prazo de seleção de 2.700 s e teto agregado de seis CPUs e 12 GiB. O braço distribuído empregou três consumidores para permitir que uma construção prosseguisse enquanto outra solução estava em busca local; o monólito executou as mesmas fases em sequência. A ordem AB/BA foi balanceada entre as sementes 42–71. Como as implementações usam runtimes distintos, efeitos residuais de implementação não podem ser completamente excluídos.

O protocolo pré-especificou como desfecho primário o tempo censurado até F1 macro de validação $\geq 0,94$, com diferença pareada definida como distribuído menos monólito e teste de permutação unilateral. A interpretação favorável também exigia que o limite inferior unilateral de 95% da diferença média de F1 macro no teste não fosse inferior a $-0,005$. Candidatos foram pontuados somente na validação; o teste reservado foi consultado uma vez para a solução final. A campanha terminou com 60 execuções válidas, ou 30 pares. Sete tentativas distribuídas anteriores falharam durante a inicialização de Kafka/ZooKeeper e foram rejeitadas e repetidas pelo contrato; nenhum dos 60 manifestos válidos apresentou erro de checksum.

Tabela 6 – Campanha causal: medianas por arquitetura e comparações pareadas em 30 sementes.

Medida	G-FShield	Monólito	Diferença pareada mediana	Resultado
F1 macro no teste	0,945529	0,944699	+0,000017	15 vitórias, 10 empates, 5 derrotas
Tempo até $F1 \geq 0,94$ (s)	940,077	701,255	+422,740	$p = 0,000795$; distribuído mais lento
AUC <i>anytime</i> normalizada	0,879884	0,897069	-0,016898	$p_{Holm} = 0,000055$
Candidatos por segundo	0,1074	0,0739	+0,0317	30/30 pares; $p_{Holm} = 0,000055$
CPU mediana utilizada (núcleos)	4,076	1,004	+3,070	30/30 pares
CPU-h por candidato	0,010875	0,003821	+0,007095	0/30 pares favoráveis ao distribuído
GiB-h por candidato	0,012366	0,004904	+0,007397	0/30 pares favoráveis ao distribuído
Sobreposição construção-busca local	99,998%	0%	+99,998 p.p.	30/30 pares; $p_{Holm} = 0,000055$
Redução dimensional	78,43%	76,47%	+1,96 p.p.	18 vitórias, 12 empates; $p_{Holm} = 0,000080$

O desfecho primário contradiz a hipótese de menor latência para uma única requisição: o G-FShield demorou mais para atingir 0,94 em 23 dos 30 pares. A diferença média pareada de F1 no teste foi $-0,001759$, e o limite inferior unilateral de 95% foi $-0,005615$; como ultrapassou por pouco a margem negativa pré-especificada de $-0,005$, a salvaguarda de não inferioridade também não foi satisfeita. O limiar 0,95 foi censurado em todos os pares e não distingue as arquiteturas.

Em contrapartida, a decomposição produziu o mecanismo arquitetural pretendido: praticamente toda a busca local ocorreu em paralelo à construção, enquanto a execução monolítica permaneceu sequencial. Isso elevou a vazão mediana de avaliações de candidatos em aproximadamente 45%, mas exigiu cerca de quatro vezes mais CPU simultânea e maior custo de CPU e memória por candidato. A Figura 16 mostra conjuntamente esses efeitos. Portanto, “melhor” é uma conclusão dependente do objetivo: o G-FShield é superior para paralelismo observável, vazão bruta e, neste protocolo, redução dimensional; o monólito é superior para latência até 0,94 e eficiência por unidade de recurso. Não se demonstrou superioridade global da arquitetura distribuída.

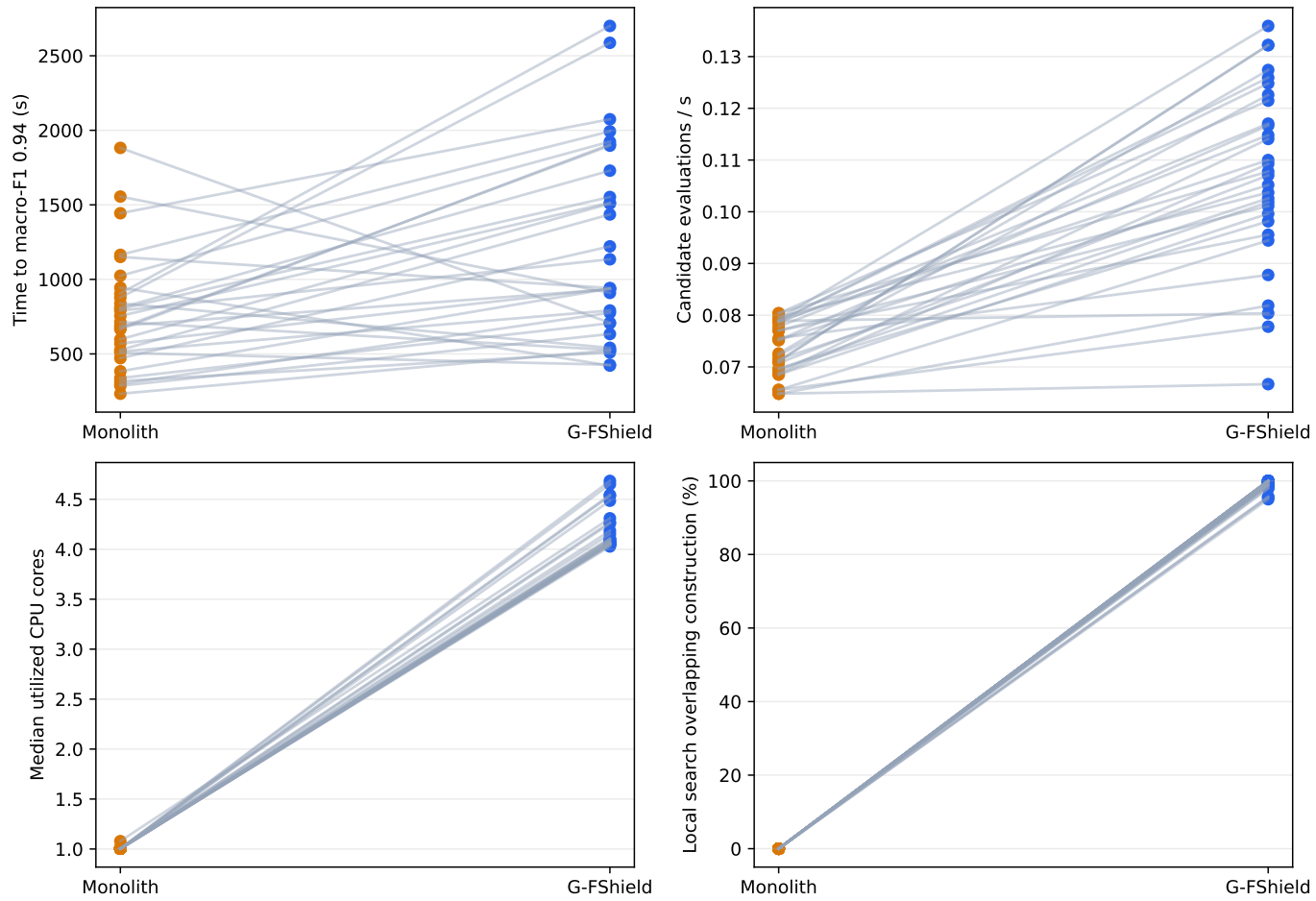


Figura 16 – Comparações pareadas de tempo até F1 macro 0,94, vazão de candidatos, CPU utilizada e sobreposição entre construção e busca local.

A trajetória mediana da melhor validação (Figura 17) explica o resultado: o monólito alcança cedo soluções boas, ao passo que a maior vazão distribuída não se converte automaticamente em progresso de qualidade. A inspeção posterior sugeriu contar subconjuntos *distintos* com F1 de validação $\geq 0,945$ por prazo como uma medida de rendimento de soluções de alta qualidade. Como esse limiar foi escolhido após observar estes dados, ele não é usado como confirmação nesta campanha; foi congelado para uma nova campanha independente com sementes 72–101.

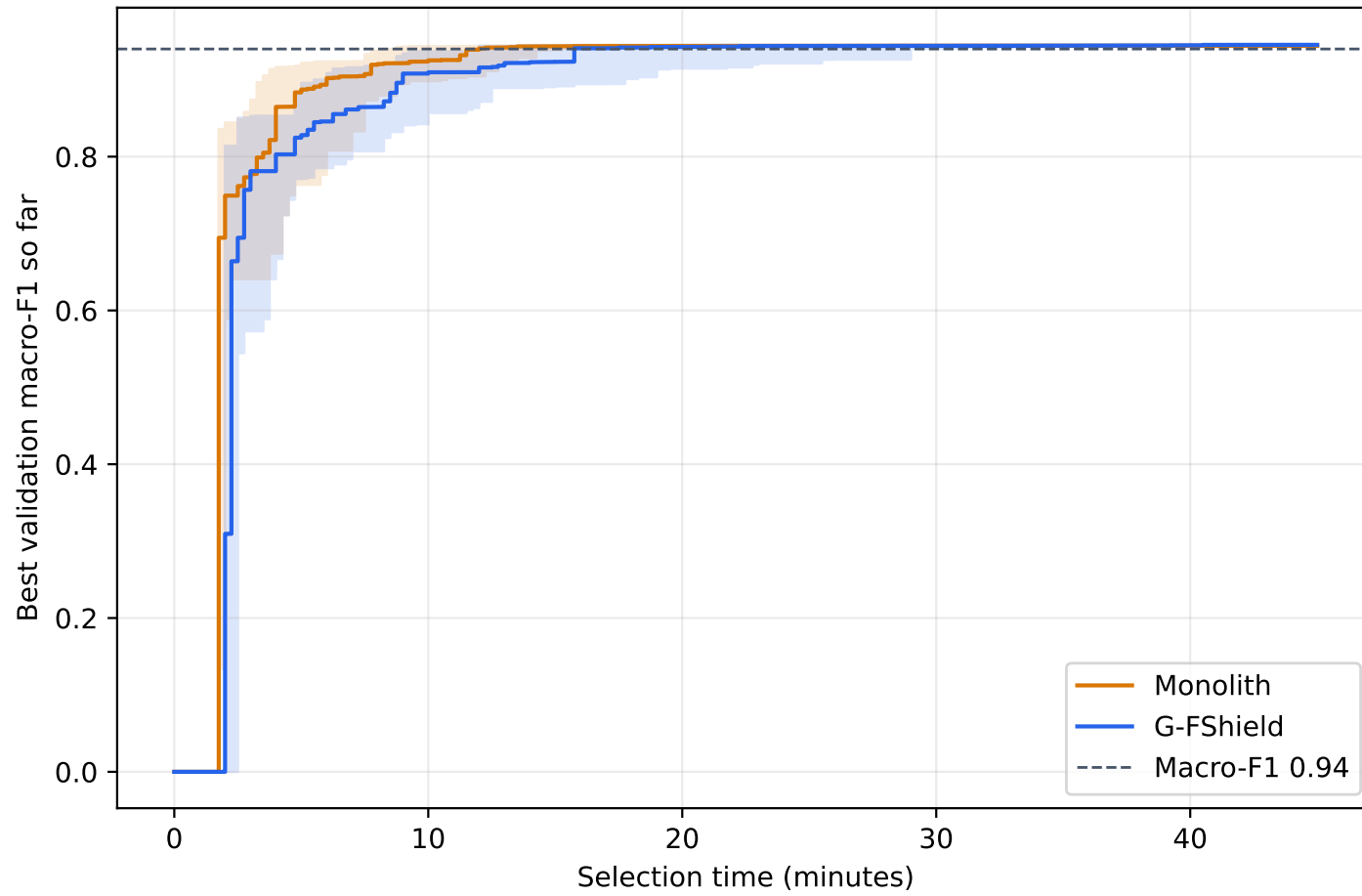


Figura 17 – Trajetória mediana da melhor solução na validação durante os 45 minutos de seleção. As faixas representam o intervalo interquartil entre sementes.

5.5 Confirmação Independente do Rendimento de Qualidade

A observação exploratória da campanha causal originou uma hipótese nova, congelada antes de examinar dados adicionais: no mesmo prazo, o pipeline distribuído produziria mais subconjuntos *distintos* com F1 macro de validação $\geq 0,945$ que o fluxo monolítico. A campanha confirmatória empregou sementes independentes 72–101, a mesma sequência RF–VND–IWSSR, dados, classificador, parâmetros, prazo de seleção de 2.700 s, teto agregado de seis CPUs e 12 GiB e ordem AB/BA balanceada. As 60 execuções planejadas terminaram válidas, sem repetição. O SHA-256 do estado analisado é 301229f7e46f67d093d5c43d36a134346c1b1e136b1dace53700b0e6438f34c8.

O desfecho primário foi a diferença pareada na contagem de subconjuntos qualificados distintos. O G-FShield obteve mediana 1,0 e o monólito 0,0. A diferença média pareada foi +0,933 subconjunto por execução, com IC bootstrap de 95% de 0,233 a 1,667 e teste unilateral pareado de permutação $p = 0,010135$. Houve 13 vitórias, 12 empates e cinco derrotas do distribuído. A diferença mediana foi zero por causa da quantidade de empates; por isso, a interpretação acompanha o estimando médio e o teste pré-especificados, sem ocultar a distribuição dos pares.

O limite inferior unilateral de 95% da diferença média de F1 macro no teste foi $-0,003791$, acima da margem de não inferioridade $-0,005$. A sobreposição mediana entre construção e busca local foi 99,37% no G-FShield e 0% no monólito. Assim, foram satisfeitos conjuntamente o desfecho primário, a salvaguarda de qualidade e a confirmação do mecanismo. A Figura 18 apresenta os pares; pontos acima da diagonal favorecem o G-FShield.

Esse resultado demonstra uma vantagem arquitetural delimitada: sob o mesmo teto e prazo, a execução distribuída aumentou o rendimento médio de soluções distintas de alta qualidade sem violar a margem pré-especificada de qualidade final. Ele não apaga o resultado de latência da campanha anterior. O monólito continuou melhor para obter a primeira solução em 0,94 e para custo por candidato; o G-FShield foi melhor para sobreposição de estágios, vazão de candidatos e rendimento de soluções qualificadas. Portanto, a conclusão é multidimensional, não uma alegação de superioridade universal.

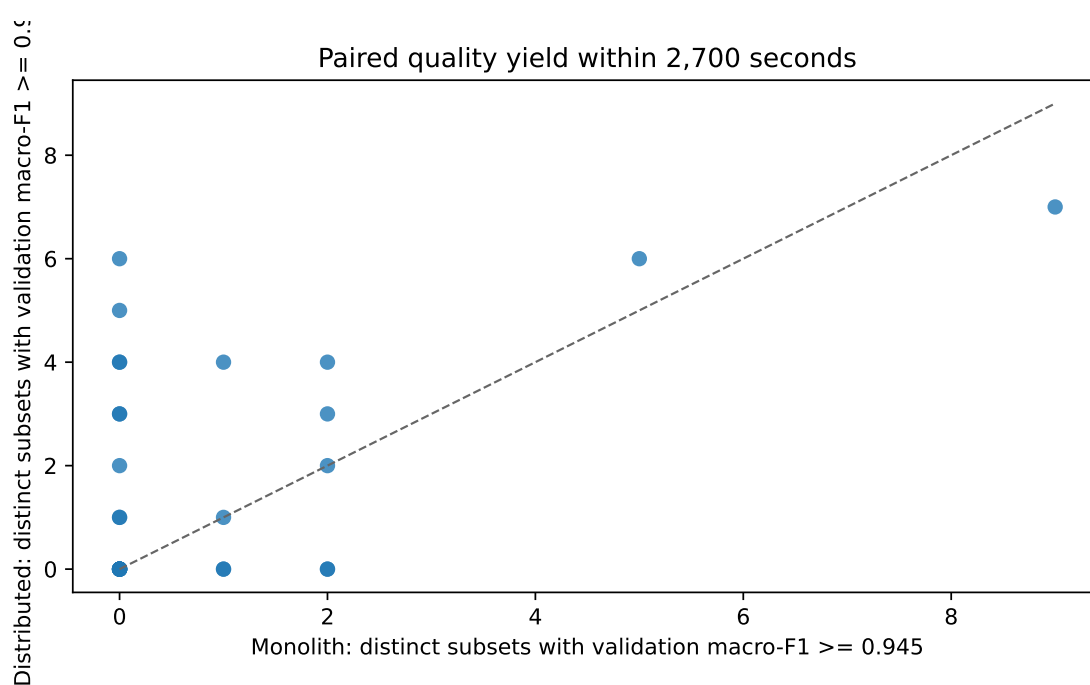


Figura 18 – Contagem pareada de subconjuntos distintos com F1 macro de validação $\geq 0,945$ em 2.700 s, nas 30 sementes confirmatórias.

5.6 Replicação no X-CANIDS e Comparação com o IWSHAP

Para avaliar se o compromisso arquitetural se mantém em outro espaço de atributos e permitir comparação externa, realizou-se uma replicação nos dois cenários públicos do X-CANIDS usados pelo IWSHAP (SCHERER et al., 2024). O código público foi fixado no commit `fb0d3093c12421d08ab3fb595d20c29ba2442e65`. Cada cenário contém 20.000 registros e 688 preditores. A divisão controlada 60/20/20 foi feita por grupos de vetores completos idênticos, de modo que duplicatas não atravessassem treino, validação e teste. Suspensão resultou em 12.081/3.892/4.027 registros e fabricação em 11.944/3.994/4.062, respectivamente.

A execução do código original, com pandas 2.2.2, NumPy 1.26.4, SHAP 0.45.1, XGBoost 2.0.3 e scikit-learn 1.5.0, reproduziu em fabricação o F1 da classe positiva 0,787387 e o subconjunto de quatro atributos do log histórico. Em suspensão, obteve F1 positivo 0,635693 com dois atributos, diferente do 0,918619 e dos 16 atributos registrados no log de junho. Este log não informa hash nem número de registros, logo não se presume que corresponda ao arquivo de 20.000 linhas. Além disso, o programa original codifica categorias antes da divisão 80/20 e reutiliza os mesmos 20% durante a seleção e no resultado reportado; esses valores são evidência de reprodução, não teste intocado.

Uma nova campanha comparou o G-FShield e o Monólito 2 com a mesma sequência ReliefF–VND–IWSSR. Foram usadas 30 sementes pareadas por cenário, janela de seleção de 1.080 s dentro do limite de 1.200 s, até 50 melhorias aceitas, três consumidores no distribuído e teto comum de seis CPUs e 12 GiB. As 120 execuções terminaram válidas. Os 120 manifestos e os 1.500 arquivos por eles enumerados foram recalculados por SHA-256 sem divergência. A Tabela 7 mostra que a replicação confirmou o mecanismo concorrente e a maior vazão IWSSR, mas também confirmou o custo de recursos e a ausência de vantagem de latência.

Tabela 7 – Replicação arquitetural no X-CANIDS: medianas de 30 sementes por arquitetura e cenário.

Cenário	Arquitetura	F1 macro teste	Tempo ao baseline (s)	IWSSR/s	CPU-s	RAM pico (MiB)	Sobreposição
Suspensão	G-FShield	0,790717	114,463	8,580	3.853,1	3.370,3	99,657%
Suspensão	Monólito	0,787396	92,071	3,698	1.100,7	770,1	0%
Fabricação	G-FShield	0,902245	101,047	8,302	3.858,4	3.169,8	99,664%
Fabricação	Monólito	0,900280	90,091	3,678	1.096,9	774,6	0%

O F1 macro final não diferiu após correção de Holm: em suspensão, a diferença pareada mediana foi +0,000665 (IC bootstrap de 95% de 0 a +0,005313; $p_{Holm} = 0,438598$); em fabricação, foi zero ($p_{Holm} = 1$). Em contraste, a AUC *anytime* favoreceu o monólito nos 30 pares de ambos os cenários ($p_{Holm} \approx 0,000030$). O tempo até o baseline de validação com todos os atributos também foi menor no monólito, mas é apresentado somente como descrição porque esse limiar, embora calculado antes da campanha, não foi incluído no manifesto congelado. A taxa de avaliações IWSSR foi aproximadamente 2,3 vezes maior no distribuído e a busca local se sobrepôs à construção em mais de 99,65% do tempo, ao custo de cerca de 3,5 vezes mais CPU-segundos e aproximadamente quatro vezes mais RAM de pico.

Para comparar seletores sem confundir classificador e partição, todos os subconjuntos congelados foram retreinados somente no treino e avaliados por um mesmo XGBoost; validação foi usada para análise e o teste intocado foi consultado uma vez. A Tabela 8 separa essa comparação controlada dos números publicados e da reprodução que reutiliza o conjunto de seleção.

Tabela 8 – Subconjuntos avaliados pelo mesmo XGBoost e no mesmo teste intocado. Valores de G-FShield e monólito são medianas de 30 sementes; comparadores fixos têm uma execução.

Cenário	Método	n	Atributos	Redução	F1 macro	F1 positivo	Precisão/Revocação positiva
Suspensão	Todos os atributos	1	688	0%	0,734966	0,532915	0,517241 / 0,549569
Suspensão	Subconjunto histórico IWSHAP	1	16	97,67%	0,830048	0,699894	0,688935 / 0,711207
Suspensão	G-FShield	30	20,5	97,02%	0,723518	0,511074	0,511256 / 0,510776
Suspensão	Monólito pareado	30	21	96,95%	0,723544	0,510315	0,513282 / 0,507543
Fabricação	Todos os atributos	1	688	0%	0,882397	0,795019	0,750452 / 0,845214
Fabricação	Subconjunto histórico IWSHAP	1	4	99,42%	0,882423	0,799667	0,675562 / 0,979633
Fabricação	G-FShield	30	17	97,53%	0,887680	0,803178	0,787196 / 0,820774
Fabricação	Monólito pareado	30	17	97,53%	0,889053	0,805639	0,788703 / 0,823829

Em suspensão, nenhuma das 30 execuções G-FShield superou o subconjunto histórico IWSHAP em F1 macro ou F1 positivo. Em fabricação, 26 de 30 o superaram em F1 macro e 16 de 30 em F1 positivo. O maior valor observado foi F1 macro 0,900958 na semente 20260916 do G-FShield, com 19 atributos e F1 positivo 0,826347. Contudo, a mediana distribuída selecionou 17 atributos contra quatro do IWSHAP e trocou revocação por precisão: 0,820774/0,787196 contra 0,979633/0,675562. Como o subconjunto IWSHAP é um comparador fixo, essas contagens e diferenças são descritivas, não testes pareados de superioridade. A conclusão externa é, portanto, condicionada ao cenário: há vantagem descritiva de F1 do G-FShield em fabricação, mas não em suspensão, e não há domínio simultâneo em qualidade, redução e revocação.

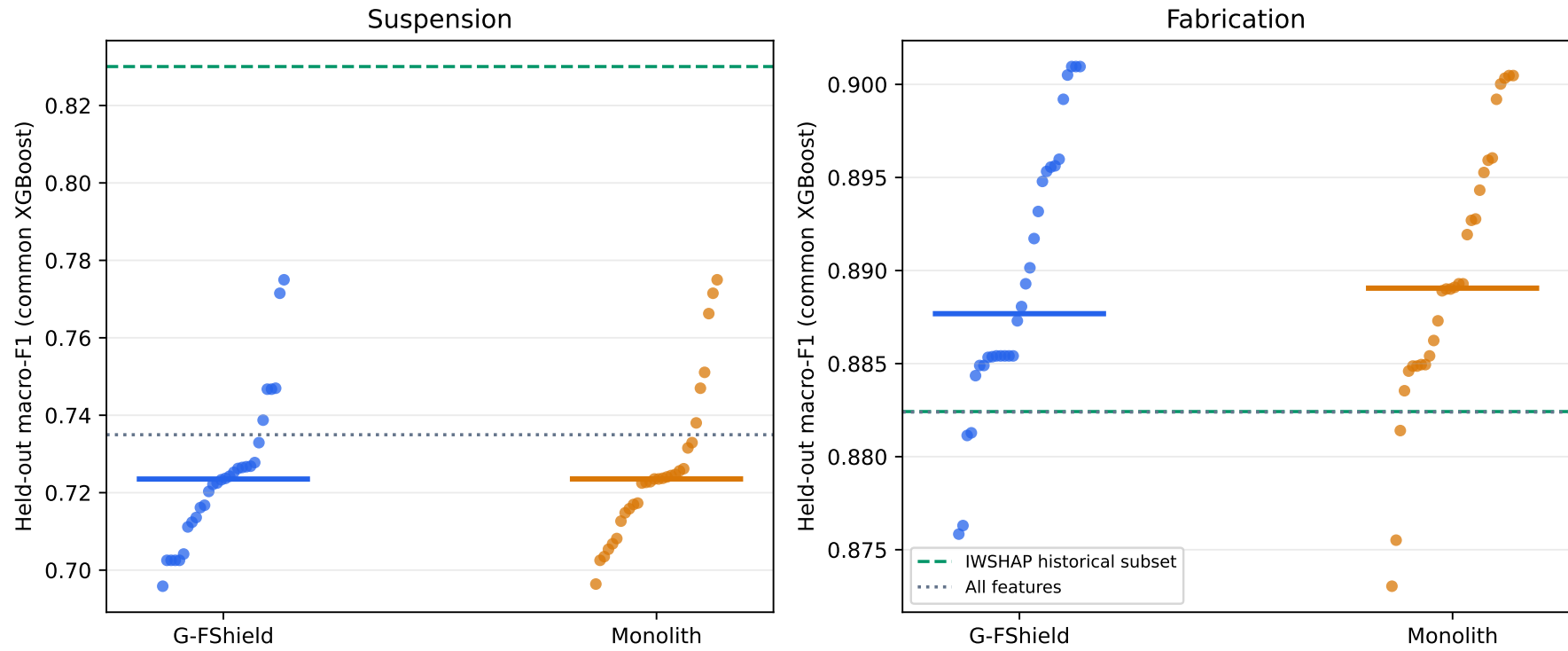


Figura 19 – F1 macro dos subconjuntos congelados sob o protocolo XGBoost comum. Traços horizontais mostram os comparadores fixos com todos os atributos e com os atributos do log histórico IWSHAP.

5.7 Observabilidade e Rastreabilidade

Cada execução preserva `campaign_id`, `run_id`, braço, semente, parâmetros, hashes dos dados, comando resolvido, estados, motivo de parada, timestamps UTC e monotônicos, solução selecionada, métricas finais, mensagens de melhor solução, CSVs internos, logs e telemetria de contêiner. Um manifesto de checksums por execução permite detectar alteração nos artefatos, e o estado da campanha permite reconstruir tentativas, rejeições e repetições.

Na aplicação, o backend persiste eventos de solução inicial, reinício de vizinhança, progresso de busca, solução refinada e melhor solução, além de tópico, partição, *offset*, estado, `seedId`, timestamps e conteúdo JSON quando disponíveis. A campanha formal acrescenta `campaign_id`, `run_id` e `request_id` aos fluxos experimentais, removendo a ambiguidade entre execuções concorrentes nesse protocolo. Ainda não foram medidos completude dos eventos, *overhead* da instrumentação, perda/duplicação sob falha, tempo de recuperação ou sucesso de reprocessamento. Portanto, tolerância a falhas e resiliência permanecem objetivos de projeto.

5.8 Ameaças à Validade

Validade interna. Na primeira campanha, CPU, memória, conjunto de CPUs, dados, sementes, limite, J48 e F1 macro foram controlados, mas os comparadores executaram sequências algorítmicas distintas. O experimento causal posterior corrigiu esse confundimento, registrou trajetórias cumulativas equivalentes e balanceou a ordem AB/BA; a confirmação independente preservou esse desenho com sementes novas. Persistem possíveis efeitos de interferência do host compartilhado e de inicialização: 12 tentativas da primeira campanha e sete tentativas distribuídas da campanha causal foram rejeitadas e repetidas por não produzirem artefato completo. As 60 execuções confirmatórias foram válidas na primeira tentativa. O procedimento preserva a matriz de sementes, mas as duas campanhas anteriores condicionam parte da análise às execuções que inicializaram corretamente.

Validade de construção. F1 macro no teste reservado mede qualidade final; execução, solução final e registro interno são unidades distintas. A campanha confirmatória mede rendimento como o número de subconjuntos canônicos distintos com F1 macro de validação $\geq 0,945$ no prazo. Registros de candidatos permanecem correlacionados dentro da execução e não são tratados como réplicas. CPU e memória medem contêineres, não energia nem custo monetário. Os limiares 0,94 e 0,945 são critérios experimentais pré-especificados, não requisitos operacionais.

Validade de conclusão. Na primeira campanha, oito sementes oferecem baixo poder, o baseline é determinístico e a escolha entre 24 configurações introduz seleção múltipla. Na campanha causal, os 30 pares, o desfecho primário e a salvaguarda de

qualidade foram definidos antes da análise; candidatos internos não foram tratados como réplicas e medidas secundárias usaram correção de Holm. A hipótese de rendimento, gerada por essa campanha, foi testada uma única vez em 30 sementes independentes: o teste unilateral pré-especificado e a salvaguarda de qualidade foram satisfeitos. A mediana da diferença foi zero, e a conclusão de vantagem acompanha explicitamente a diferença média pré-especificada, seu intervalo e as 13 vitórias, 12 empates e cinco derrotas.

Validade externa. A evidência cobre um servidor, um corpus IEC 61850, um particionamento, J48 e uma configuração de pipeline. Não demonstra que a vantagem se generalize para outros corpora, classificadores, números de réplicas, hardware, falhas ou ingestão contínua; tampouco estabelece eficiência de recursos ou escala horizontal. Estudos posteriores devem repetir o protocolo com múltiplos conjuntos de dados e medir curvas de escala e recuperação separadamente.

6 CONCLUSÃO

Esta dissertação apresentou o *G-FShield*, uma arquitetura distribuída, orientada a eventos e observável para GRASP-FS em IDS aplicados a CPS. A implementação separa rankings e construção inicial, controle de vizinhanças, operadores de busca local, verificação e monitoramento; Apache Kafka integra os estágios, enquanto o backend e a interface permitem consultar sinais persistidos da execução.

A campanha formal respondeu à RQ1 com unidades ponta a ponta comuns. Foram avaliados 24 braços G-FShield, dois monólitos e um baseline com todas as características, cada qual em oito sementes independentes. RF-VND-IWSSR foi escolhido pela validação e obteve F1 macro mediano de 0,9448 no teste com 11 dos 51 preditores, redução de 78,4%. O baseline obteve 0,9378, o Monólito 1 0,9397 e o Monólito 2 0,8411. A diferença diante do baseline é descritiva; nenhuma das 72 comparações exploratórias ajustadas por Holm apresentou $p < 0,05$.

Para as RQ2 e RQ3, uma campanha causal adicional removeu o confundimento entre algoritmo e arquitetura: 30 sementes parearam implementações distribuída e monolítica da mesma sequência RF-VND-IWSSR, sob prazo de 2.700 s e teto agregado de seis CPUs e 12 GiB. O G-FShield avaliou 0,1074 candidato/s contra 0,0739 do monólito e sobrepôs praticamente 100% da busca local à construção, contra 0% no fluxo sequencial. Esses dois resultados ocorreram nos 30 pares. A maior vazão, contudo, não produziu menor latência de qualidade: o tempo mediano até F1 macro de validação 0,94 foi 940,1 s no distribuído e 701,3 s no monólito; a diferença pareada mediana foi +422,7 s, com $p = 0,000795$ na direção contrária à hipótese. A AUC *anytime* também favoreceu o monólito.

O paralelismo utilizou mediana de 4,076 núcleos, contra 1,004, e apresentou maior custo de CPU-h e GiB-h por candidato em todos os pares. Em compensação, a redução dimensional mediana foi 78,43% no G-FShield e 76,47% no monólito. Assim, a evidência demonstra a capacidade arquitetural de executar estágios concorrentemente e aumentar vazão bruta, mas não sustenta superioridade global: para uma única requisição, o monólito atingiu o limiar mais cedo e foi mais eficiente por recurso.

Uma campanha confirmatória independente, com sementes 72–101, testou a hipótese gerada pela primeira campanha sem reutilizar suas observações. Sob o mesmo prazo de 2.700 s e teto de seis CPUs e 12 GiB, o G-FShield produziu em média 0,933 subconjunto distinto adicional com F1 macro de validação $\geq 0,945$ por execução (IC95%: 0,233–1,667; $p = 0,010135$). Foram 13 vitórias, 12 empates e cinco derrotas; as medianas foram 1,0 contra 0,0. A salvaguarda de qualidade no teste foi satisfeita, pois seu limite inferior unilateral foi $-0,003791$, acima da margem $-0,005$. A vantagem arquitetural sustentada pelos dados é, portanto, maior rendimento médio de soluções distintas de alta qualidade, além de sobreposição e vazão, e não superioridade em todas as dimensões.

A replicação X-CANIDS ampliou essa evidência para dois cenários com 688 predi-

tores e mais 120 execuções pareadas. O G-FShield novamente sobrepôs mais de 99,65% da busca local à construção e executou aproximadamente 2,3 vezes mais avaliações IWSSR por segundo, mas o monólito manteve melhor AUC *anytime* e menor consumo. Sob um XGBoost comum e teste intocado, o resultado externo dependeu do ataque: em fabricação, o F1 macro mediano do G-FShield foi 0,887680 contra 0,882423 do subconjunto histórico IWSHAP, com 26 de 30 execuções acima do comparador; em suspensão, foi 0,723518 contra 0,830048 e nenhuma execução o superou. Mesmo em fabricação, o G-FShield usou mediana de 17 atributos contra quatro e trocou revocação por precisão. Portanto, há vantagem descritiva de F1 somente nesse cenário, não superioridade geral ao IWSHAP.

Quanto à RQ4, manifesto, comandos resolvidos, hashes, identificadores de campanha e execução, sementes, estados, motivos de parada, timestamps, mensagens, CSVs internos, logs, métricas finais e telemetria permitem reconstruir o que foi capturado. A aplicação também persiste eventos por semente e estágio. Não foram medidos completude ou custo da telemetria, perda e duplicação sob falhas, tempo de recuperação ou reprocessamento; resiliência, tolerância a falhas e escalabilidade permanecem objetivos de projeto.

O confundimento entre arquitetura e algoritmo permanece na comparação ampla de 27 braços, mas foi removido nas campanhas pareadas. As principais ameaças remanescentes são as tentativas distribuídas repetidas após falhas de inicialização e o escopo restrito a um servidor, dois recortes de um corpus, um particionamento agrupado, J48 na comparação arquitetural, XGBoost na comparação externa, uma configuração de concorrência e 30 sementes por cenário e arquitetura. Os resultados não demonstram escalabilidade horizontal, tolerância a falhas ou generalização para outras cargas.

Como trabalhos futuros:

- executar uma ablação do pipeline com um, dois, três e quatro consumidores, preservando o teto agregado, para estimar a contribuição causal do paralelismo;
- repetir a comparação causal com mais sementes e cargas simultâneas independentes;
- executar estudos de *scale-out* que variem réplicas e concorrência e relatem vazão por custo;
- medir completude, *overhead*, recuperação e reprocessamento da telemetria;
- repetir o protocolo com outros corpora, classificadores, particionamentos e hardware.

Em síntese, o G-FShield demonstra uma integração distribuída e rastreável de GRASP-FS e produz um conjunto experimental auditável. A campanha causal confirma sobreposição entre construção e busca local e maior vazão de candidatos; a campanha independente confirma maior rendimento médio de subconjuntos distintos de alta qualidade. O preço observado foi maior consumo por candidato e pior tempo até a primeira

solução em F1 0,94. A contribuição arquitetural é, portanto, uma vantagem demonstrada para paralelismo, vazão e rendimento de qualidade, acompanhada de compromissos mensurados, não uma alegação de superioridade universal ou prontidão operacional.

REFERÊNCIAS

- AFTAB, M. A. et al. Iec 61850 based substation automation system: A survey. **International Journal of Electrical Power & Energy Systems**, v. 120, 2020. Citado 2 vezes nas páginas 14 e 16.
- ALCARAZ, C.; LOPEZ, J. A security analysis for wireless sensor mesh networks in highly critical systems. **IEEE Transactions on Systems, Man, and Cybernetics, Part C: Applications and Reviews**, v. 40, n. 4, p. 419–428, 2010. Citado 2 vezes nas páginas 12 e 16.
- ANDERSON, C. Docker [software engineering]. **IEEE Software**, IEEE, v. 32, n. 3, p. 102–c3, 2015. Citado na página 15.
- BEYER, B. et al. **Site Reliability Engineering: How Google Runs Production Systems**. [S.l.]: O'Reilly Media, 2016. Citado 2 vezes nas páginas 15 e 17.
- CÁRDENAS, A. A.; AMIN, S.; SASTRY, S. Research challenges for the security of control systems. In: **3rd USENIX Workshop on Hot Topics in Security (HotSec 08)**. San Jose, CA: USENIX Association, 2008. Disponível em: <<https://www.usenix.org/conference/hotsec-08/research-challenges-security-control-systems>>. Citado 3 vezes nas páginas 12, 14 e 16.
- DUARTE, A. et al. Variable neighborhood descent. In: MARTÍ, R.; PARDALOS, P. M.; RESENDE, M. G. C. (Ed.). **Handbook of Heuristics**. Cham: Springer International Publishing, 2018. p. 341–367. Citado na página 28.
- ESSEGHIR, M. A. Effective wrapper-filter hybridization through grasp schemata. In: LIU, H. et al. (Ed.). **Proceedings of the Fourth International Workshop on Feature Selection in Data Mining**. Hyderabad, India: PMLR, 2010. (Proceedings of Machine Learning Research, v. 10), p. 45–54. Disponível em: <<https://proceedings.mlr.press/v10/esseghir10a.html>>. Citado na página 16.
- FARIA, N. N. R. et al. Microserviços para geração de rcls no grasp-fs: Uma abordagem escalável para seleção de features na detecção de intrusões em sistemas ciber-físicos. In: **Anais do XXIII Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (SBSeg 2023)**. [S.l.]: Sociedade Brasileira de Computação, 2023. p. 55–68. Citado 2 vezes nas páginas 17 e 23.
- FASEEHA, U. et al. Observability in microservices: An in-depth exploration of frameworks, challenges, and deployment paradigms. **IEEE Access**, v. 13, p. 72011–72039, 2025. Citado na página 17.
- FRANCIS, G. T.; SOURI, A.; İNANÇ, N. A systematic review of metaheuristic based feature selection strategies for cyber-attack detection in the iiot. **Artificial Intelligence Review**, v. 59, 2026. Citado na página 16.
- GUYON, I.; ELISSEEFF, A. An introduction to variable and feature selection. **Journal of Machine Learning Research**, v. 3, n. Mar, p. 1157–1182, 2003. Disponível em: <<https://www.jmlr.org/papers/v3/guyon03a.html>>. Citado 3 vezes nas páginas 12, 14 e 16.

HALL, M. A. **Correlation-based Feature Selection for Machine Learning**. Tese (Doutorado) — University of Waikato, 1999. Disponível em: <<https://hdl.handle.net/10289/15043>>. Citado 3 vezes nas páginas 12, 14 e 16.

INTERNATIONAL ELECTROTECHNICAL COMMISSION. **IEC 61850-7-2: Communication Networks and Systems for Power Utility Automation – Part 7-2: Basic Information and Communication Structure – Abstract Communication Service Interface (ACSI)**. 2.0. ed. [S.l.], 2010. Disponível em: <<https://webstore.iec.ch/en/publication/6015>>. Citado 2 vezes nas páginas 14 e 16.

KONONENKO, I. Estimating attributes: Analysis and extensions of relief. In: BERGADANO, F.; RAEDT, L. D. (Ed.). **Machine Learning: ECML-94**. [S.l.]: Springer Berlin Heidelberg, 1994. (Lecture Notes in Computer Science, v. 784), p. 171–182. Citado na página 14.

KREPS, J.; NARKHEDE, N.; RAO, J. Kafka: A distributed messaging system for log processing. In: **Proceedings of the NetDB Workshop**. [S.l.: s.n.], 2011. Citado 4 vezes nas páginas 15, 17, 22 e 32.

MACKIEWICZ, R. E. Overview of iec 61850 and benefits. In: **2006 IEEE Power Engineering Society General Meeting**. [S.l.]: IEEE, 2006. p. 1–8. Citado 2 vezes nas páginas 14 e 16.

MATEUS-COELHO, N.; CRUZ-CUNHA, M.; FERREIRA, L. G. Security in microservices architectures. **Procedia Computer Science**, Elsevier, v. 181, p. 1225–1236, 2021. Citado na página 17.

MITCHELL, R.; CHEN, I.-R. A survey of intrusion detection techniques for cyber-physical systems. **ACM Computing Surveys**, v. 46, n. 4, p. 1–29, 2014. Citado 3 vezes nas páginas 12, 14 e 16.

MLADENović, N.; HANSEN, P. Variable neighborhood search. **Computers & Operations Research**, v. 24, n. 11, p. 1097–1100, 1997. Citado na página 28.

NEWMAN, S. **Building Microservices**. 2. ed. [S.l.]: O'Reilly Media, 2021. Citado 3 vezes nas páginas 15, 17 e 22.

QUINCOZES, S. E. et al. A survey on intrusion detection and prevention systems in digital substations. **Computer Networks**, v. 184, 2021. Citado na página 16.

QUINCOZES, S. E. et al. Ereno: An extensible tool for generating realistic iec-61850 intrusion detection datasets. In: **Anais Estendidos do XXII Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais (SBSeg Estendido 2022)**. [S.l.]: Sociedade Brasileira de Computação, 2022. p. 1–8. Citado na página 16.

QUINCOZES, S. E. et al. On the performance of grasp-based feature selection for cps intrusion detection. **IEEE Transactions on Network and Service Management**, v. 19, n. 1, p. 614–626, 2022. Citado 5 vezes nas páginas 12, 15, 16, 23 e 28.

QUINCOZES, S. E. et al. An extended assessment of metaheuristics-based feature selection for intrusion detection in cps perception layer. **Annals of Telecommunications**, v. 77, n. 7–8, p. 457–471, 2022. Citado 5 vezes nas páginas 12, 15, 16, 23 e 28.

- QUINCOZES, S. E. et al. Grasp-based feature selection for intrusion detection in cps perception layer. In: **2020 4th Conference on Cloud and Internet of Things (CIoT)**. [S.l.]: IEEE, 2020. p. 41–48. Citado 5 vezes nas páginas 12, 15, 16, 23 e 28.
- QUINLAN, J. R. Induction of decision trees. **Machine Learning**, v. 1, n. 1, p. 81–106, 1986. Citado na página 14.
- QUINLAN, J. R. **C4.5: Programs for Machine Learning**. San Mateo, CA: Morgan Kaufmann, 1993. ISBN 978-1-55860-238-0. Disponível em: <<https://www.sciencedirect.com/book/9780080500584/c4-5>>. Citado na página 14.
- RESENDE, M. G. C.; RIBEIRO, C. C. **Optimization by GRASP: Greedy Randomized Adaptive Search Procedures**. New York, NY: Springer, 2016. Citado 4 vezes nas páginas 12, 15, 16 e 23.
- ROBNIK-ŠIKONJA, M.; KONONENKO, I. Theoretical and empirical analysis of relieff and rrelieff. **Machine Learning**, v. 53, n. 1, p. 23–69, 2003. Citado na página 14.
- SAEYS, Y.; INZA, I.; LARRAÑAGA, P. A review of feature selection techniques in bioinformatics. **Bioinformatics**, v. 23, n. 19, p. 2507–2517, 2007. Citado 3 vezes nas páginas 12, 14 e 16.
- SCHERER, F. H. et al. IWSHAP: Um método de seleção incremental de características para redes CAN baseado em inteligência artificial explicável (XAI). In: **Anais do XXIV Simpósio Brasileiro de Segurança da Informação e de Sistemas Computacionais**. Porto Alegre, RS, Brasil: SBC, 2024. p. 351–366. Disponível em: <<https://sol.sbc.org.br/index.php/sbseg/article/view/30036>>. Citado 2 vezes nas páginas 16 e 64.
- SILVA, E. F. C. et al. Gdls-fs: Scaling feature selection for intrusion detection with grasp-fs and distributed local search. In: **Advanced Information Networking and Applications: Proceedings of the 37th International Conference on Advanced Information Networking and Applications (AINA-2023), Volume 2**. [S.l.]: Springer, Cham, 2023. (Lecture Notes in Networks and Systems, v. 654), p. 199–210. Citado 2 vezes nas páginas 17 e 28.
- SUBRAMANIAN, A. et al. A parallel heuristic for the vehicle routing problem with simultaneous pickup and delivery. **Computers & Operations Research**, Elsevier, v. 37, n. 11, p. 1899–1911, 2010. Citado 2 vezes nas páginas 17 e 28.
- TANENBAUM, A. S.; STEEN, M. van. **Distributed Systems: Principles and Paradigms**. 2. ed. Upper Saddle River, NJ, USA: Prentice Hall, 2007. Citado 3 vezes nas páginas 15, 17 e 22.
- TERMANINI, A. et al. Using machine learning to detect network intrusions in industrial control systems: A survey. **International Journal of Information Security**, v. 24, n. 1, 2025. Citado na página 16.
- USMAN, M. et al. A survey on observability of distributed edge & container-based microservices. **IEEE Access**, v. 10, p. 86904–86919, 2022. Citado 2 vezes nas páginas 15 e 17.

WU, X. et al. Online feature selection with streaming features. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, IEEE, v. 35, n. 5, p. 1178–1192, 2013. Citado na página 17.

YUSTA, S. C. Different metaheuristic strategies to solve the feature selection problem. **Pattern Recognition Letters**, v. 30, n. 5, p. 525–534, 2009. Citado na página 16.

Apêndices

APÊNDICE A – GALERIA DO FRONT-END

Este apêndice reúne capturas em alta resolução da interface operacional do *G-FShield*. A intenção é mostrar a navegação, os painéis analíticos e as telas administrativas sem poluir o fluxo principal da dissertação. As imagens foram organizadas por função de uso para deixar clara a progressão da interface, desde a autenticação até as telas de comparação, monitoramento e administração.

A.1 Acesso e Visão Geral

As primeiras telas apresentam o ponto de entrada da plataforma e o painel inicial, que resumem o estado geral do sistema logo após a autenticação. As Figuras 20 e 21 mostram essa passagem entre acesso e visão consolidada.

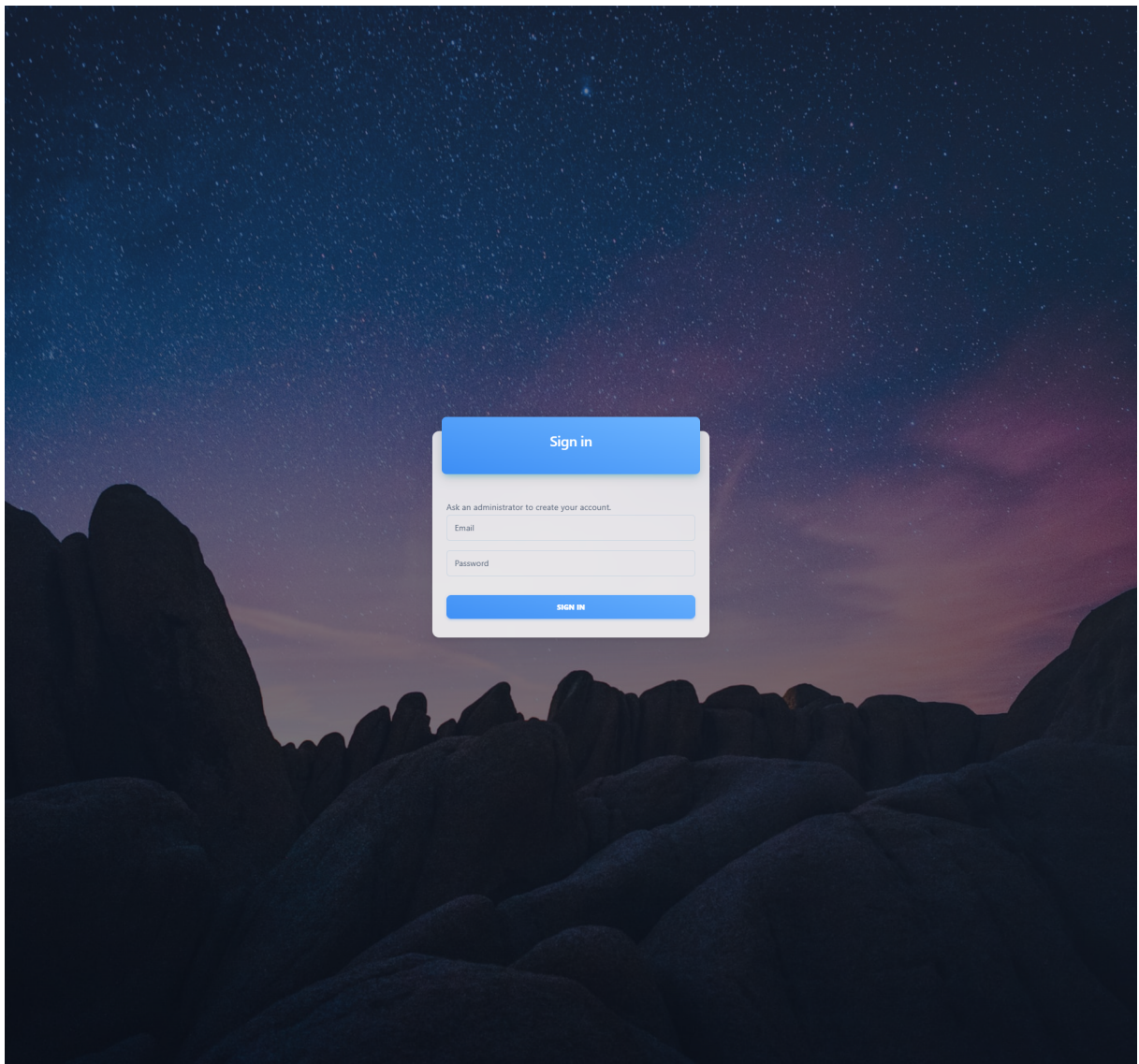


Figura 20 – Tela de autenticação.

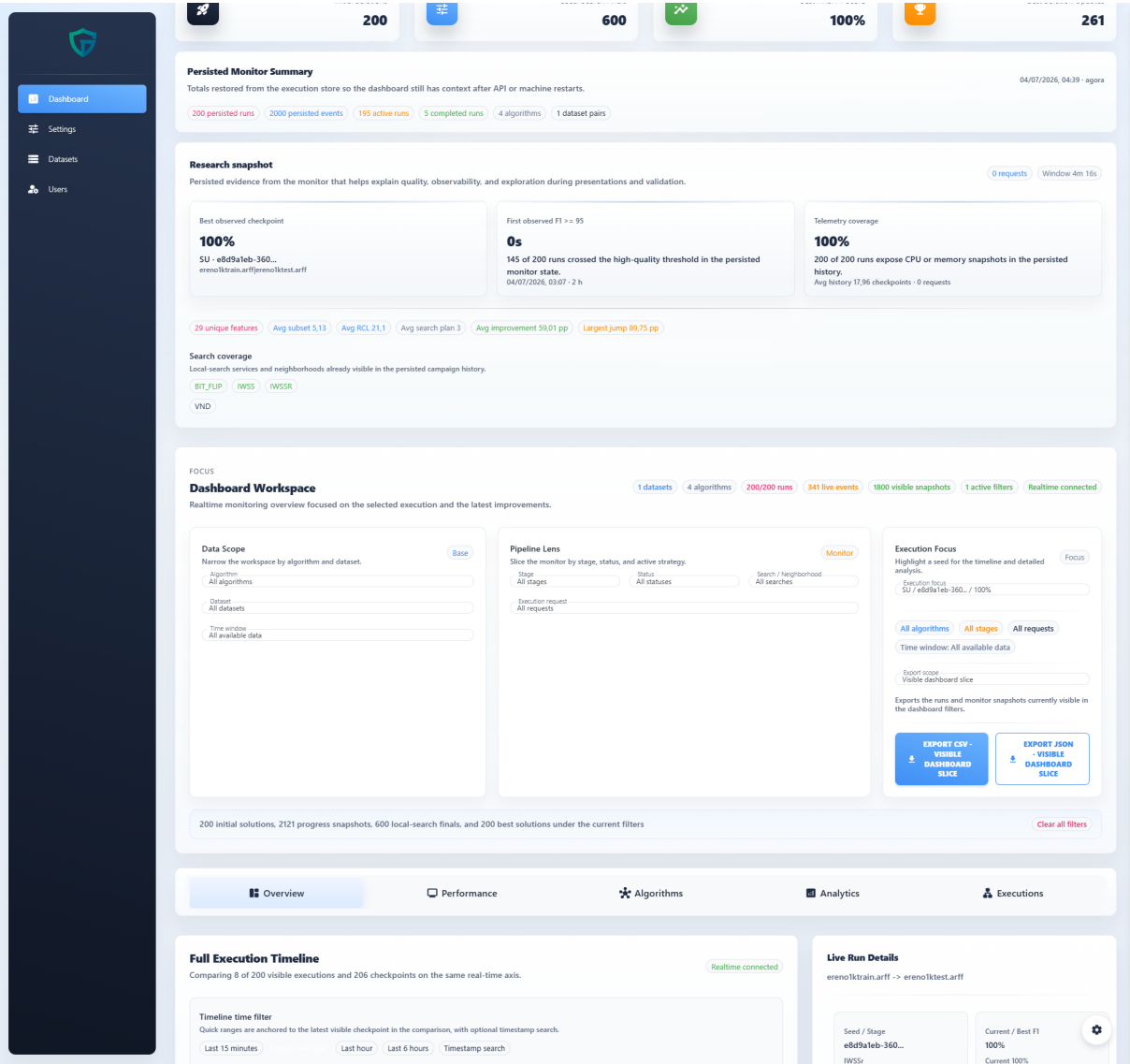


Figura 21 – Painel de visão geral.

A.2 Performance e Algoritmos

Esta seção mostra como o usuário navega entre os painéis de desempenho e de comparação algorítmica, o que é importante para demonstrar a leitura operacional do sistema. As Figuras 22 e 23 detalham essa leitura.

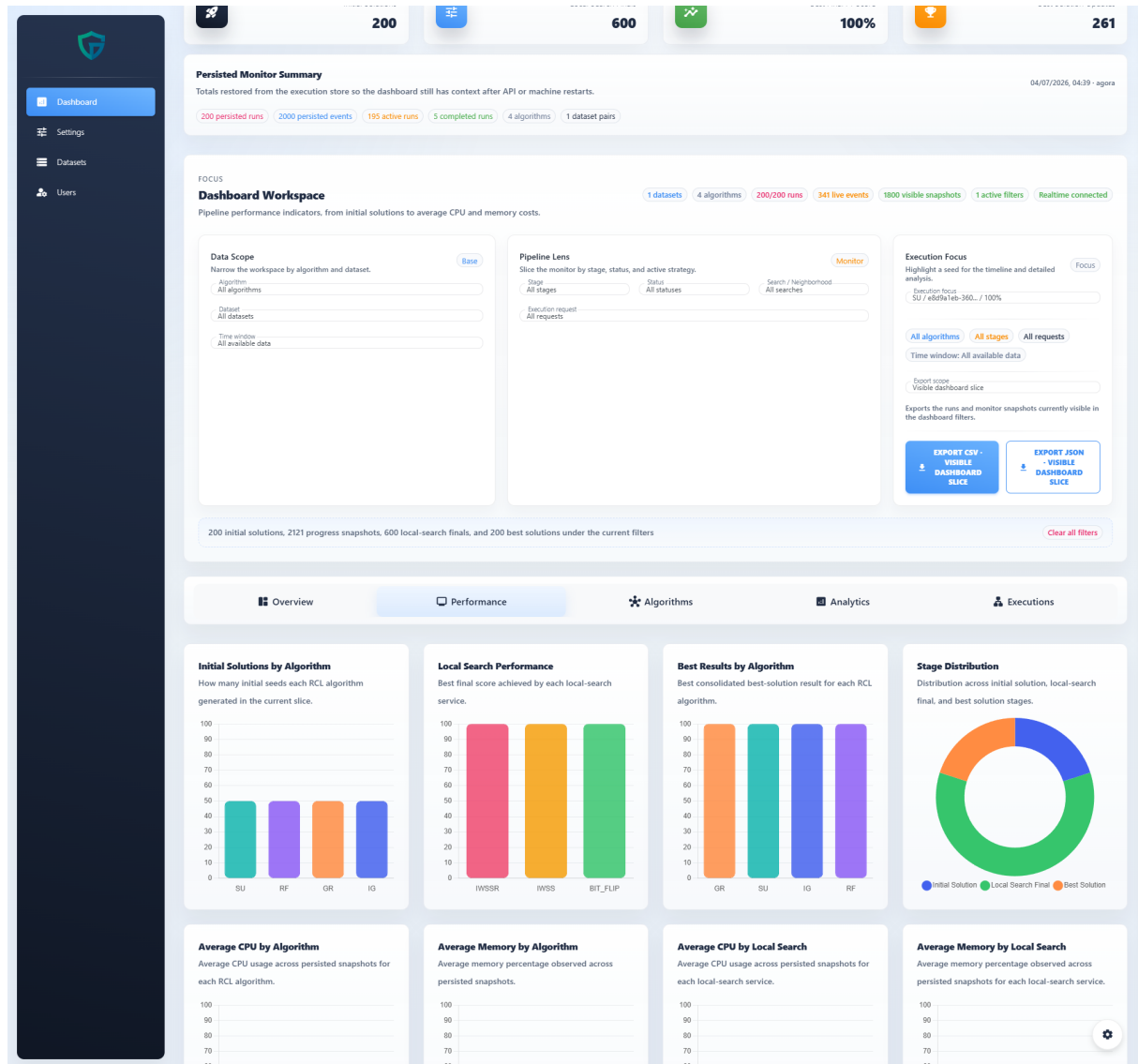


Figura 22 – Tela de performance.

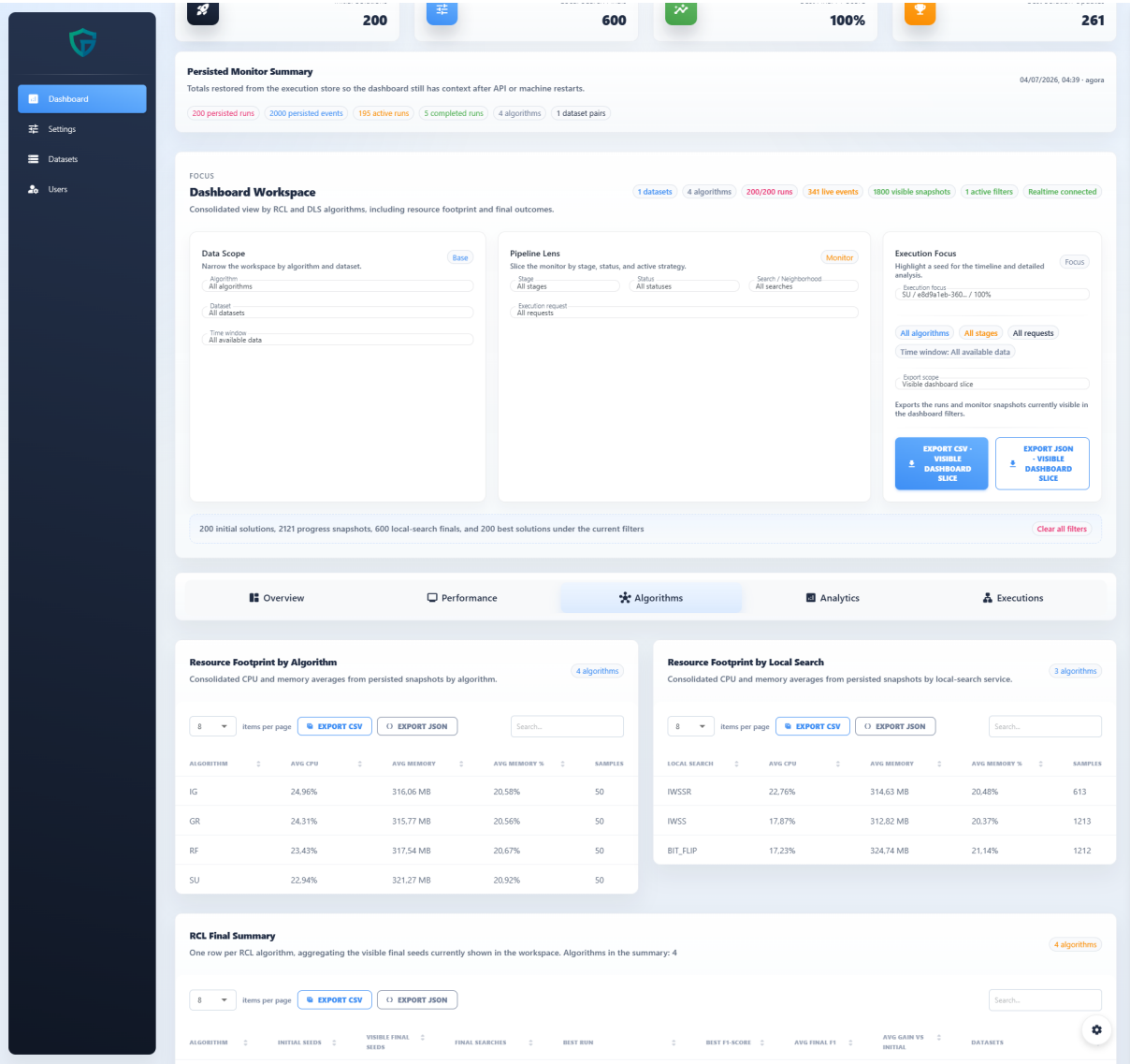


Figura 23 – Tela de comparação de algoritmos.

A.3 Analytics

Os painéis analíticos resumem a atividade de execução e ajudam a observar tendências agregadas, detalhes por evento e tópicos persistidos. As Figuras 24, 25, 26 e 27 organizam essa visão em camadas.

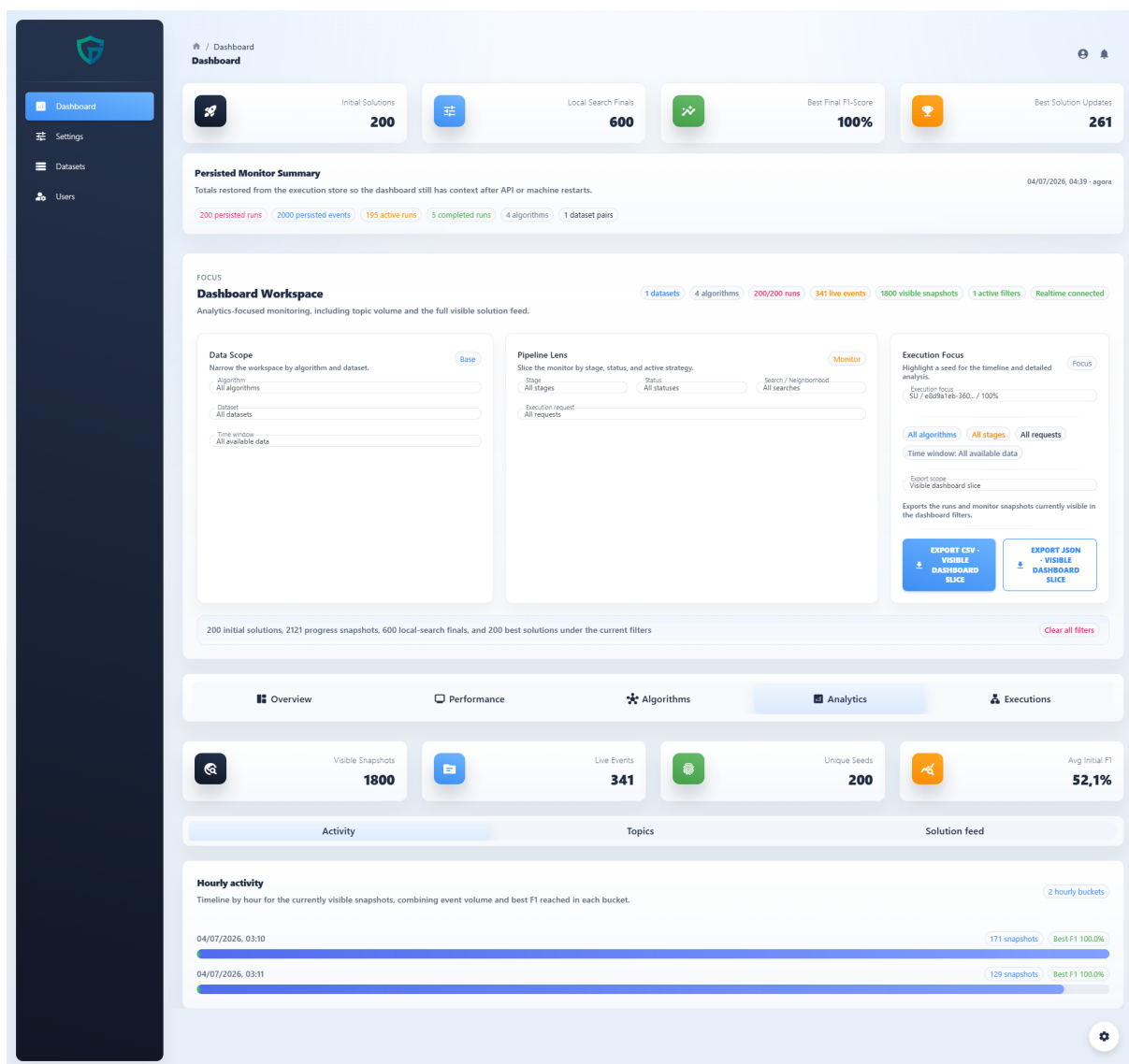


Figura 24 – Atividade analítica agregada.

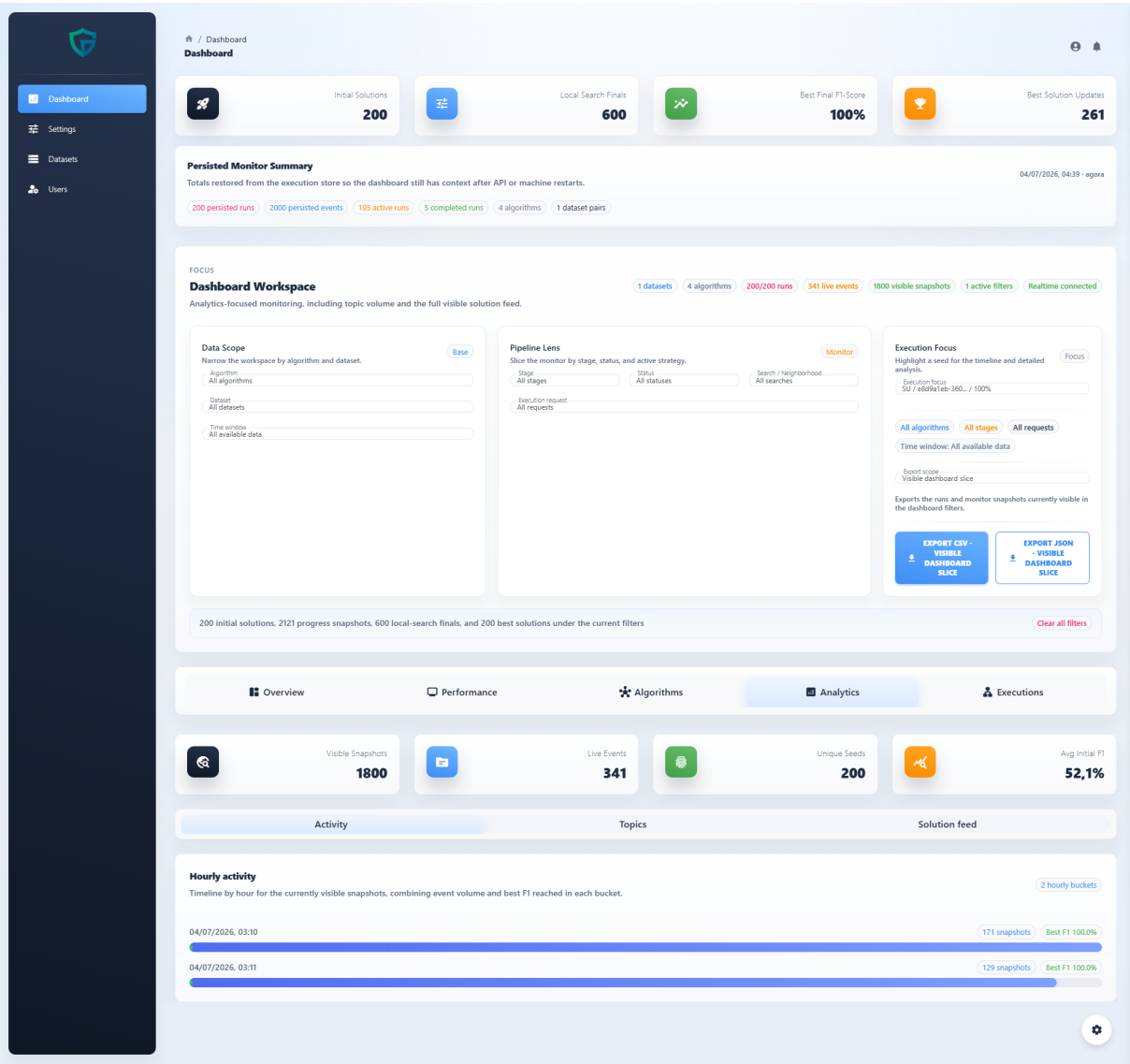


Figura 25 – Detalhe da atividade analítica.

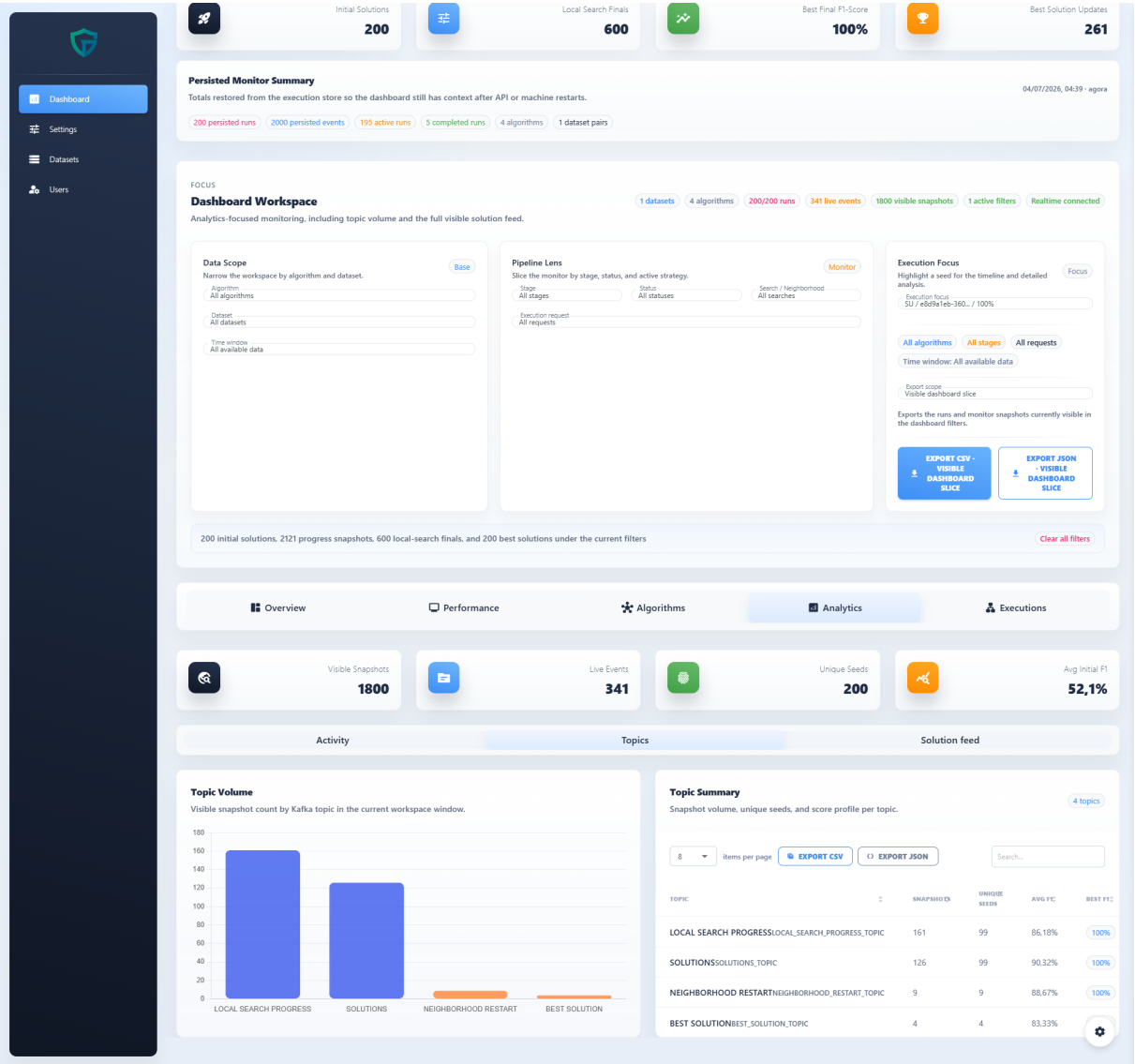


Figura 26 – Tópicos analíticos.

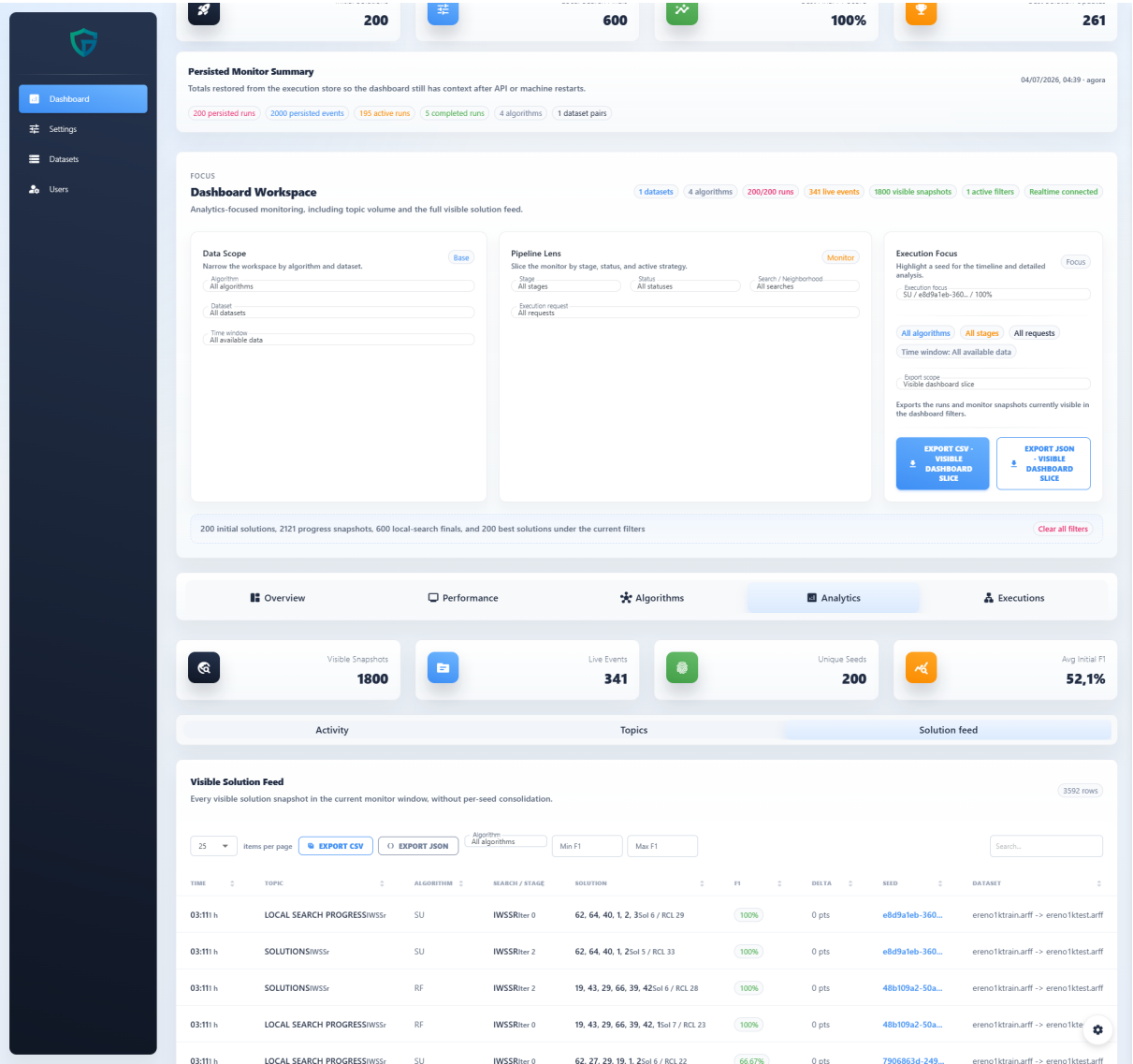


Figura 27 – Feed analítico persistido.

A.4 Comparação de Execuções

Esta parte evidencia a comparação entre *runs* e a inspeção de sementes selecionadas, reforçando a natureza explorável e auditável da plataforma. As Figuras 28, 29 e 30 mostram como essa comparação é feita na prática.

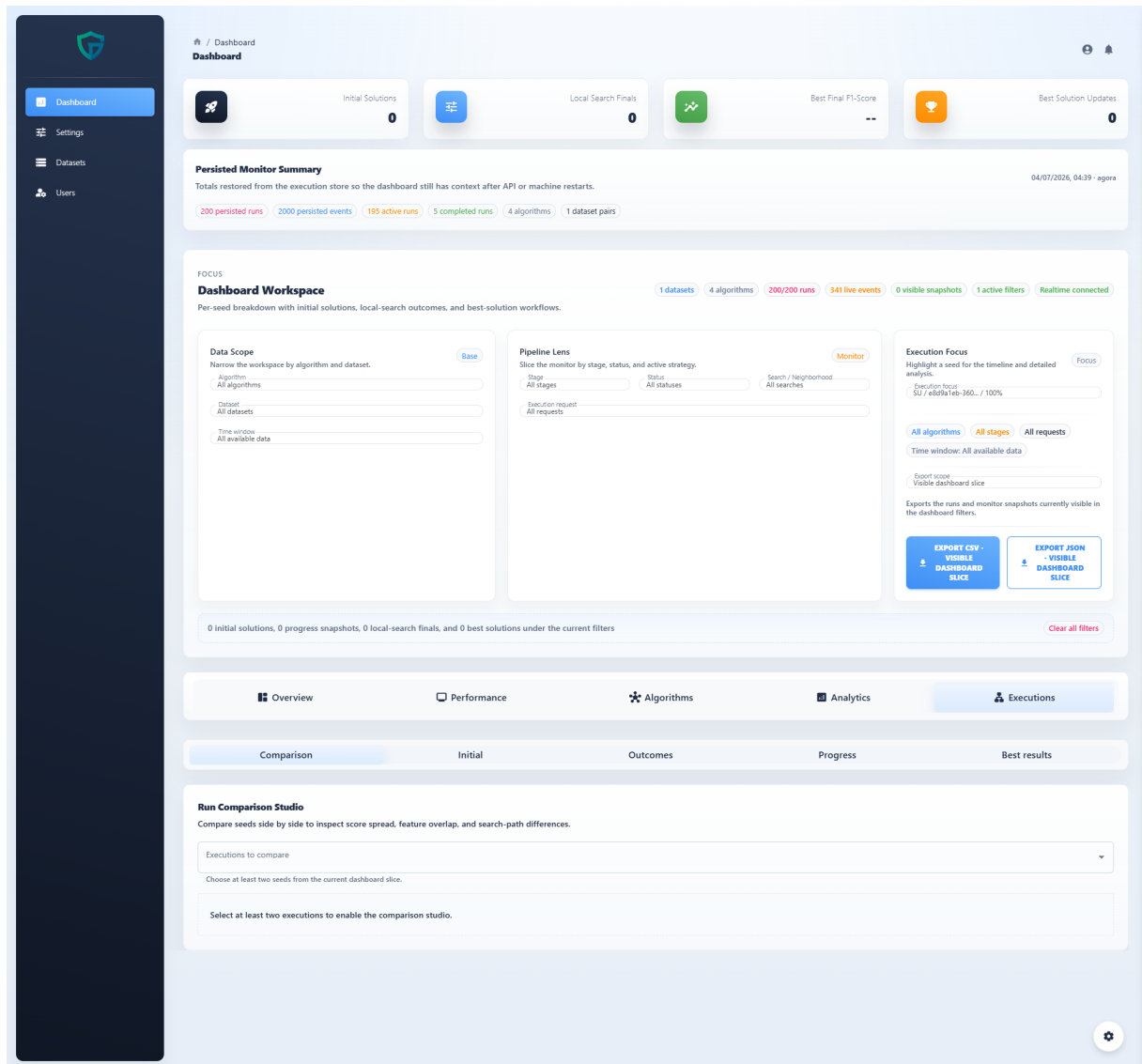


Figura 28 – Tela de comparação de execuções.

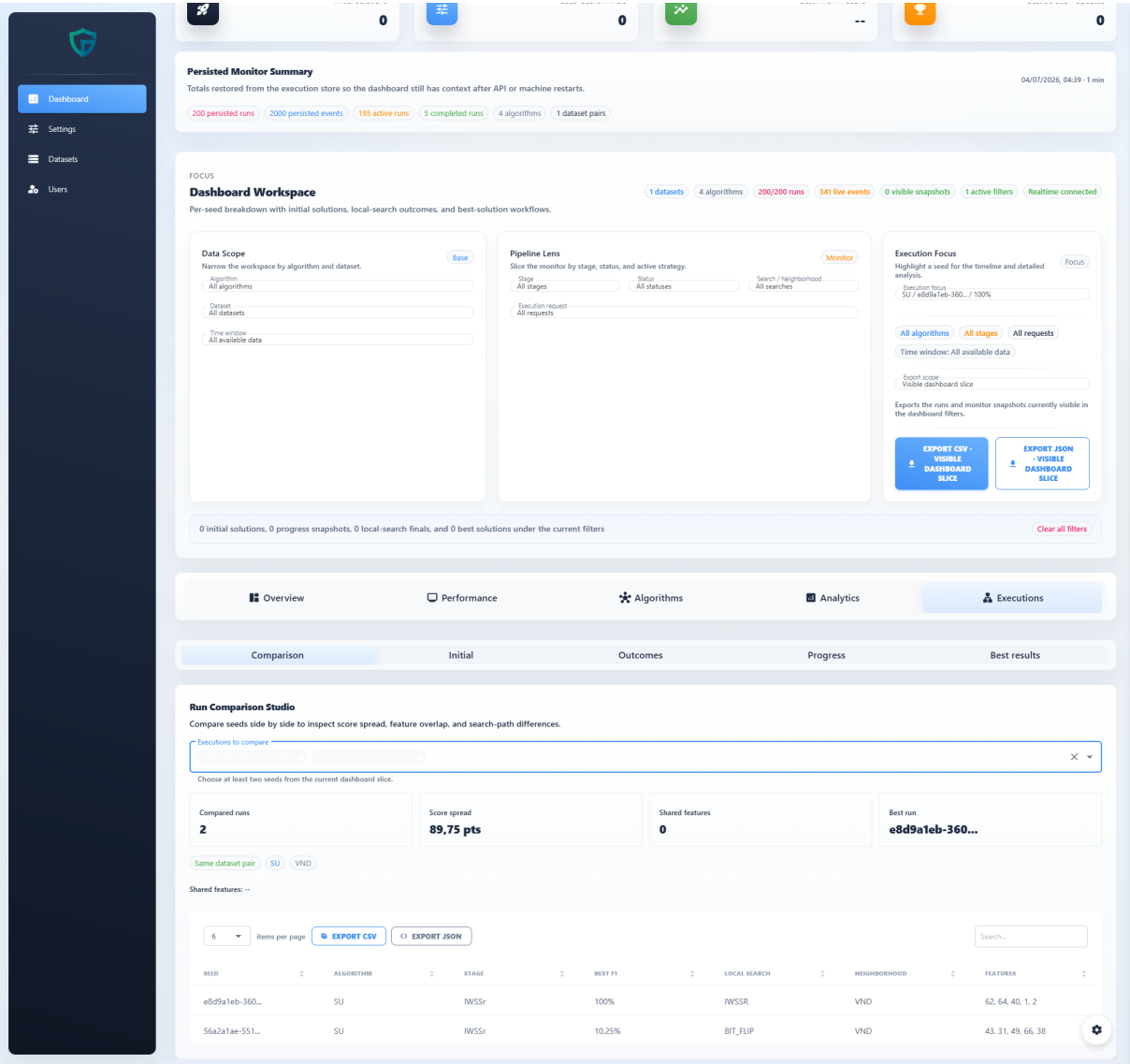


Figura 29 – Comparação com sementes selecionadas.

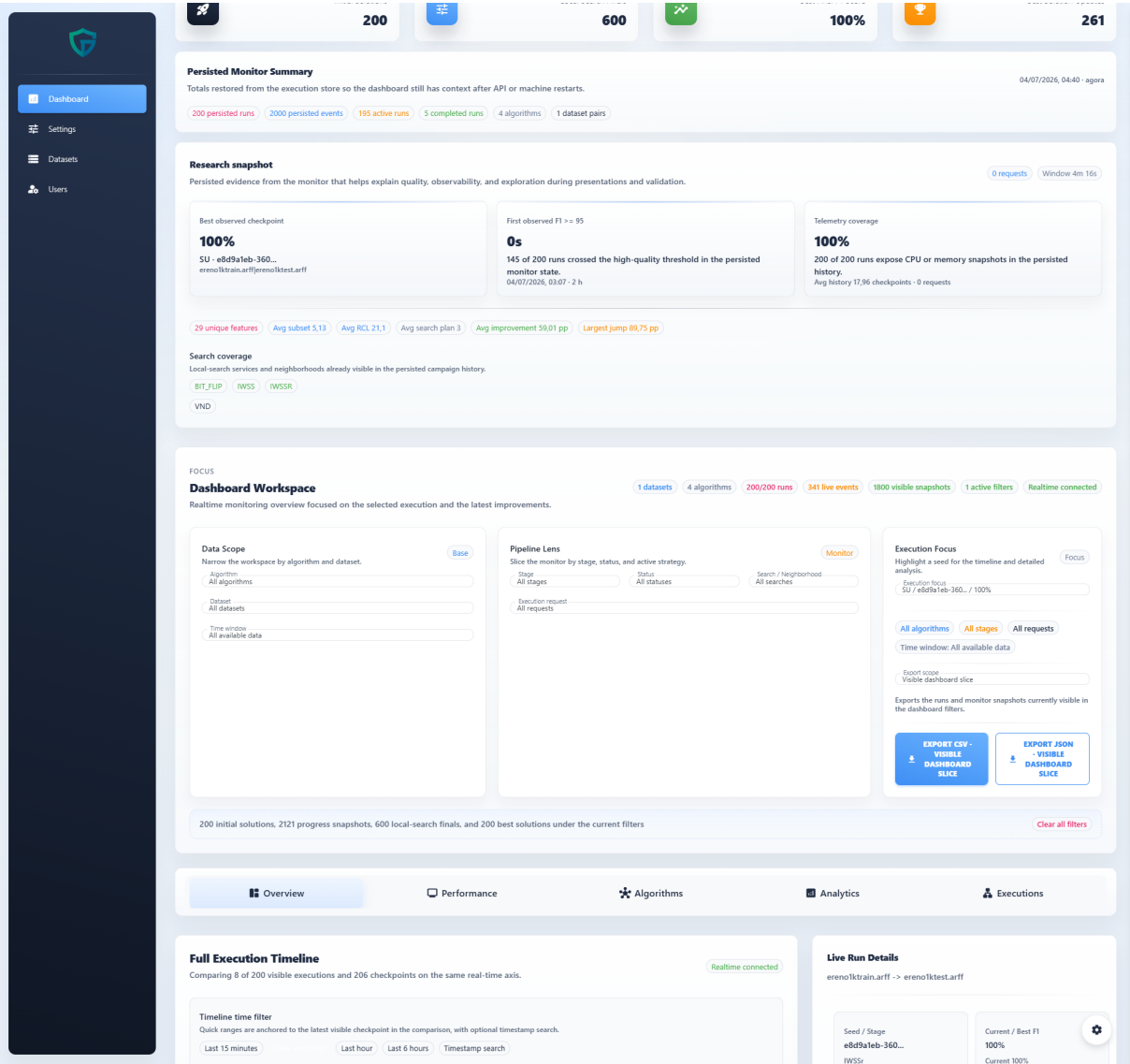


Figura 30 – Detalhe de uma execução específica.

A.5 Linha do Tempo e Resultados

As capturas desta seção mostram como o front exibe a evolução das soluções ao longo do tempo, apoiando a leitura do progresso e da qualidade das execuções. As Figuras 31, 32, 33 e 34 completam esse ciclo visual.

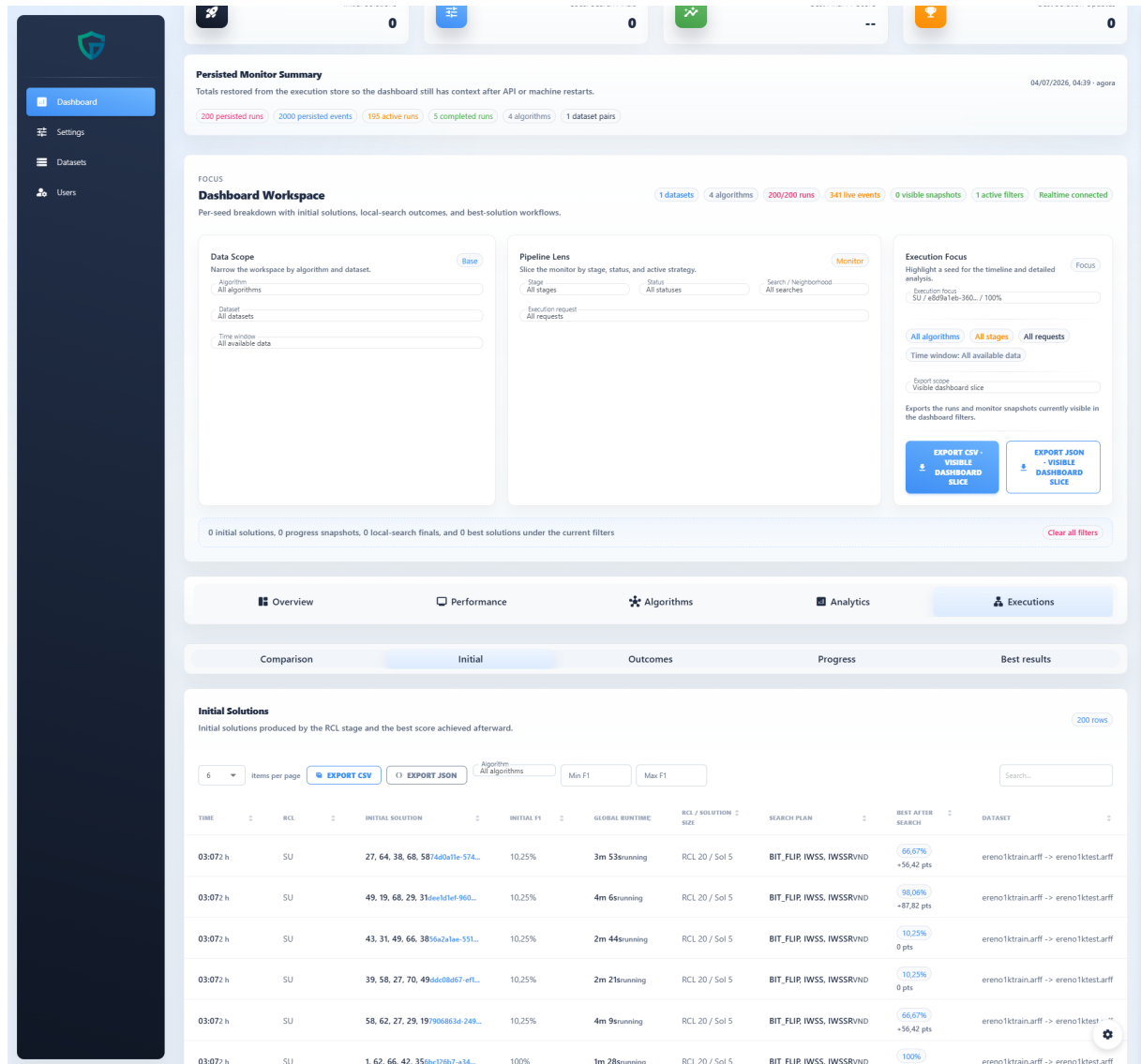


Figura 31 – Soluções iniciais.

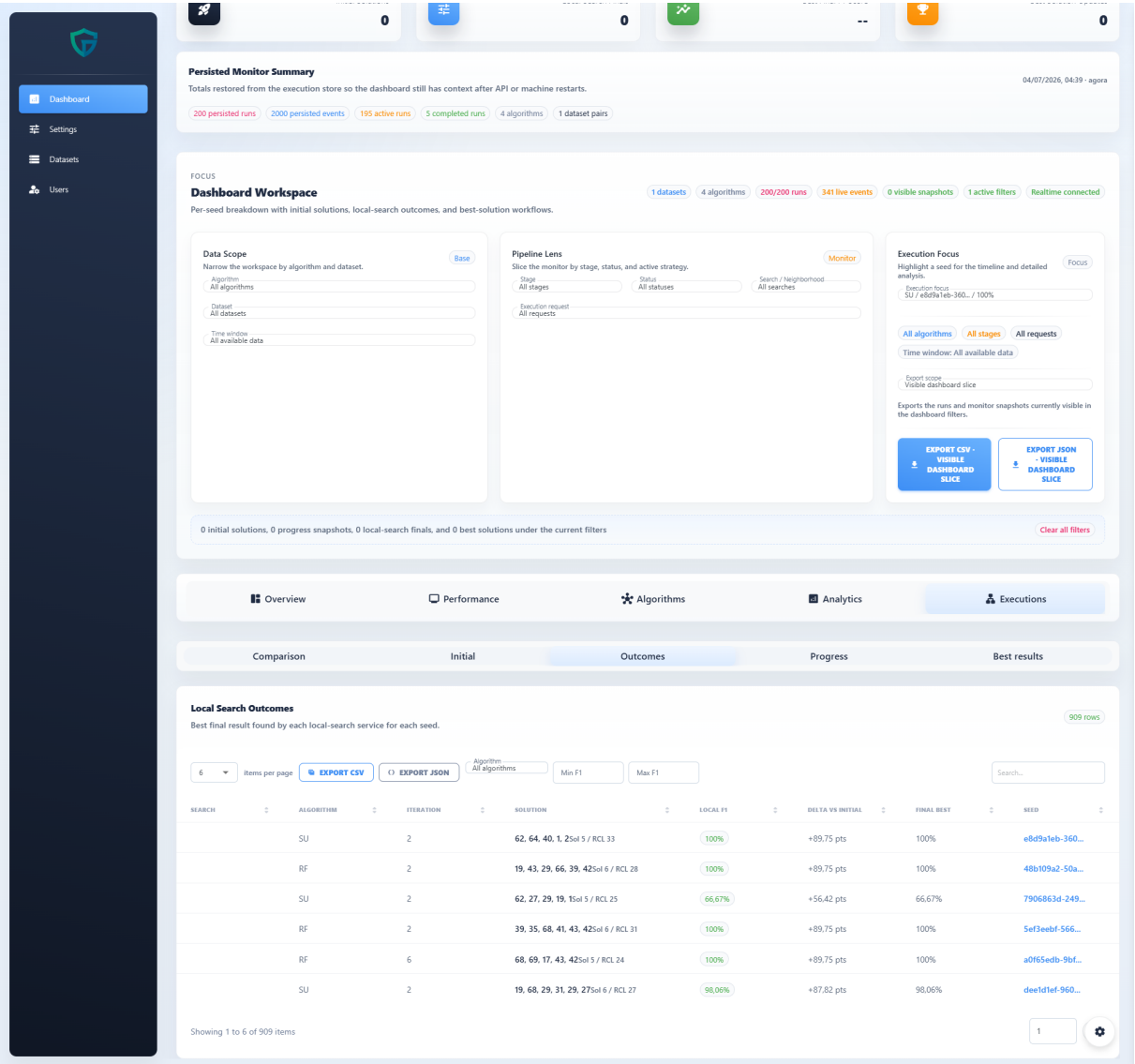


Figura 32 – Resultados consolidados.

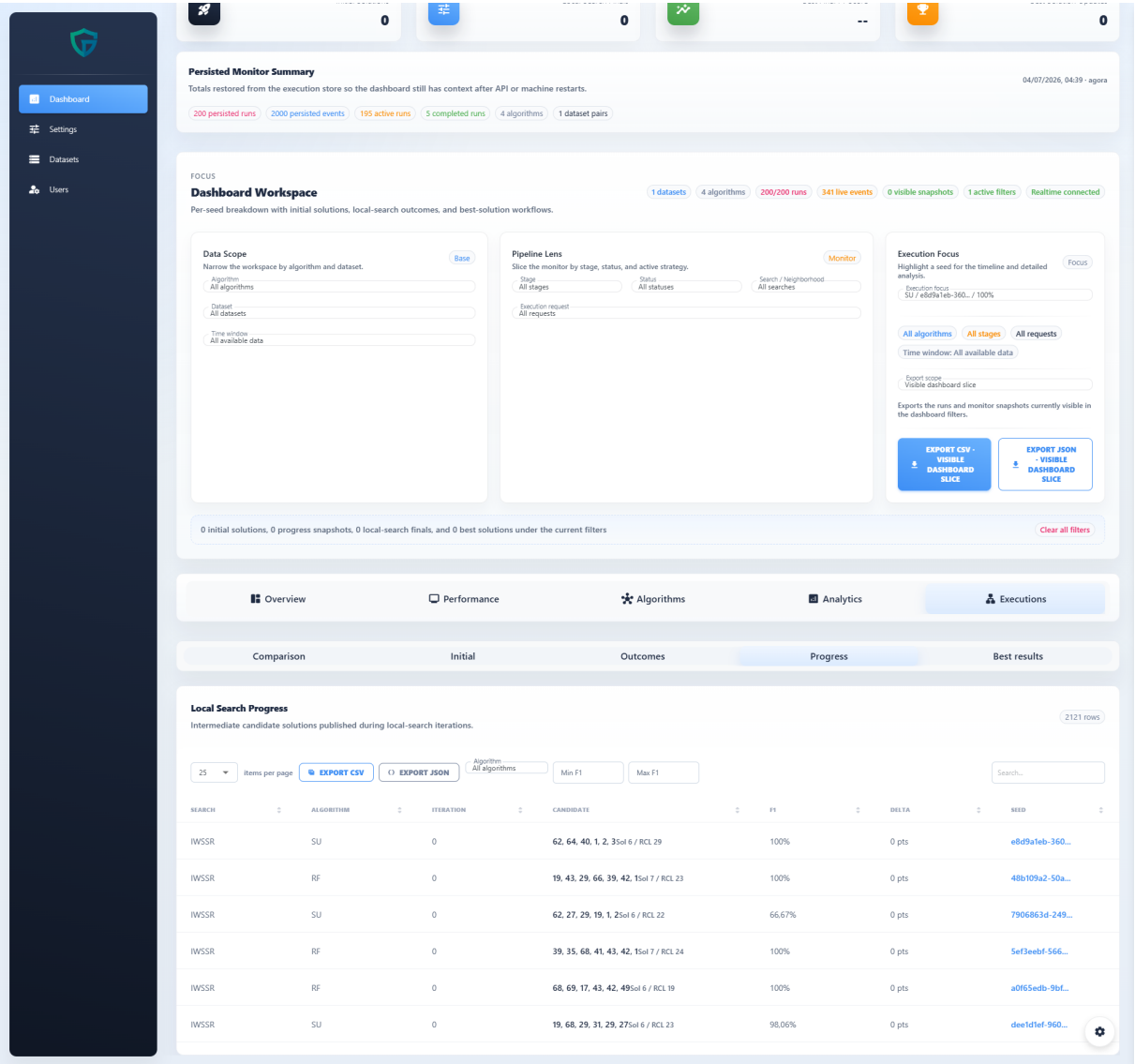


Figura 33 – Progresso da execução.

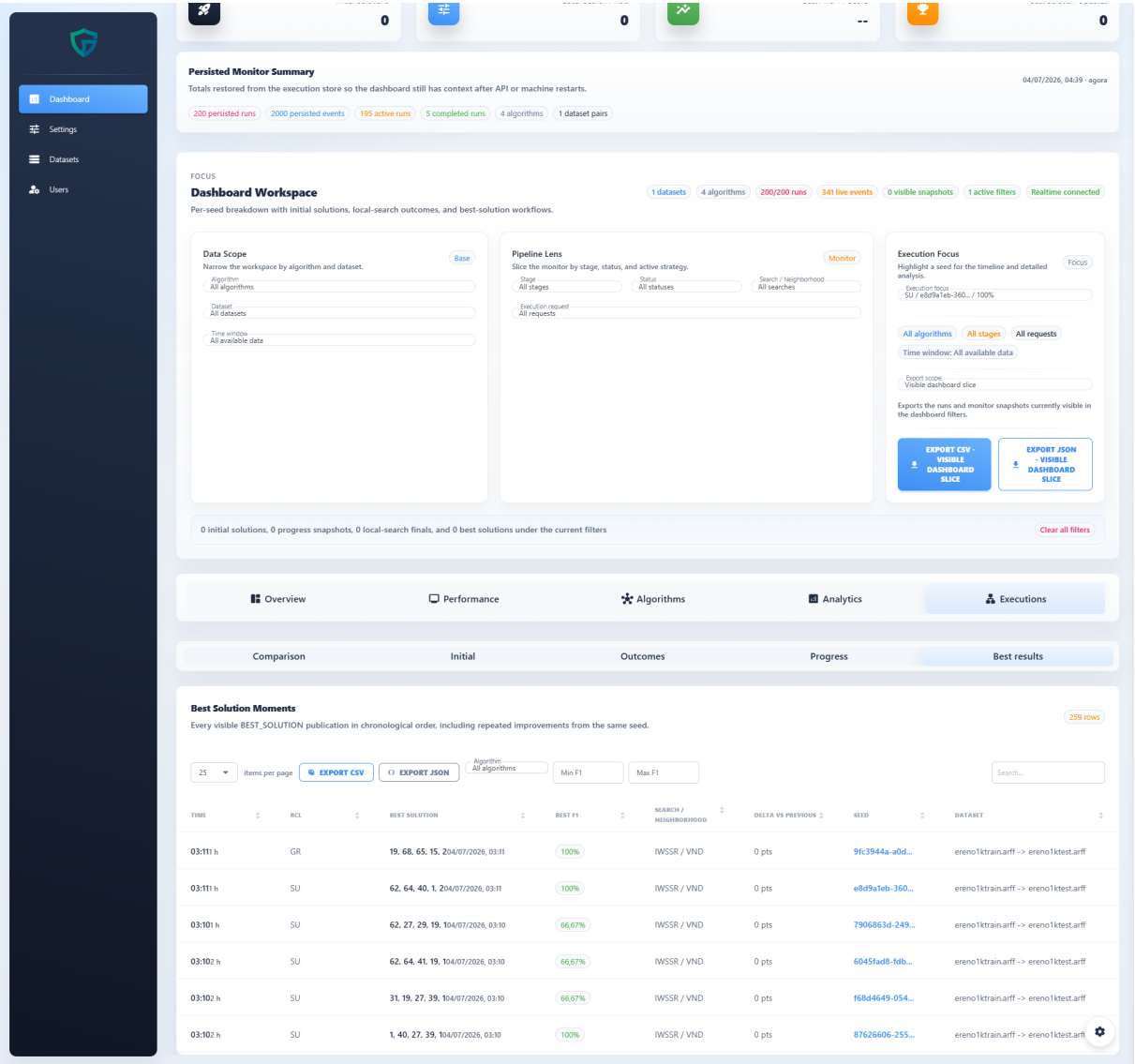


Figura 34 – Melhores soluções.

A.6 Configurações e Administração

Por fim, as telas administrativas demonstram que a plataforma também cobre os aspectos de operação e manutenção, não apenas a visualização de resultados. As Figuras 35, 36, 37, 38, 39, 40 e 41 mostram o conjunto administrativo.

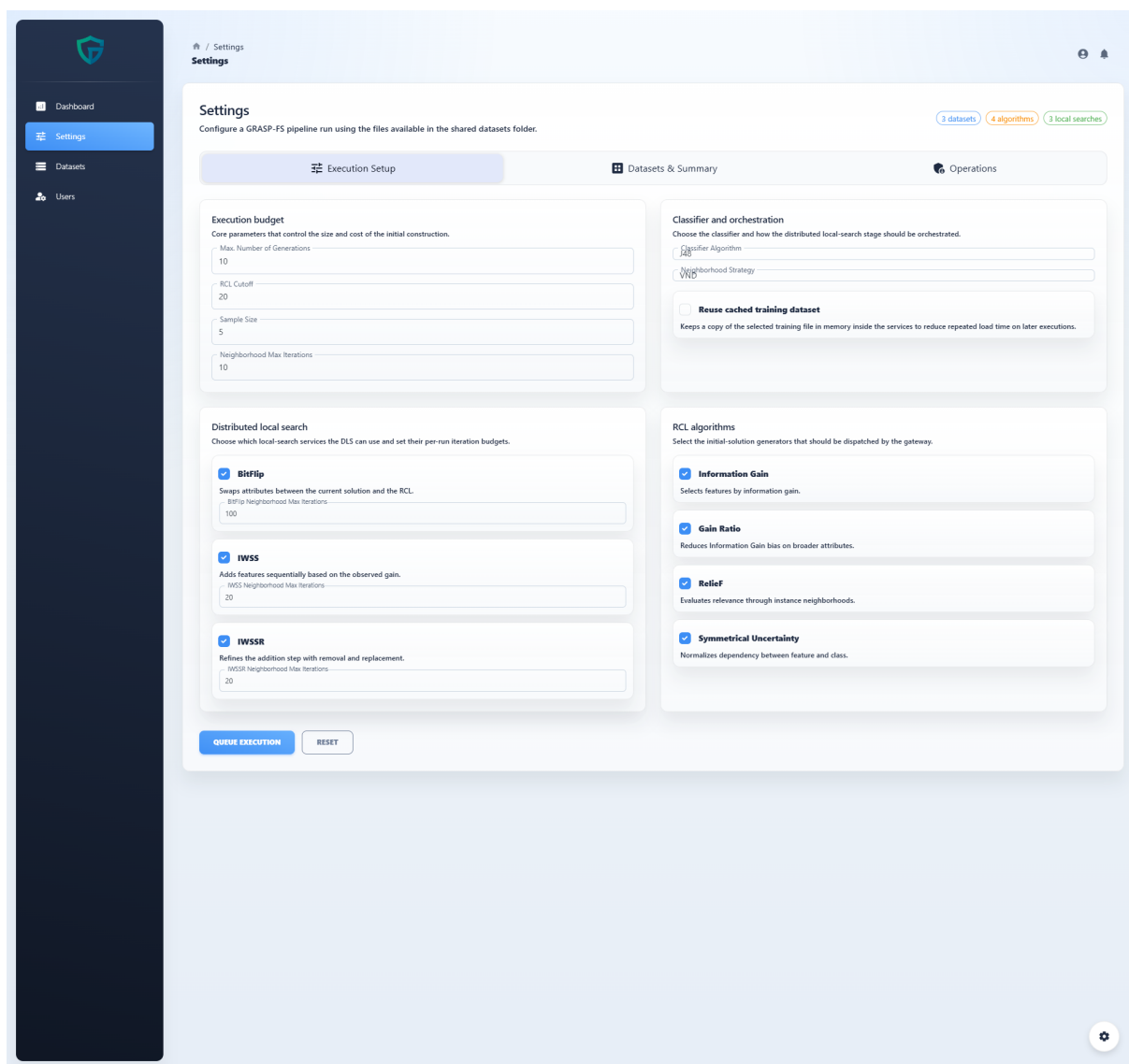


Figura 35 – Configuração de execução.

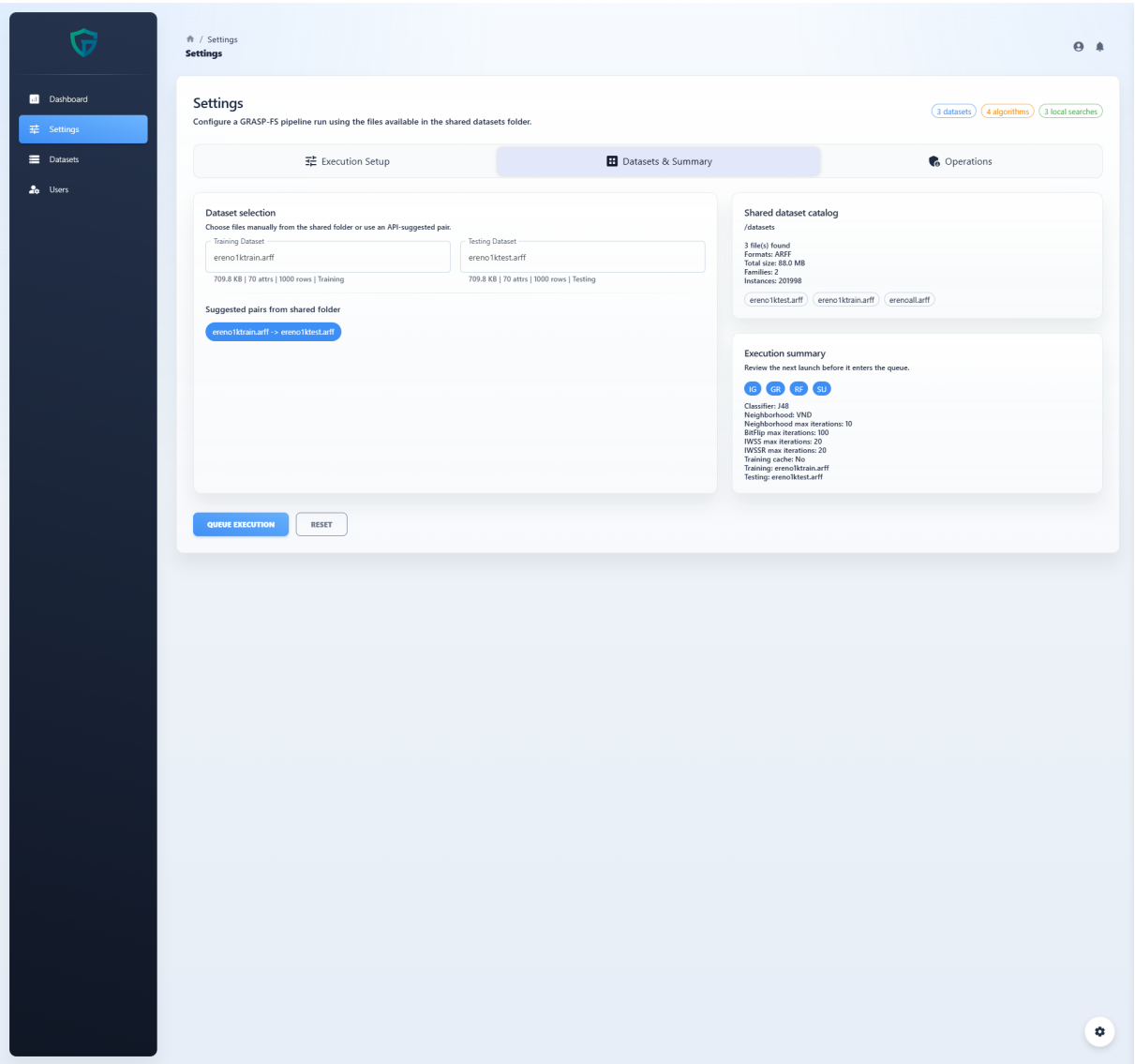


Figura 36 – Configuração de conjuntos de dados.

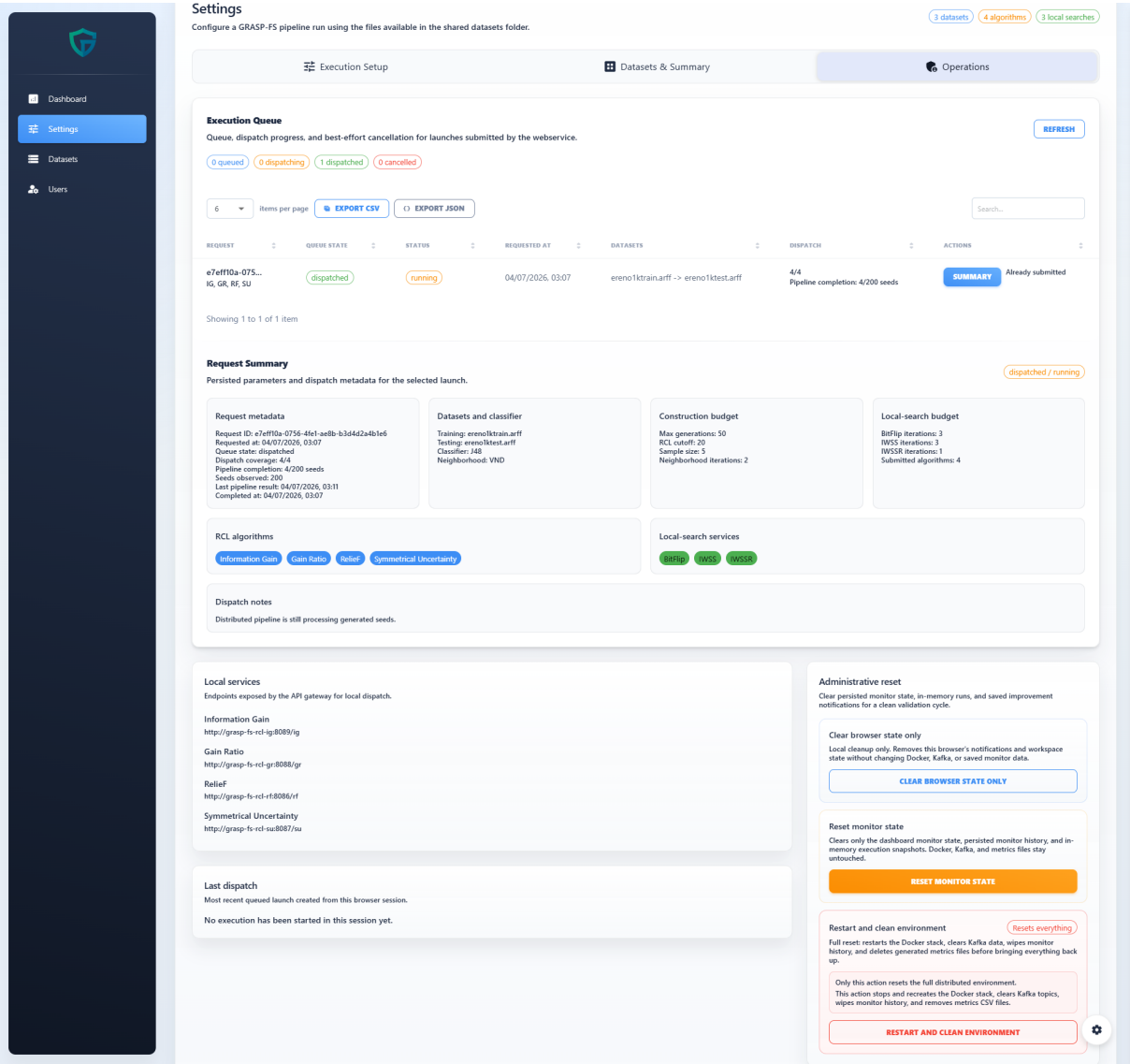


Figura 37 – Configuração de operações.

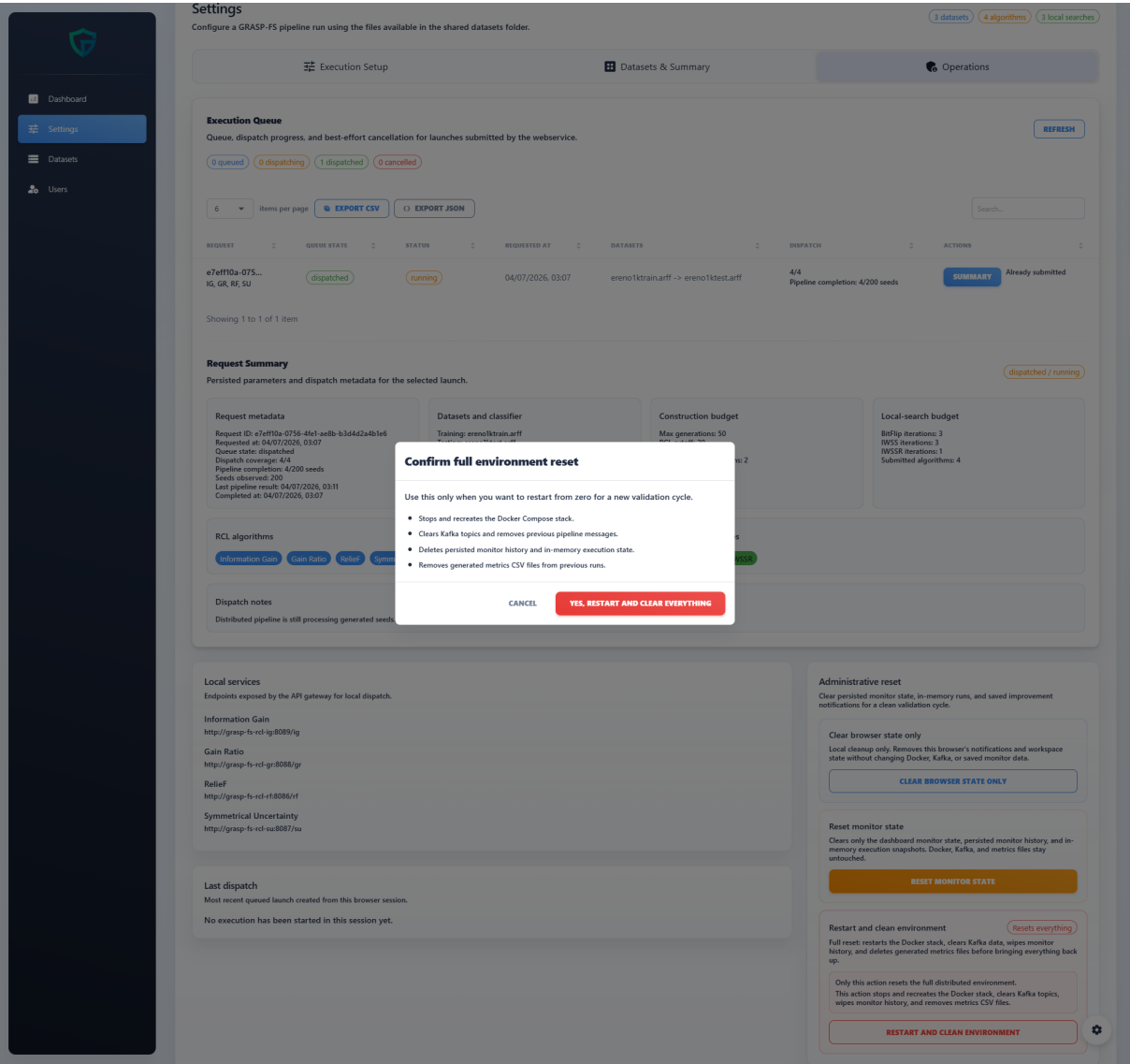


Figura 38 – Diálogo de reset completo.

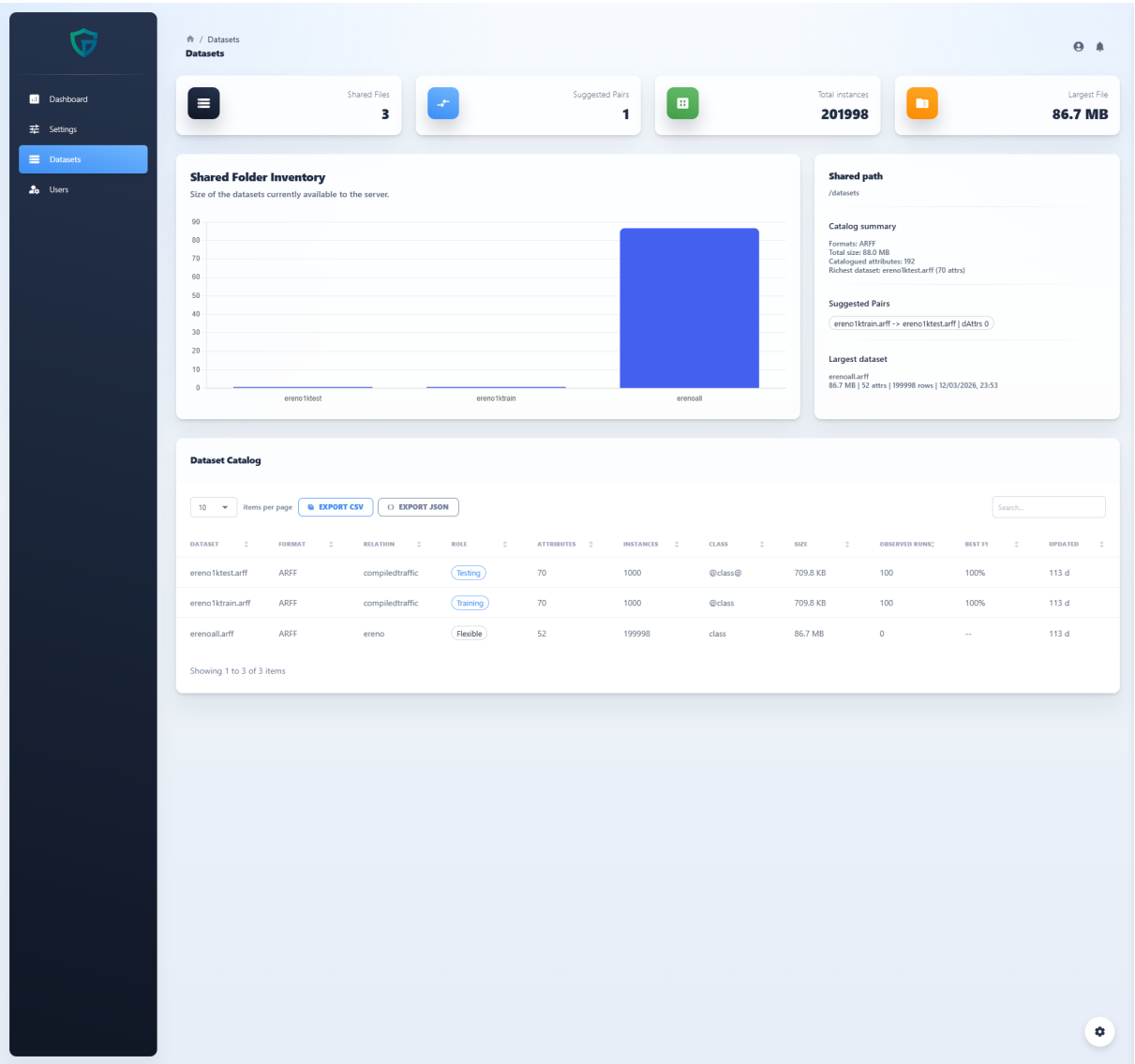


Figura 39 – Listagem de datasets.

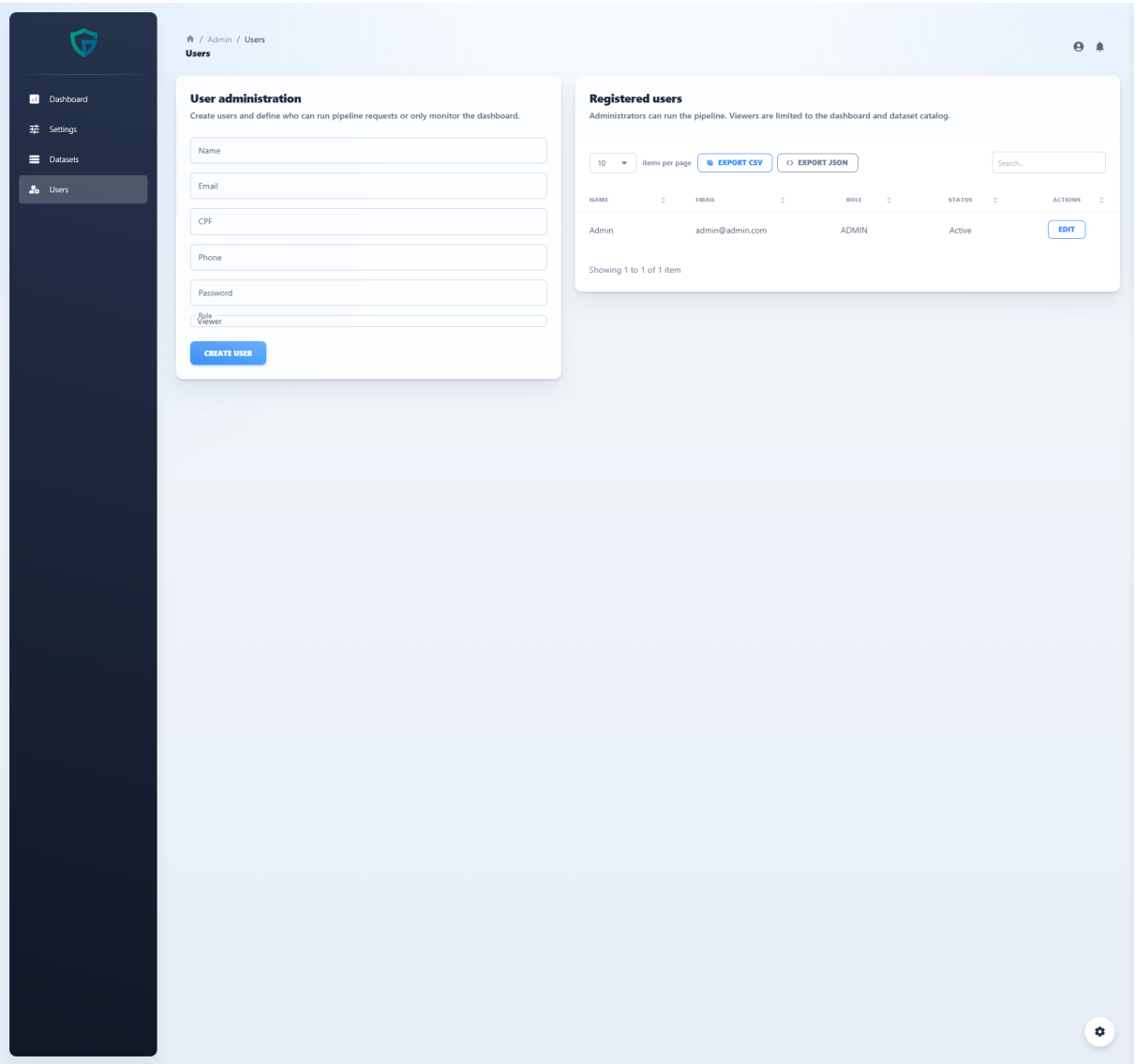


Figura 40 – Listagem de usuários.

Edit user

Update access role, status, and profile details for the selected user.

Name	Admin	Email	admin@admin.com
CPF		Phone	
Role	Administrator	Status	Active
New password			

Fill this only if you want to reset the password.

[SAVE CHANGES](#) [BACK](#)

User created on 22/03/2026, 22:30:13.

Figura 41 – Edição de usuário.

APÊNDICE B – VÍDEO E DOCUMENTAÇÃO

O vídeo demonstrativo da plataforma foi incluído como apoio à defesa. Como arquivos MP4 nem sempre são renderizados diretamente pelo leitor de PDF, o material é disponibilizado por link e também mantido na pasta de figuras do projeto. O poster ajuda a identificar rapidamente o conteúdo do vídeo mesmo quando o PDF é acessado em um visualizador que não embute mídia. As Figuras 42 e 43 completam o material de apoio com a demonstração e a documentação da API.

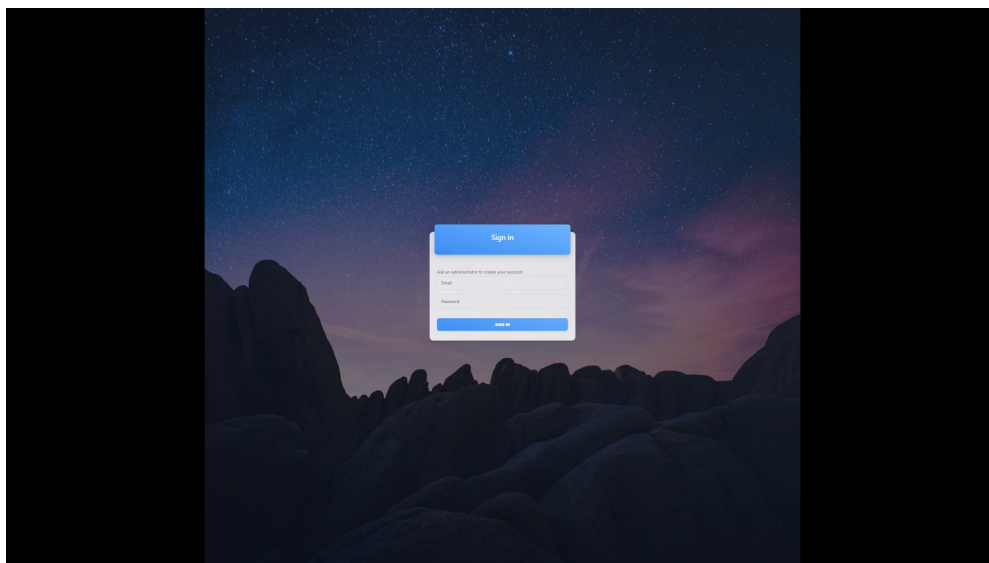


Figura 42 – Poster do vídeo demonstrativo da plataforma.

Vídeo demonstrativo: <<https://youtu.be/-uqZPaTOjsU>>

O vídeo de demonstração deve ser disponibilizado em um repositório ou artefato persistente, identificado pela versão correspondente à dissertação.

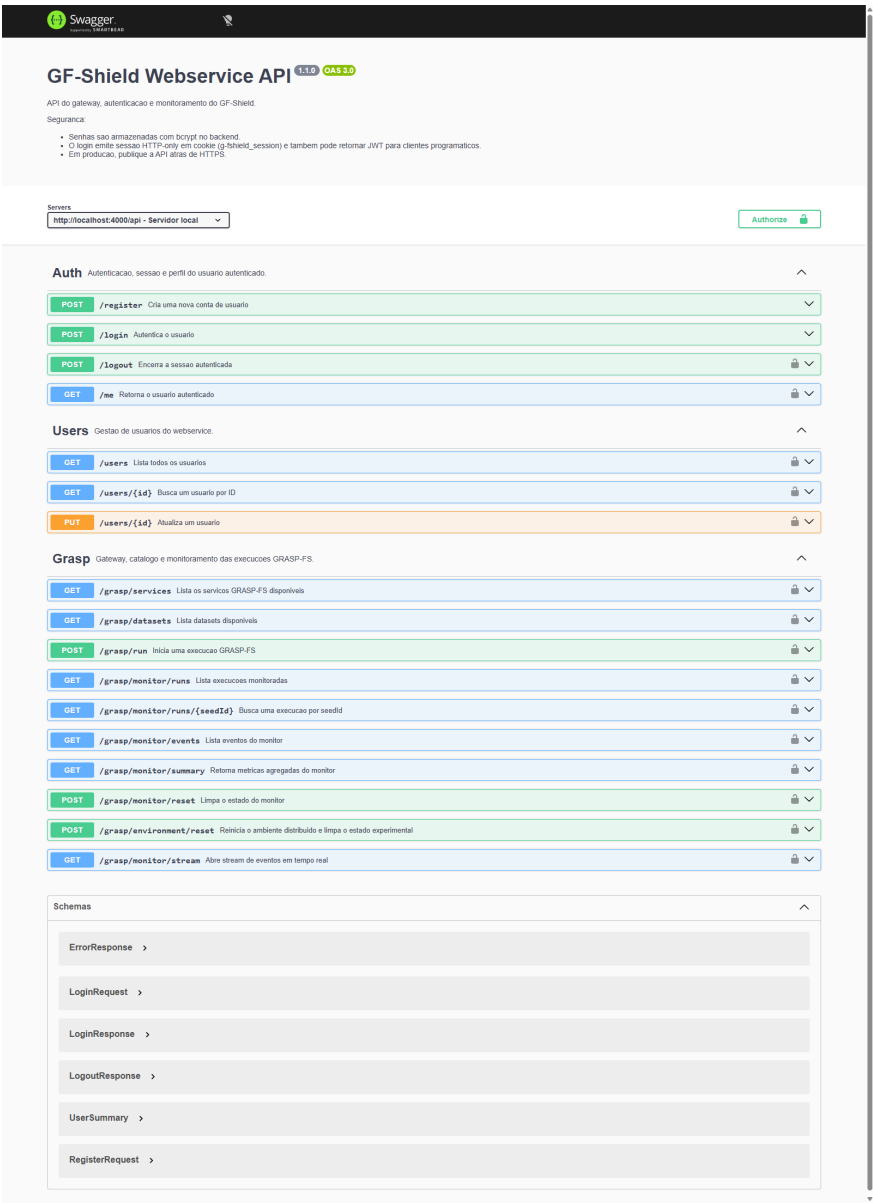


Figura 43 – Documentação Swagger da API do backend.

APÊNDICE C – DIAGRAMAS DE ARQUITETURA

Este apêndice reúne os diagramas específicos que complementam a leitura da arquitetura principal. Eles foram incluídos para conectar o material visual do desenvolvimento às telas e aos fluxos operacionais mostrados nas demais seções. As Figuras 44, 45, 46 e 47 mostram a cadeia arquitetural completa.

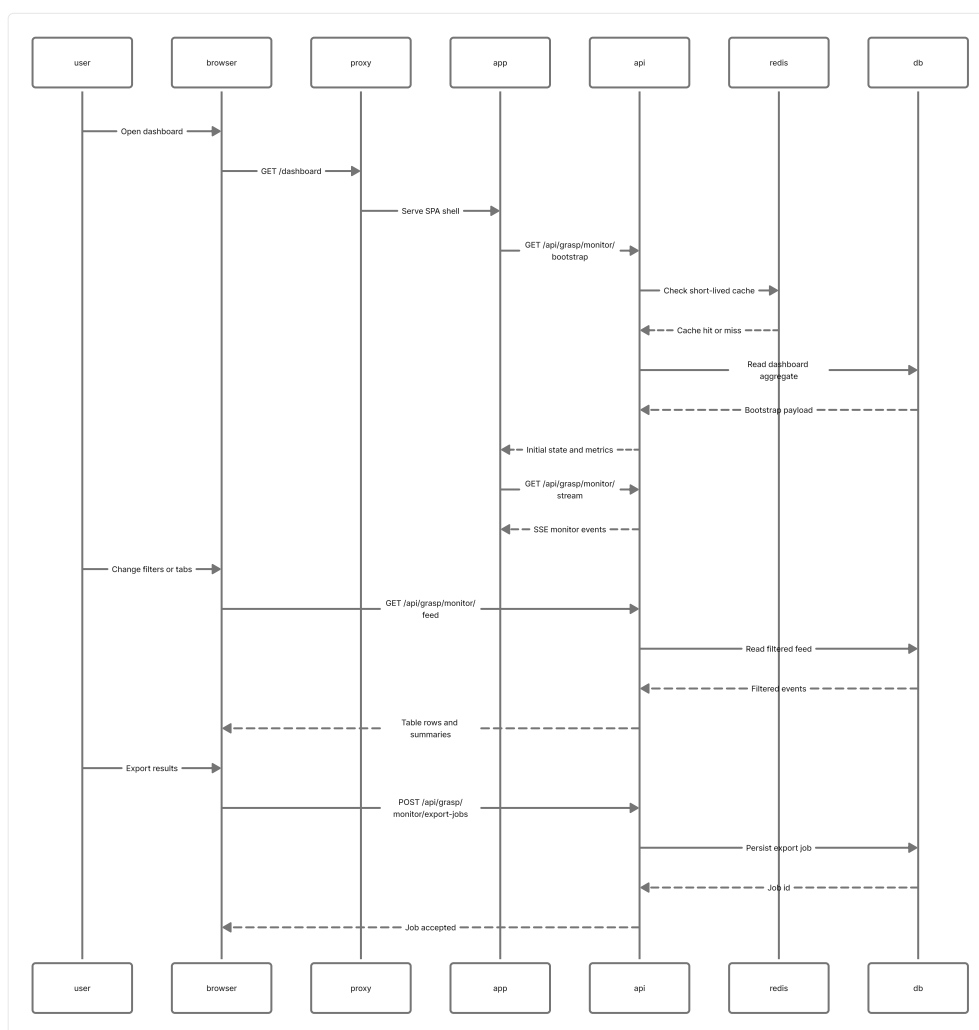


Figura 44 – Diagrama do funcionamento do front-end.

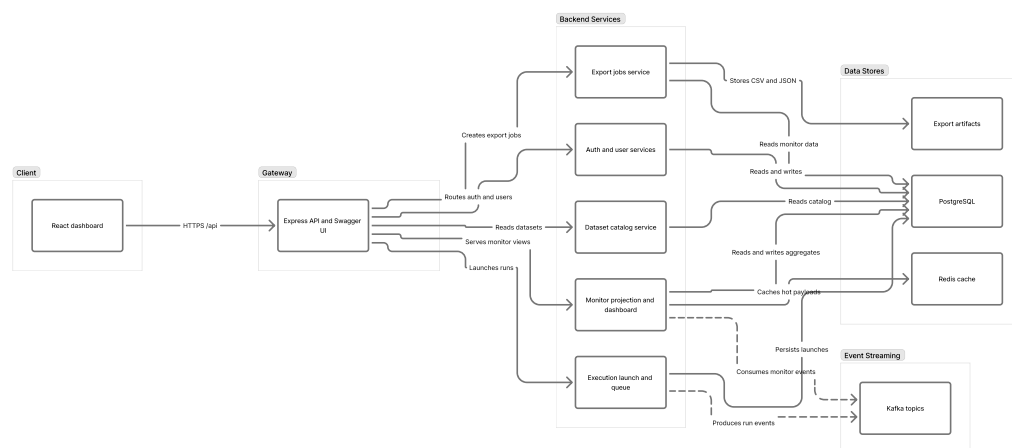


Figura 45 – Diagrama da arquitetura do backend.

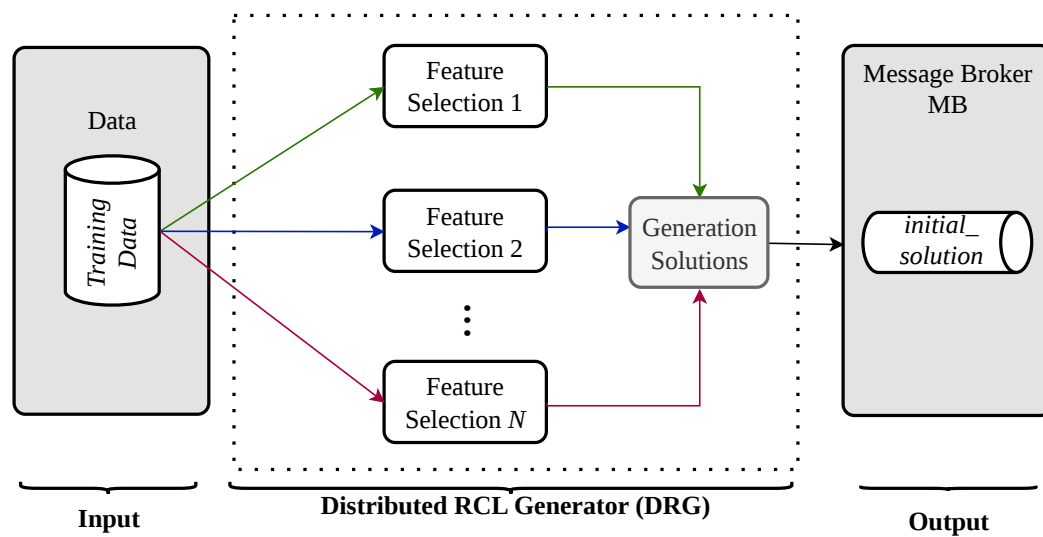


Figura 46 – Versão genérica do DRG com entrada de dados abstrata.

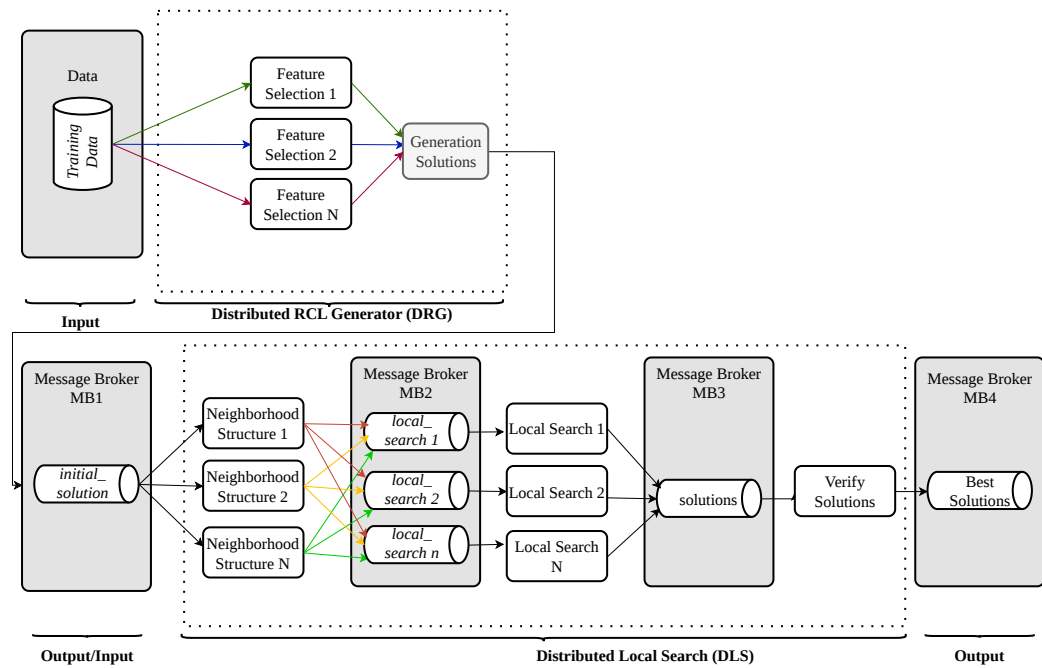


Figura 47 – Resumo visual da arquitetura completa do sistema.